

Capital University of Science and Technology Islamabad

LAB MANUAL

SOFTWARE REQUIREMENT ENGINEERING (SE2261)

DEPARTMENT OF SOFTWARE ENGINEERING

Lab Course Development Team

Supervision and Coordination

Mr Mudassar Adeel Ahmed

Senior Lecturer

Lab Designer

Tehreem Ahmed

Lab Instructor

Faculty of Computing

Lab Manual for Software Requirement Engineering

Lab-01

Introduction to SRE Tools

Lab 1: Introduction to SRE Tools

1 OBJECTIVE

- Understand the basics of Software Requirements Engineering (SRE).
- Install and set up **Visual Paradigm Community Edition** and **Click Up**.
- Gain familiarity with the tools to be used throughout the course.

4.1 Relevant Lecture Material

a) Textbooks:

Managing Software Requirements: A Use Case Approach, Second Edition by Dean Leffingwell, Don Widrig, Addison Wesley

b) Reference Material:

Requirements Engineering: From System Goals to UML Models to Software Specifications, Axel V. Lamsweerde, (2009) ISBN-10: 0470012706

2 ACTIVITY TIME BOXING:

ACTIVITY NAME	ACTIVITY TIME	TOTAL
Quiz	25 min	25 min
Setting Up and Setting	10 min	10 min
Walkthrough Task	15 min	60 min
Practice Task	12 to 15 min for each task	60 min

3 QUIZ

[Expected time 25 min]

4 CONCEPT MAP:

Requirements engineering is the process which enables us to systematically determine the requirements for a software product. Requirement: Something required, something wanted or needed. There is a huge difference between wanted and needed and it should be kept in mind all the time.

Requirements Engineering Process

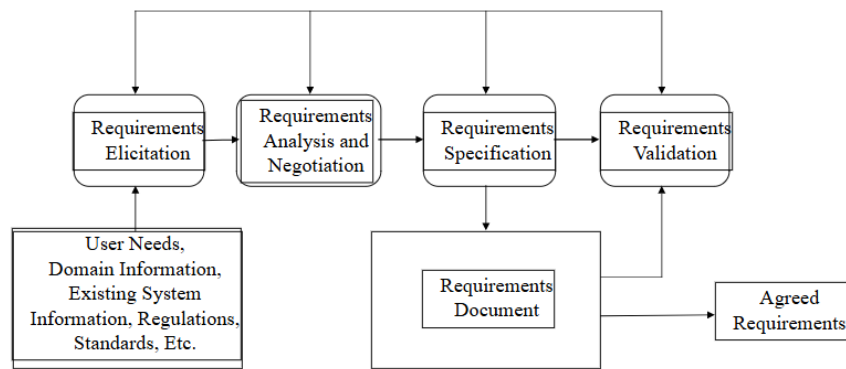


Figure 4-1 Requirement Engineering Process

Elicitation techniques such as Brainstorming- Scenarios, Observations and social analysis and Requirements reuse along with modeling methods like UML diagrams (Use Case, domain model system sequence diagram), which are created using Visual Paradigm Community Edition. Requirement Analysis includes Analysis checklists (A checklist is a list of questions which analysts may use to assess each requirement). Interaction matrices (Interaction matrices are used to discover interactions between requirements and to highlight conflicts and overlaps). On the other hand, Requirement Management focuses on documenting requirements through a Software Requirements Specification (SRS) document, which follows the IEEE 830 standard, and tracking requirement changes using project management tools like Click Up. Within the lab, students install and set up Visual Paradigm to create Use Case Diagrams, helping them understand system functionalities and actor interactions. Simultaneously, Click Up is introduced for task and milestone management, allowing students to create work packages, set priorities, and track progress. These tools together support requirement visualization, documentation, and change management, forming an essential foundation for effective Software Requirements Engineering practices.

Introduction to Software Requirements Engineering

Software Requirements Engineering (SRE) is the process of defining, documenting, and maintaining software requirements. It plays a crucial role in the software development lifecycle (SDLC) by ensuring that all stakeholders have a shared understanding of the system's needs.

Key Aspects of SRE:

- Requirement elicitation
- Requirement analysis & negotiation
- Requirement Specification
- Requirement validation

4.1.1 Requirement Elicitation

Requirement elicitation is the process of gathering user needs and expectations for a software system. It involves techniques like **interviews**, **brainstorming**, and **workshops** to collect accurate and complete requirements from stakeholders.

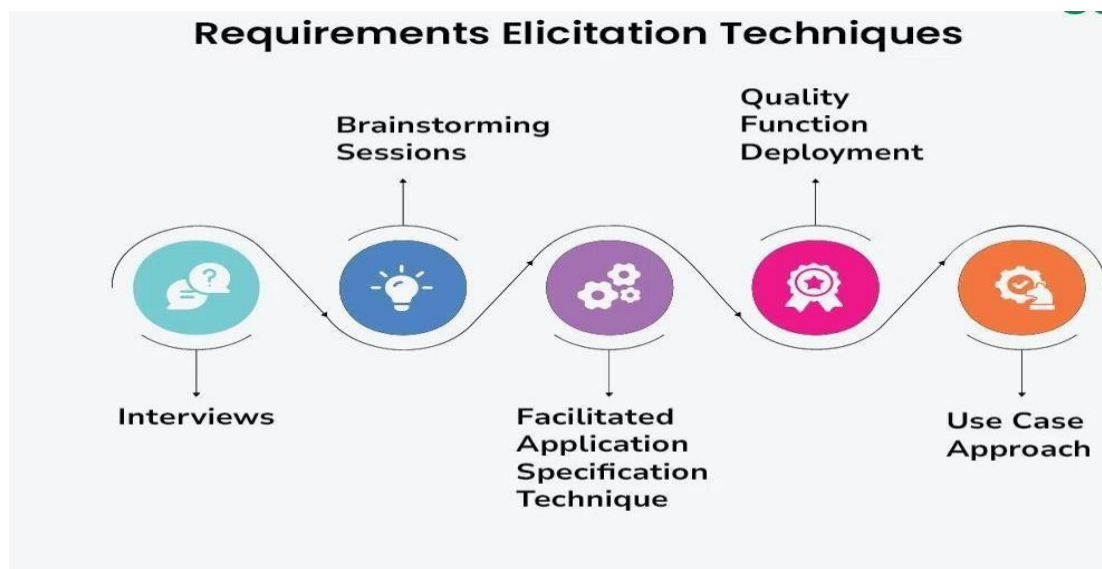


Figure 4-2 Requirement Elicitation Technique

4.1.2 Requirement Analysis

Requirement analysis involves examining, refining, and structuring the collected requirements to ensure they are clear, consistent, and feasible. It includes Analysis checklists (A checklist

is a list of questions which analysts may use to assess each requirement). Interaction matrices (Interaction matrices are used to discover interactions between requirements and to highlight conflicts and overlaps).

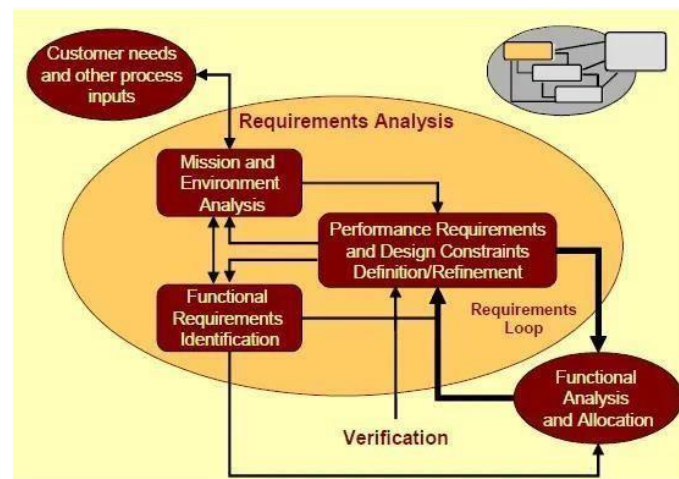


Figure 4-3 Requirement Analysis

4.1.3 Requirement Specification

Requirement documentation involves recording the gathered and analysed requirements in a structured format, typically in a **Software Requirements Specification (SRS)** document. This document serves as a reference for developers, testers, and stakeholders throughout the software development lifecycle.



Figure 4-4 Requirement Documentation

4.1.4 Requirement Validation

Requirement validation ensures that the documented requirements accurately reflect the stakeholders' needs, while requirement verification checks that the requirements are correctly implemented in the software. Techniques include **reviews, prototyping, test case generation, and formal inspections.**

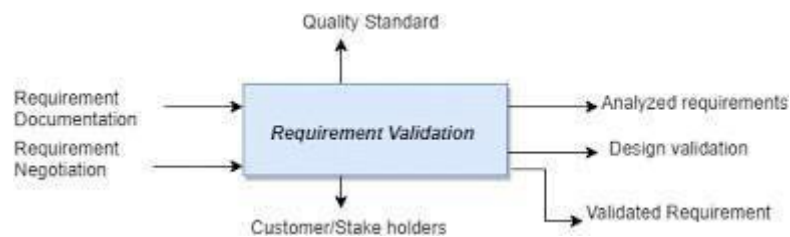


Figure 4-5 Requirement Validation

In this lab, we will explore **Visual Paradigm Community Edition** and **Click Up** which are commonly used tools for software requirement analysis, modelling, and management.

4.2 Installation and Setup

3.2.1 Installing Visual Paradigm Community Edition

Visual Paradigm is a UML modelling and diagramming tool used for requirement analysis and system design.

Steps to Install:

1. Download **Visual Paradigm Community Edition** from the official website:
<https://www.visual-paradigm.com/download/community.jsp>
 2. Execute the downloaded Visual Paradigm installer file. The setup wizard appears as below.
-

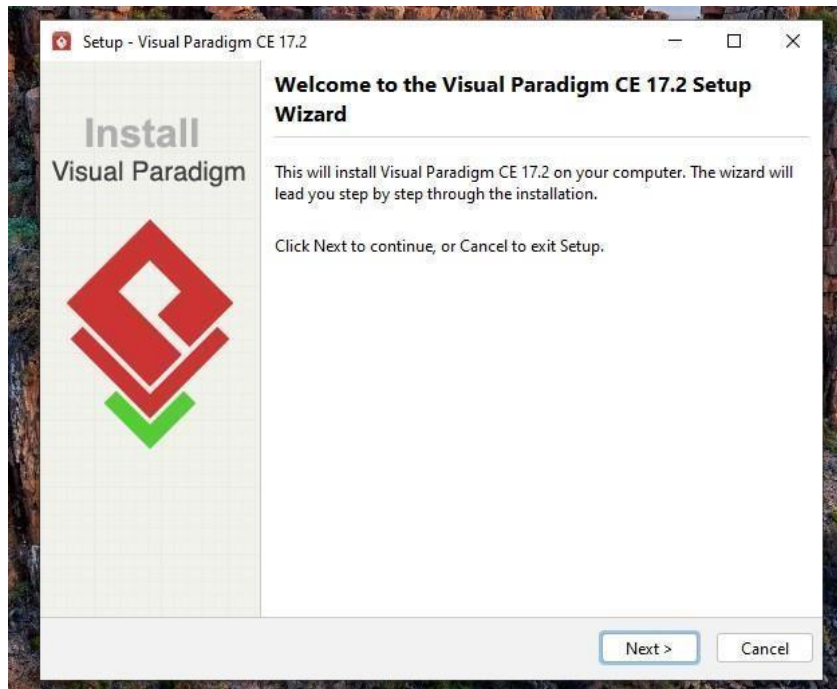


Figure 4-6 Visual Paradigm setup wizard

3. Click Next to proceed to the License Agreement page.
4. Read through the license agreement. If you fully understand and agree with them, choose I accept the agreement to accept the terms. Click Next.

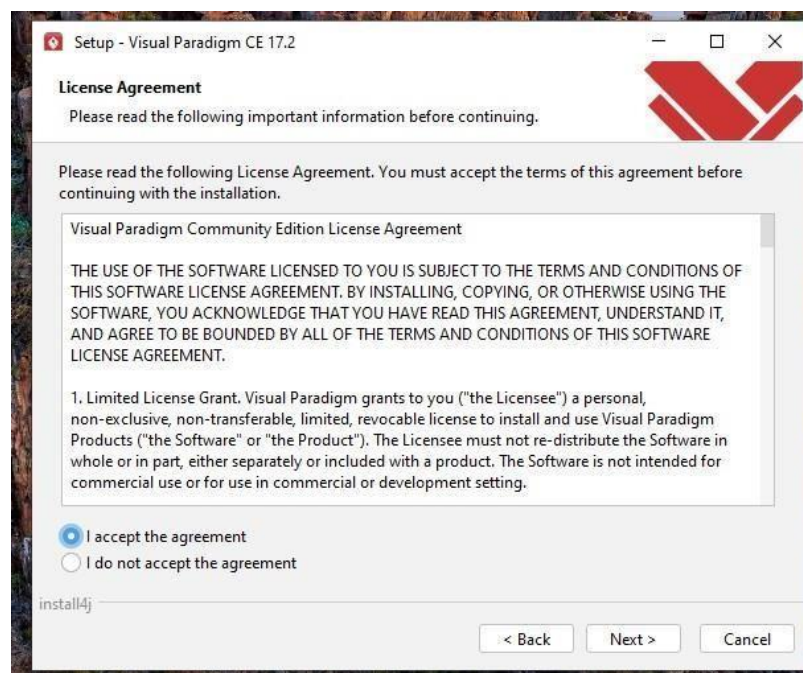


Figure 4-7 license Agreement page

5. Specify the directory for installing Visual Paradigm. Click Next to continue.
-

- Specify the name of the Start Menu folder that will be used to store the shortcuts. Keep Create shortcuts for all users checked if you want the shortcut(s) to be available in all the user accounts in the machine. Click Next to continue.

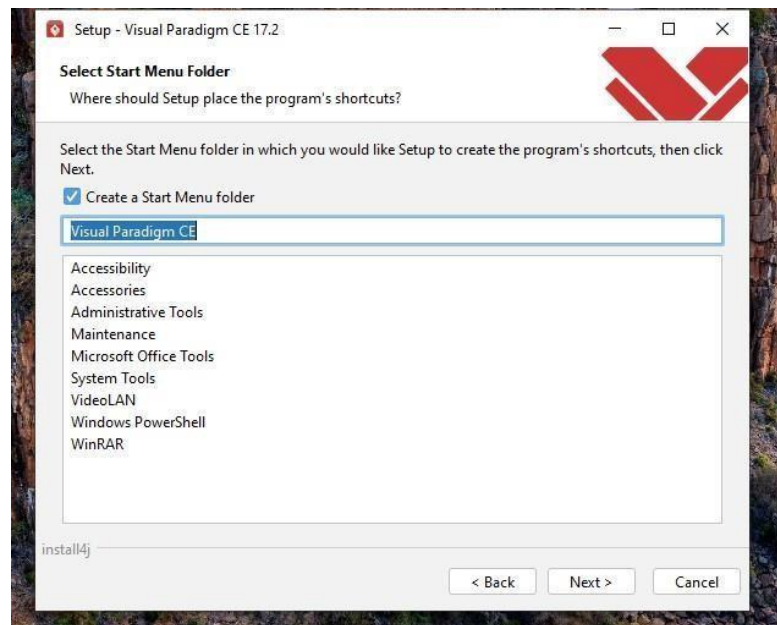


Figure 4-8 Start Menu Folder

- In the Select File Associations page, keep Visual Paradigm Project (*.vpp) checked if you want your system to be able to open the project file upon direct execution (i.e. double click). Click Next to start the file copying process

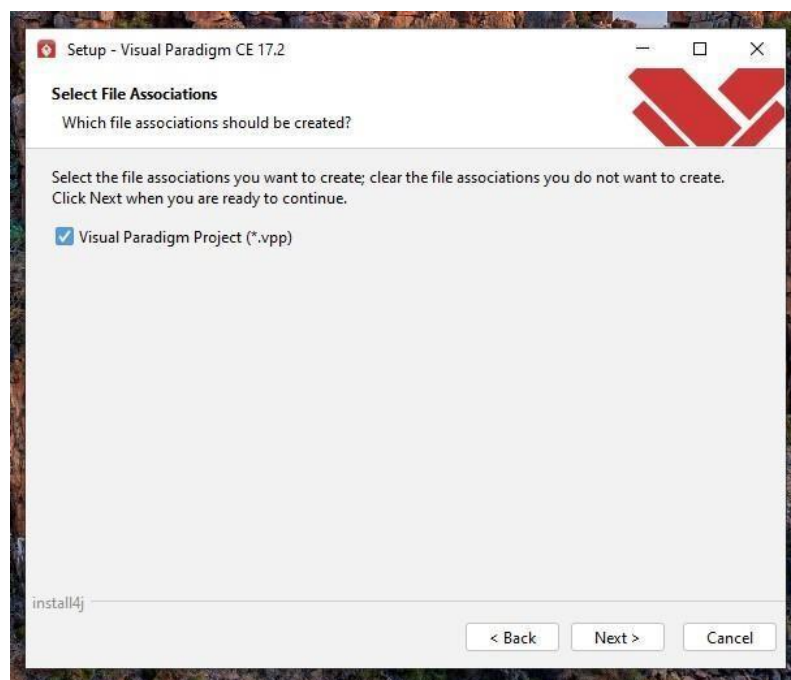


Figure 4-9 select file Association

- Once the file copying is finished, you can choose to start Visual Paradigm immediately. On the other hand, you can finish the installation without starting Visual Paradigm.

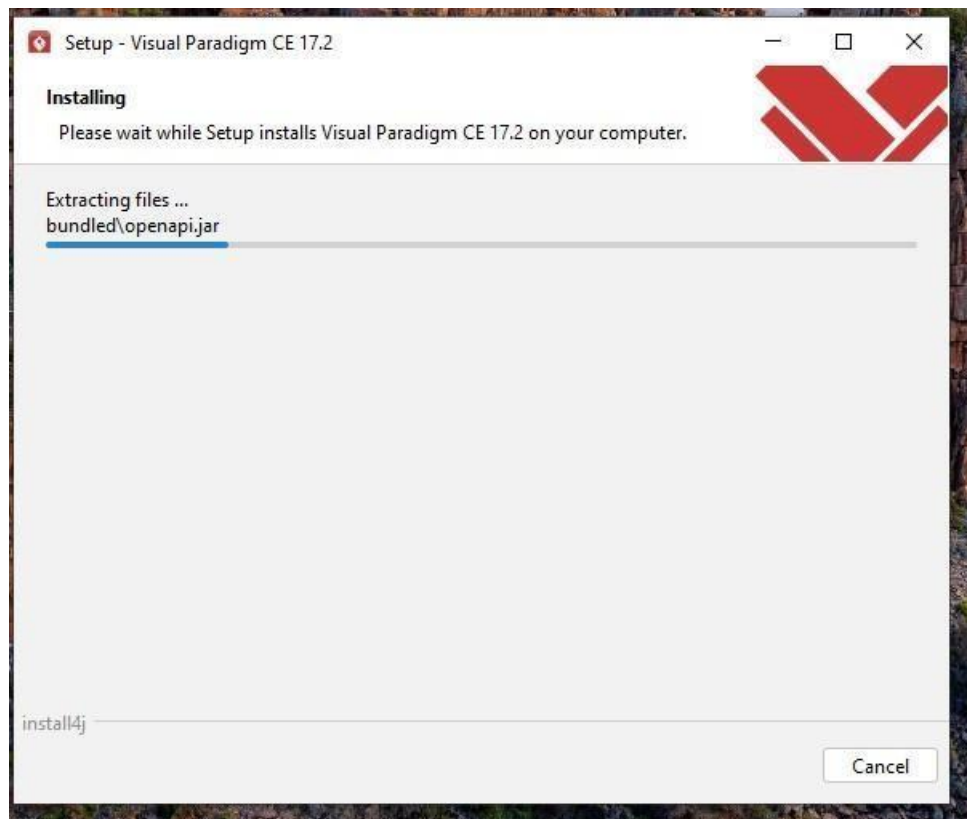


Figure 4-10 Installation in progress

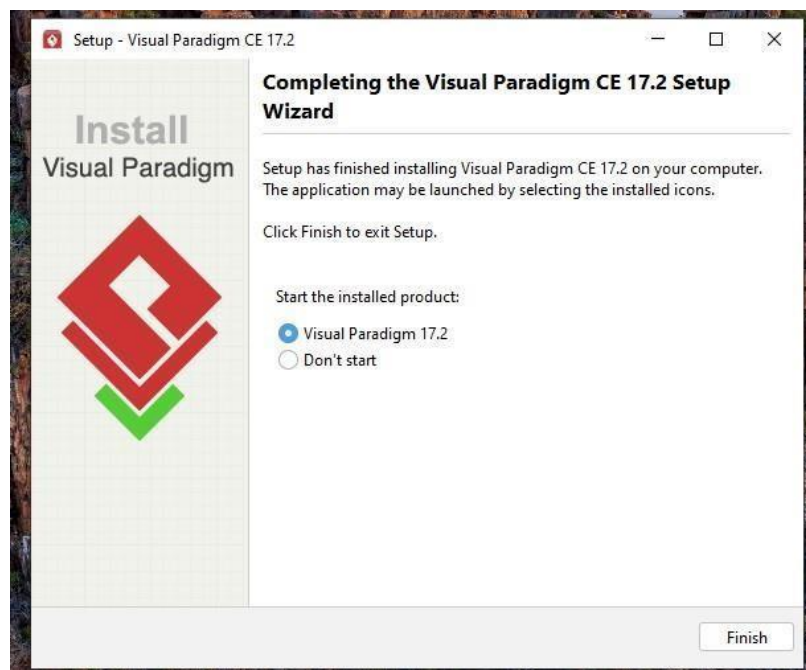


Figure 4-11 Visual Paradigm installation complete

9. Choose the option Don't Start and click install visual paradigm. This leads to the installation of Visual Paradigm.



Figure 4-12 Welcome to Visual Paradigm Community

10. Once the application is launched, you will be greeted by the homepage, which serves as the central hub for all your design and modelling activities. The homepage is designed to provide a user-friendly interface that allows you to easily navigate through the various features and functionalities of Visual Paradigm

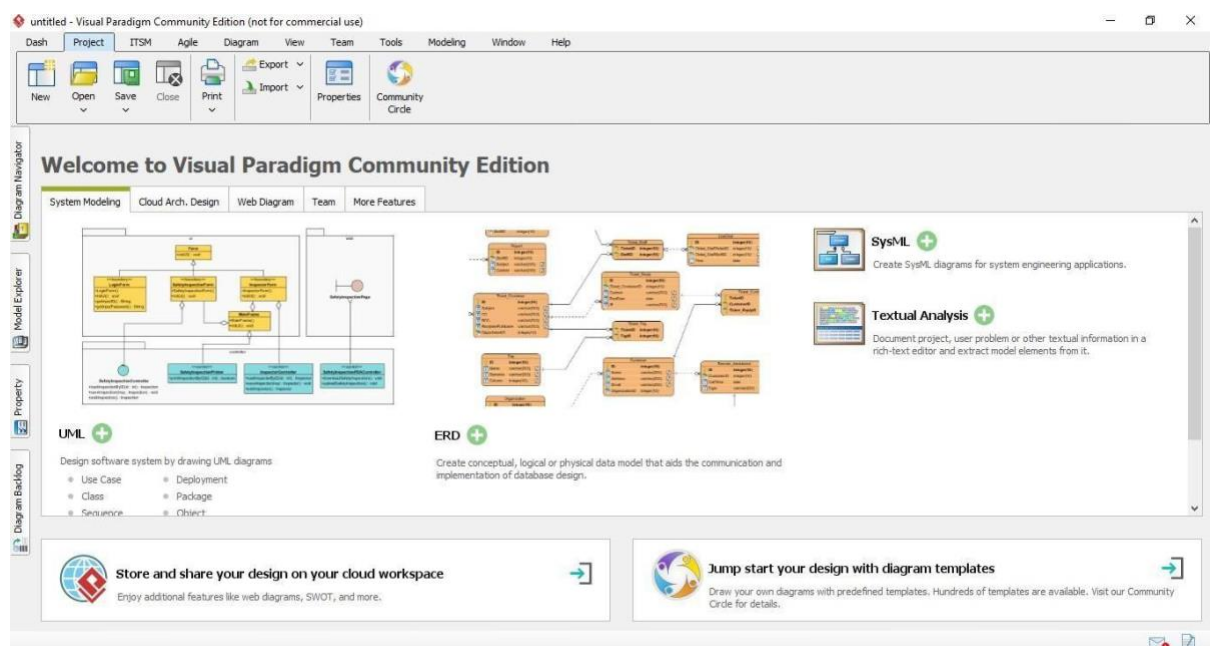


Figure 4-13 Homepage

3.2.2 Installing and Setting Up Click Up:

1. Download the Installer: Go to the Click Up website and download the Windows installer.



Figure 4-14 Download the Click Up

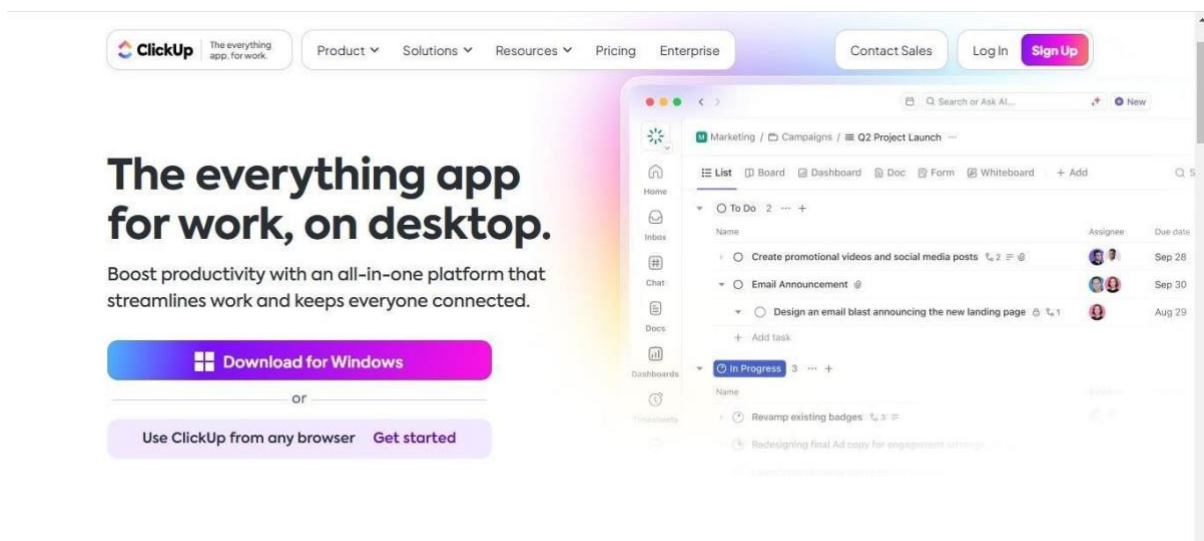


Figure 4-15 Click on the Download for windows button

2. Run the Installer: Double-click the downloaded installer file to start the installation wizard. Upon launching the app for the first time, you will be greeted with a welcome screen that provides options to either log in or sign up.

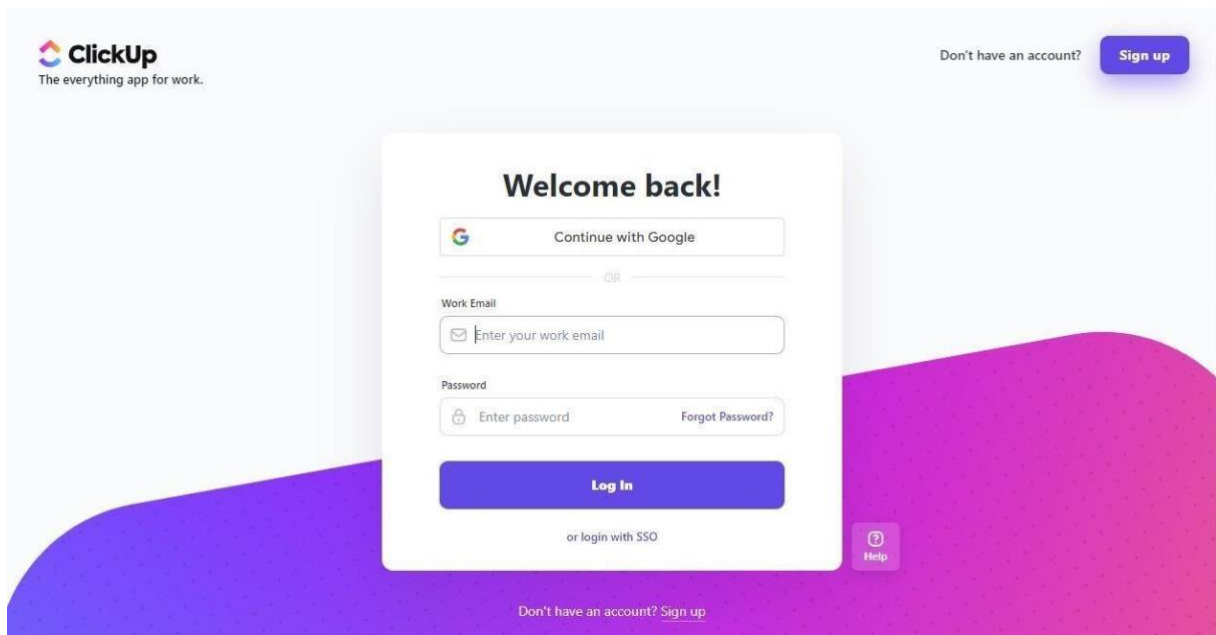


Figure 4-16 Welcome Screen

3. Click on the **"Sign Up"** button to begin the registration process.

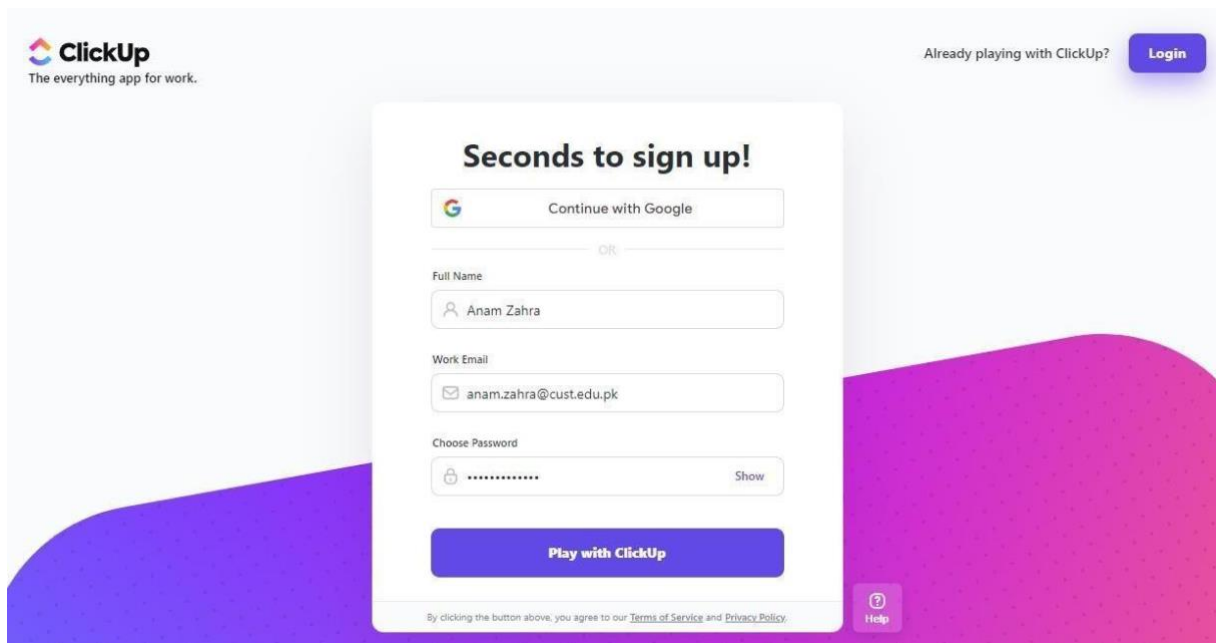


Figure 4-17 Sign-Up Page

1. Once your email is verified, return to the Click Up app. You may be prompted to complete your profile by providing additional information, such as your name and company name (if applicable).
2. Fill in the required fields and click "**Continue**" or "**Next**" to proceed.

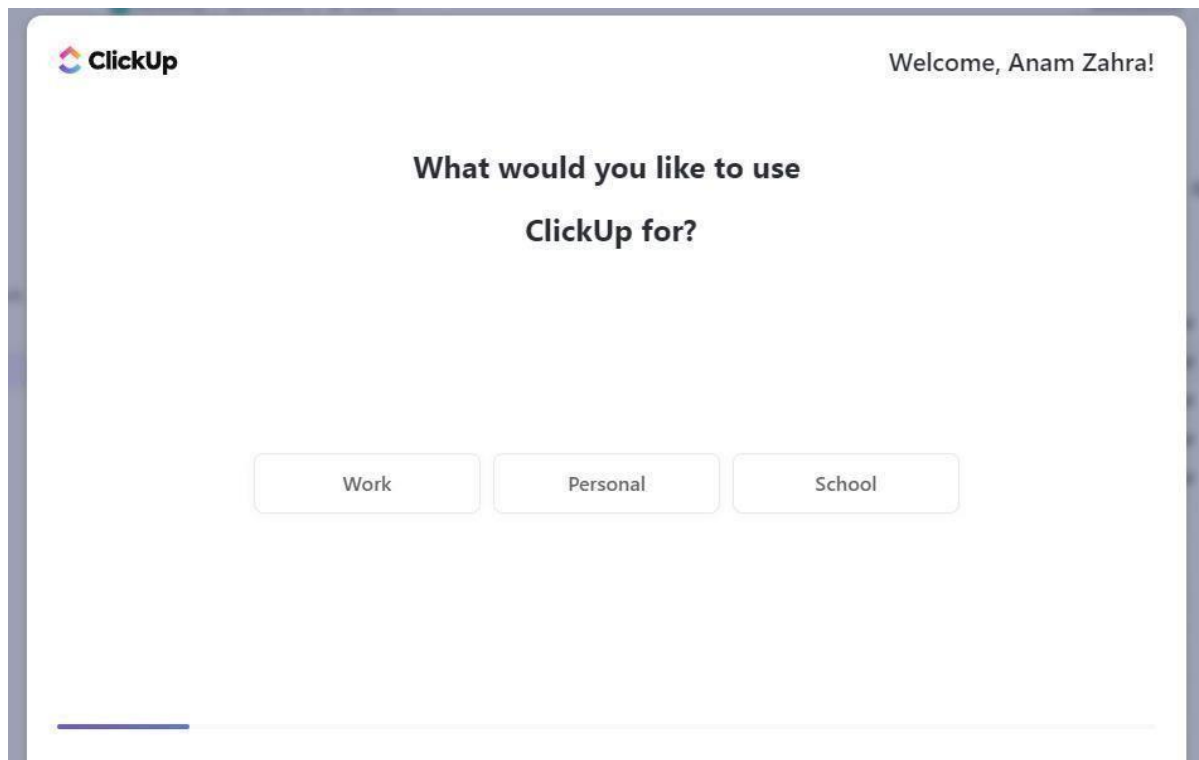


Figure 4-18 Select option to use Click Up for



What would you like to manage?

Startup

Marketing

Creative & Design

Support

HR & Recruiting

Professional Services

Sales & CRM

Software Development

PMO

IT

Personal Use

Finance & Accounting

Operations

Other

Don't worry, you can always add more in the future.

Figure 4-19 What would you like to manage



How did you hear about us?

LinkedIn

Reddit

Software Review Sites

Search Engine (Google, Bing, etc.)

YouTube

Billboard

TikTok

Friend / Colleague

TV / Streaming (Hulu, NBC, etc.)

Facebook / Instagram

Other

< Back

Next >

Figure 4-20 How did you hear about

The screenshot shows a ClickUp account setup screen. At the top left is the ClickUp logo. Below it, the word "Optional" is centered. The main heading reads "Enter your phone number if you'd like help with account setup:". Below this is a form with a country dropdown menu showing the United States flag and a text input field containing "(201) 555-0123". A progress bar is located below the form. At the bottom, there are "Back" and "Skip" buttons.

ClickUp

Optional

Enter your phone number if you'd like help with account setup:

(201) 555-0123

Our team might reach out to help you get started. We won't share your phone number, ever. By entering your phone number, you consent to receiving calls and messages from us for informational and promotional purposes. You may opt out of these at any time.

Back Skip

Figure 4-21 Additional option for account setup

The screenshot shows a ClickUp account setup screen. At the top left is the ClickUp logo. Below it, the heading reads "Do you use any of these tools?". Below this is a grid of 18 tool icons, each with a checkbox. The tools are: Slack, Zoom, MS Teams, Dropbox, Monday, Todoist, GitHub, Figma, Confluence, Asana, Notion, Trello, Wrike, Jira, Basecamp, Excel & CSV, G Drive, and Salesforce. A progress bar is located below the grid. At the bottom, there are "Back" and "Next" buttons.

ClickUp

Do you use any of these tools?

Slack Zoom MS Teams Dropbox Monday Todoist

GitHub Figma Confluence Asana Notion Trello

Wrike Jira Basecamp Excel & CSV G Drive Salesforce

Bring all of your work into one place.

Back Next

Figure 4-22 additional tools



Which features are you interested in trying?

Goals & OKRs ☐	Boards & Kanban ☒	Forms ☒	Sprints ☐	Dashboards ☒	Automations ☒	
Clips ☐	Tasks & Projects ☒	Gantt Charts ☒	Calendar ☒	Workload ☐	Ask AI ☒	CRM ☐
Whiteboards ☐	Docs & Wikis ☐	Scheduling ☒	Time Tracking ☒	Chat ☐		

Don't worry, you'll have access to all of these in your Workspace.

< BackNext >

Figure 4-23 Additional Features



Lastly, what would you like to name your Workspace?

Capital University of Science and Te

Press enter to continue

By completing this form, you agree to our [Terms of Service](#) and [Privacy Policy](#).

< BackFinish

Figure 4-24 Name your Workplace

1. Once your workspace is set up, you will be taken to the Click Up dashboard. Here, you can start creating tasks, projects, and collaborating with your team.
2. Take a moment to explore the features and tools available in Click Up to familiarize yourself with the platform

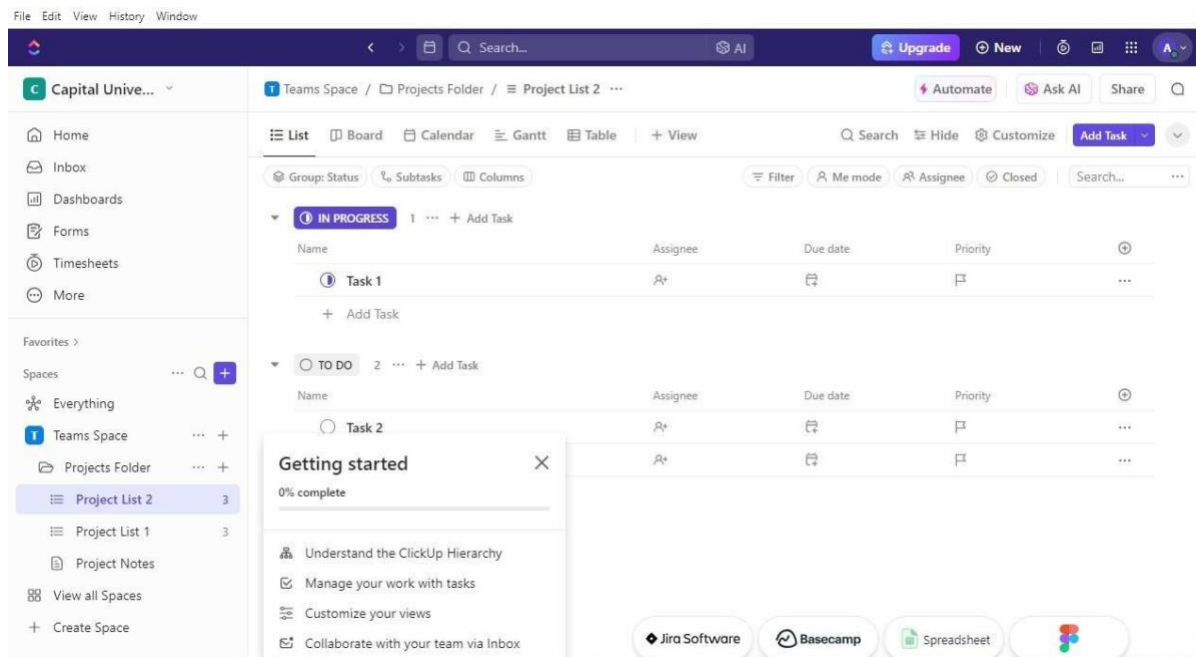


Figure 4-25 Click Up Dashboard

WALK THROUGH TASK

This section is designed such a way that you can complete the following tasks independently. However, if there is any ambiguity you can refer it to the lab instructor.

5.1 Walkthrough task 01 [expected time 15 min]

What are the steps to download and install Visual Paradigm Community Edition on your operating system and run the installer and complete the setup for Visual Paradigm Community Edition?

5.2 Walkthrough task 02 [expected time 15 min]

What are the steps to download and install Click Up on your operating system and run the installer and complete the setup for Click Up

5.3 Walkthrough task 03 [expected time 15 min]

What are the key features of Visual Paradigm that support software modelling and design? What are the main features of Click Up that enhance task and project management?

6. PRACTICE TASK:

This section will provide practice exercises which you need to finish during the lab. You need

to finish the tasks in the required time. When you finish them, put these tasks into your lab designated Folder announced by lab instructor

6.1 Practice task 01 [expected time 15 min]

How do you open Visual Paradigm and initiate the process to create a new project?

What steps are involved in setting up a new project in Visual Paradigm after launching the application?

6.2 Practice task 02 [expected time 15 min]

How do you open Click Up and start the process of setting up a simple project?

What steps are required to configure and organize a basic project in Click Up?

6.3 Practice task 03 [expected time 15 min]

What are the key menus and tools available in Visual Paradigm for project management and design?

How can you navigate the menus, tools, and settings in Click Up to optimize your project management experience?

7. EVALUATION CRITERIA:

The evaluation criteria for this lab will be based on the completion of the following tasks. Each task is assigned the marks percentage which will be evaluated by the instructor in the lab whether the student has finished the complete/partial task(s).

S.NO	Task	Description	Marks
1	3	Quiz	
2	4.1 to 4.3	Walkthrough	
3	5.1 to 5.3	Practice task	

Lab Manual for Software Requirement Engineering

Lab-02

Types of Software Requirements

Lab 2: Types of Software Requirements Objective

The objective of this lab is to equip students with the skills to identify and distinguish between functional and non-functional requirements in software development. By utilizing Visual Paradigm, students will learn how to document these requirements effectively for a sample project. This understanding is crucial for ensuring that software meets both user needs and quality standards.

Relevant Lecture Material

a) Textbooks:

Managing Software Requirements: A Use Case Approach, Second Edition by Dean Leffingwell, Don Widrig, Addison Wesley

b) Reference Material:

Requirements Engineering: From System Goals to UML Models to Software Specifications, Axel V. Lamsweerde, (2009) ISBN-10: 0470012706

Activity Time Boxing:

ACTIVITY NAME	ACTIVITY TIME	TOTAL
Quiz	25 min	25 min
Setting Up and Setting	5min	5 min
Walkthrough Task	20 min	40 min
Practice Task	20 to 30 min for each task	60 min

Quiz [Expected time 25 min]

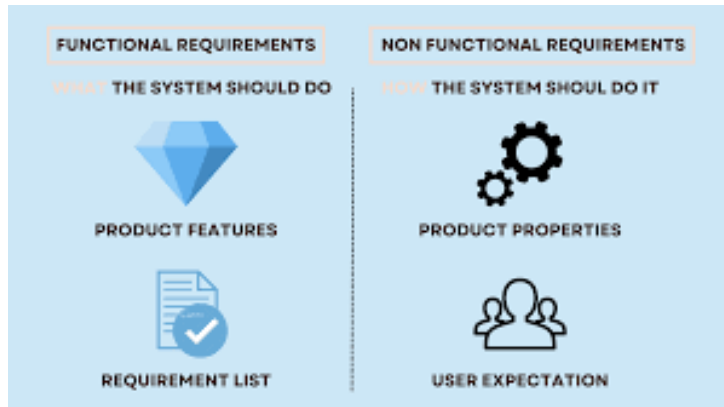
Concept Map:

Software requirements are essential for guiding the development process and ensuring that the final product meets the expectations of stakeholders. Functional requirements define what the system should do, detailing specific behaviors and functions. Examples include user authentication, data processing, and reporting features. Non-functional requirements, on the other hand, specify how the system performs its functions, focusing on quality attributes such as performance, security, and usability. Understanding the distinction between these two types of requirements is vital for successful software engineering.

Software requirements:

The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product. The requirements can be obvious or hidden, known or unknown, expected or unexpected from client's point of view. A software requirement is a capability or condition that must be met in order for software

to solve a real-world problem. More formally, a software requirement can be defined as a feature or non-functional constraint that the software must provide to fulfil the needs of users and other stakeholders. Requirements represent the core functionality that software must deliver to be useful and meet business process or user goals.



TYPES OF REQUIREMENTS:

Functional Requirements:

Functional requirements are important statements that outline what a system is supposed to do, including its main features and services. They explain how the system should react to different inputs and behave in various situations, making sure all interactions are covered. These requirements also include how tasks are organized for efficiency and how the system handles unexpected issues. It's crucial for these requirements to be thorough and consistent, as they guide the system's development. Both customers and developers pay close attention to these requirements because they greatly affect user experience and the system's overall performance.

Example:

The functional requirements for the Online Food Delivery System include:

1. User Registration: Users must be able to create an account by providing their email, password, and personal information.
2. Menu Browsing: Users should be able to view the menu items offered by various restaurants, including descriptions and prices.
3. Order Placement: Users must be able to select items, customize their orders, and add them to a shopping cart.
4. Payment Processing: The system should support multiple payment methods, including credit cards and digital wallets.
5. Order Tracking: Users should be able to track the status of their orders in real time.

Non-Functional Requirements:

Non-functional requirements focus on the overall system rather than specific functions. They include important factors like timing, performance, reliability, security, maintainability, and

accuracy. These requirements are often more crucial than individual functional ones because they describe how the system behaves as a whole. By addressing these aspects, non-functional requirements ensure that the system works effectively and reliably, leading to a better user experience.

Example:

The non-functional requirements for the system include:

1. **Performance:** The system should handle up to 500 concurrent users with a response time of less than 2 seconds for menu browsing.
 2. **Security:** User data must be encrypted, and the system should comply with data protection regulations (e.g., GDPR).
 3. **Usability:** The user interface should be intuitive and accessible, ensuring that users can navigate the application easily.
 4. **Availability:** The system should have an uptime of 99.9% to ensure that users can access it at any time.
 5. **Scalability:** The architecture should support future growth, allowing for the addition of new features and increased user load.
-

Walk Through Task

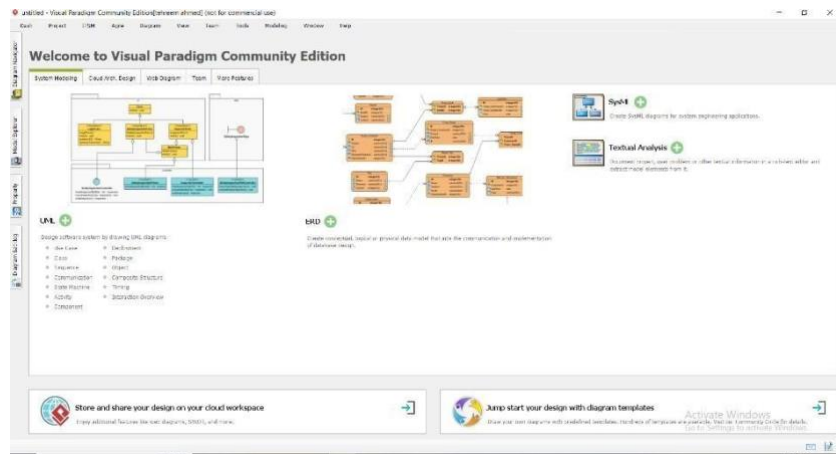
Walkthrough task: E-commerce Website Case Study
min]

[expected time 30

XYZ Electronics is a growing online retailer specializing in electronic products. The company aims to provide a seamless online shopping experience to its customers. The website allows users to browse products, add them to a shopping cart, and complete secure transactions. Users can also leave reviews and track their orders in real time. To ensure scalability, the system must support thousands of users simultaneously, while maintaining quick page load times and secure payment processing. Additionally, the platform should be mobile-friendly and support various payment methods, including credit cards, digital wallets, and bank transfers. The website must also include features such as personalized recommendations and a chatbot for customer support.

Identify key functional and non-functional requirements.

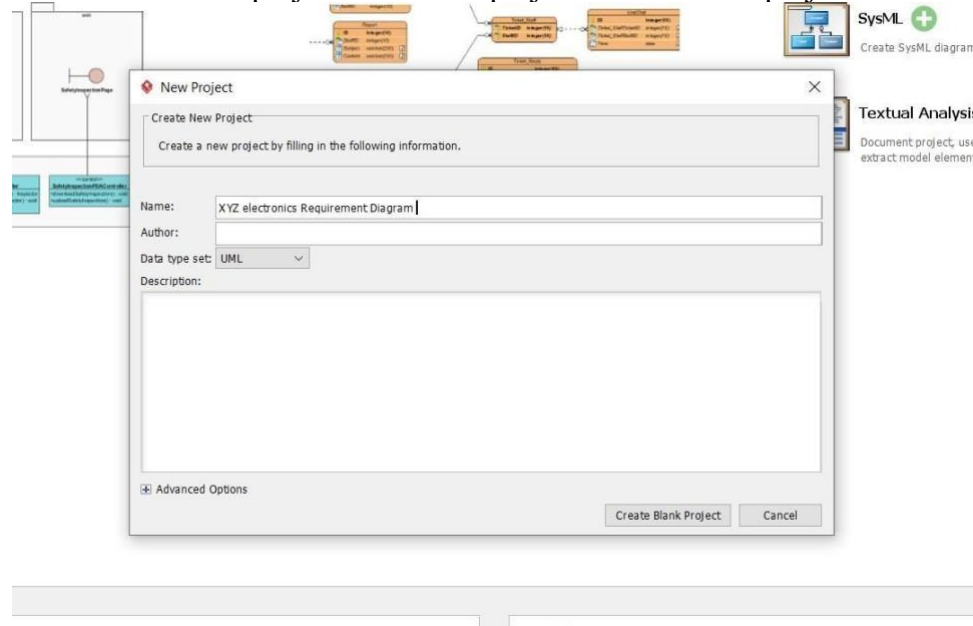
Functional Requirements:



1. Open Visual Paradigm:

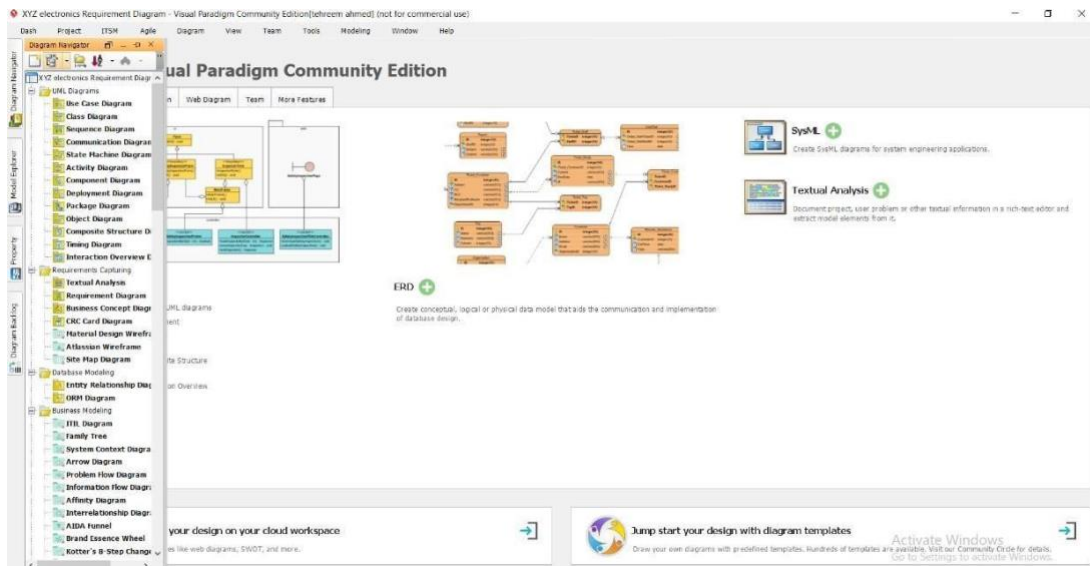
Open the visual paradigm.

Click on the project-> new project and the project creation dialog box appears

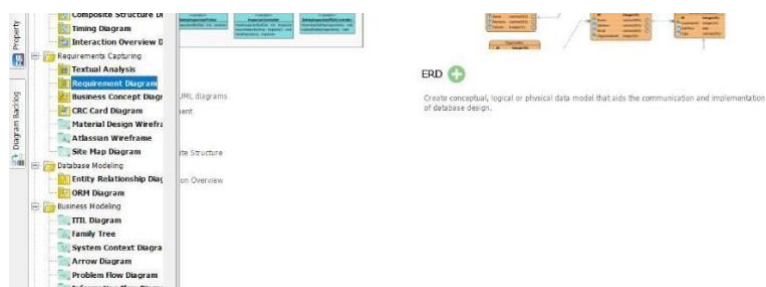


Write the project name and author name in required fields.

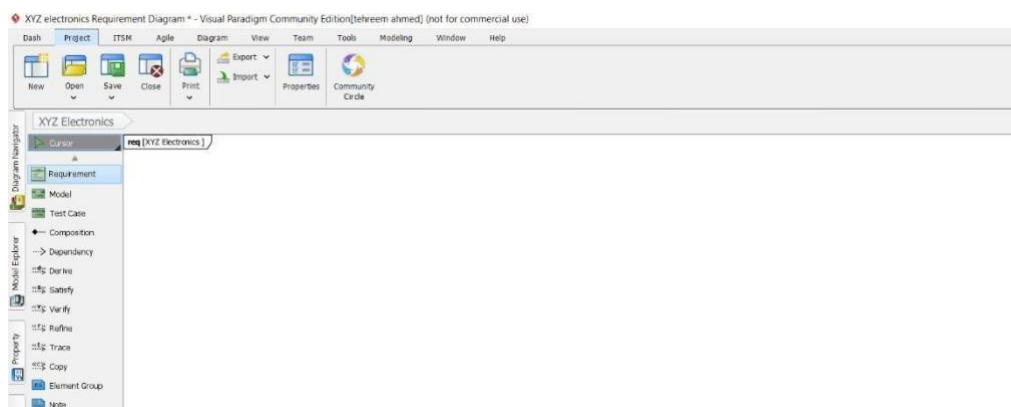
Click on the create blank project option and new project is created successfully.



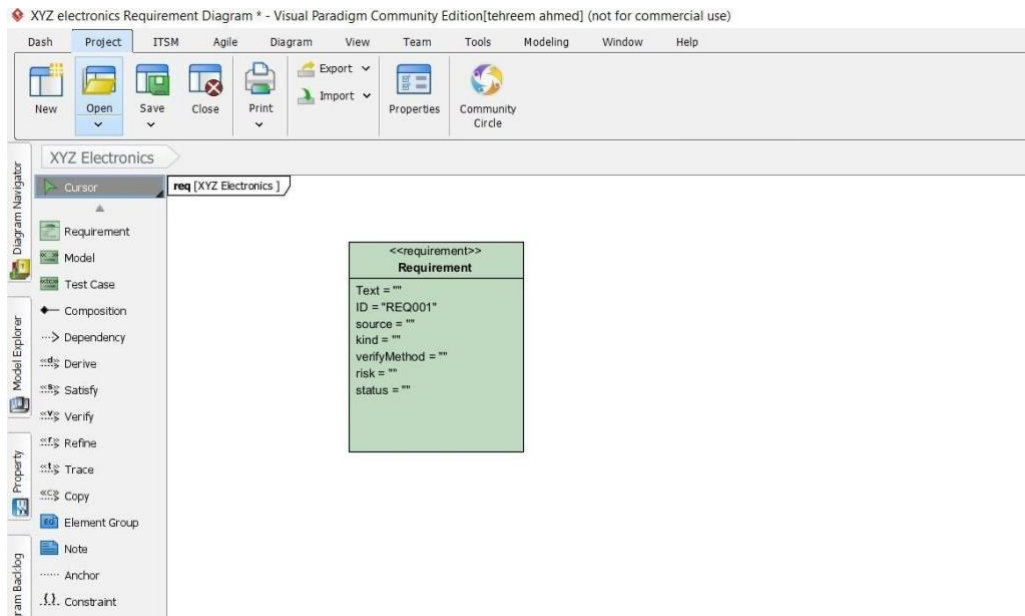
Create a new **Requirement Diagram** from diagram Navigator



Requirement diagram project is created

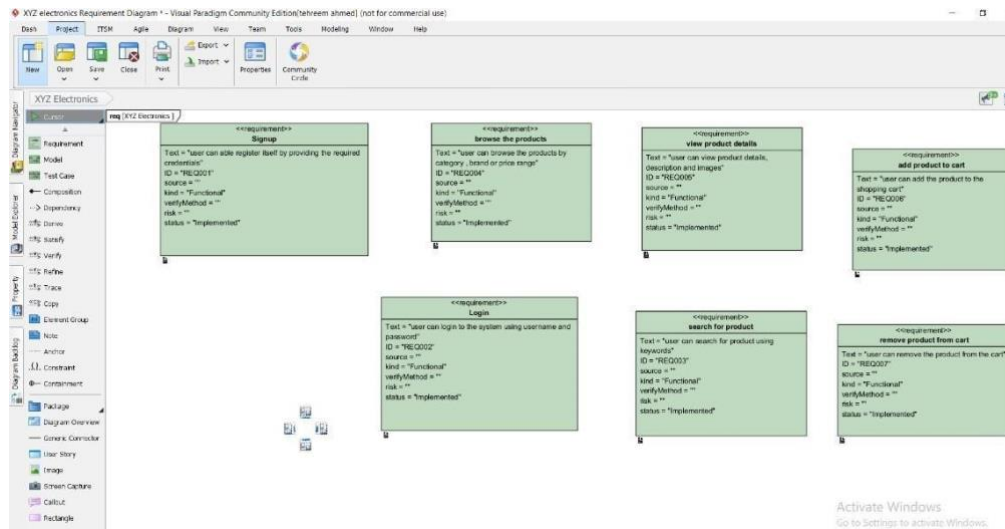


Drag and drop requirements onto the diagram Name
one requirement

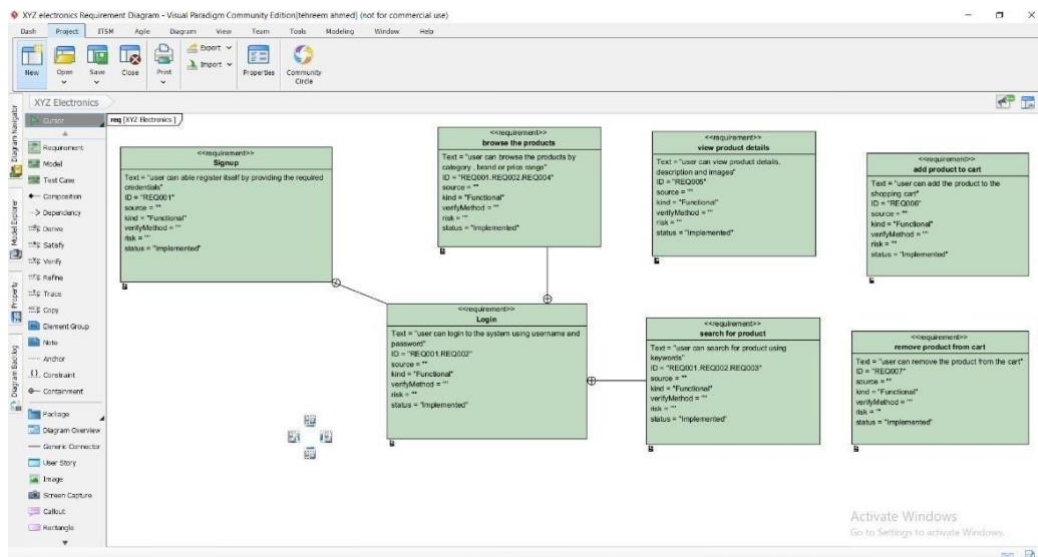


Right click on anywhere and select the option Tabular View where you can write requirements more easily and after that you can switch on the diagram view.

You can switch to the diagram view by right click on the page and select diagram view option.



Draw a dependency arrow from the Login to the Signup Requirement. This indicates that the login includes the Signup Requirement.



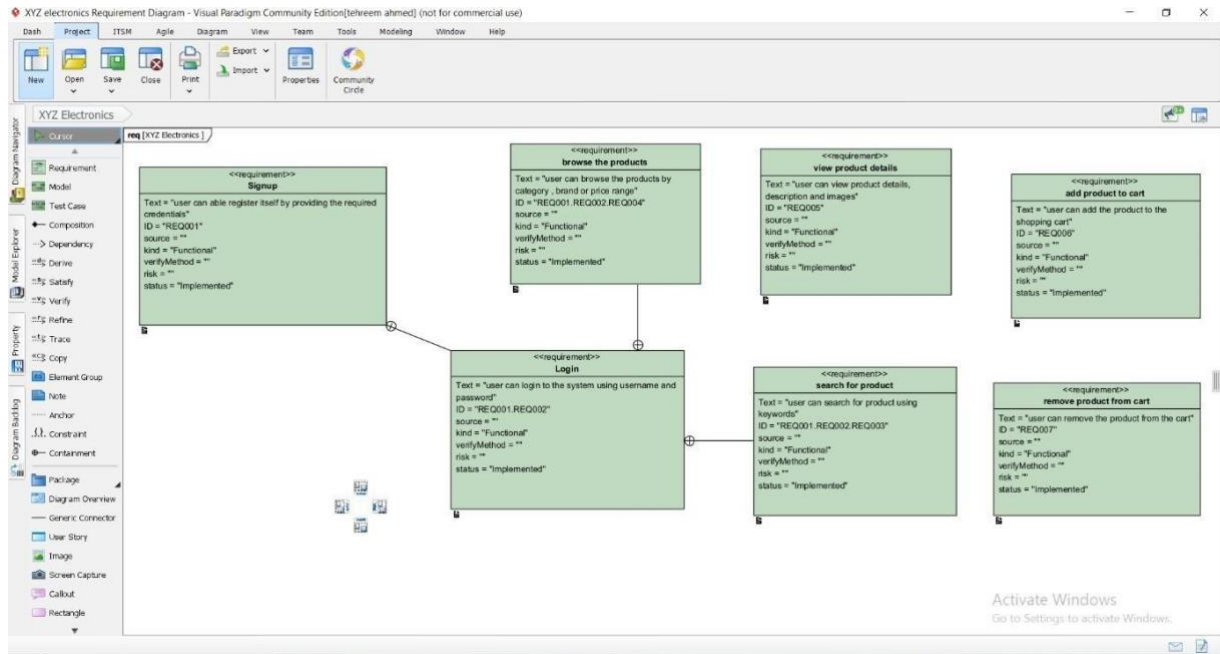
Double-click on the dependency arrow to open its properties.

In the Stereotype field, add a custom stereotype like <<includes>>.

This makes it clear that the relationship is an "includes" relationship.

Finalize the Diagram:

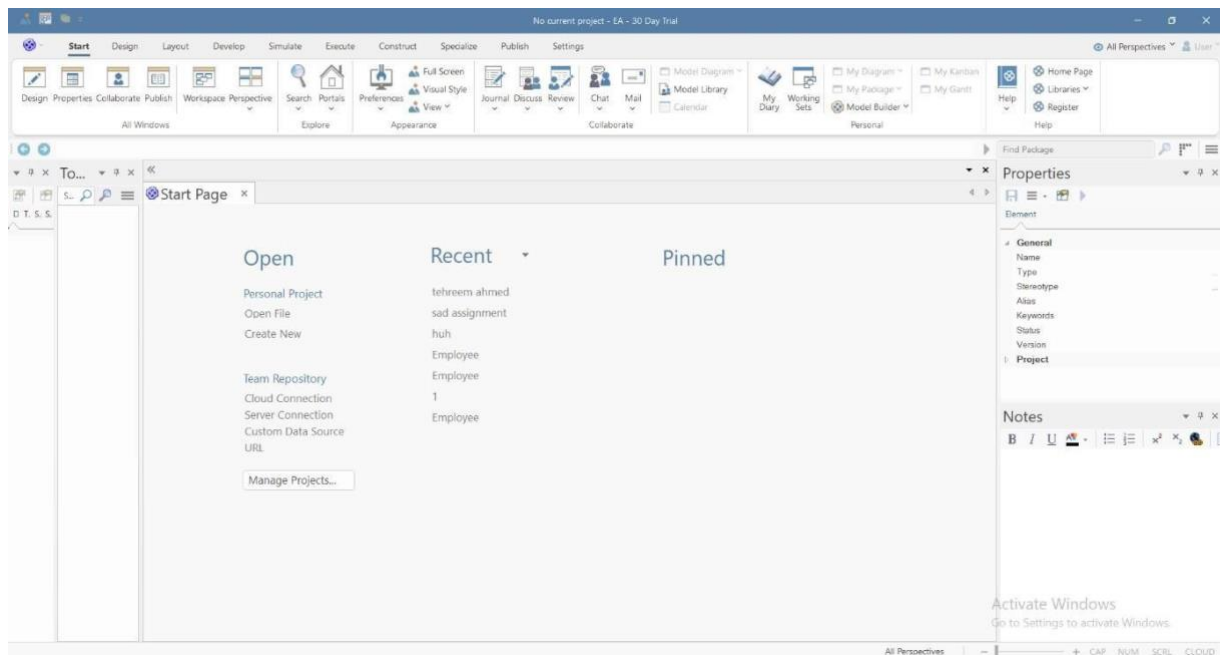
Save the final diagram.



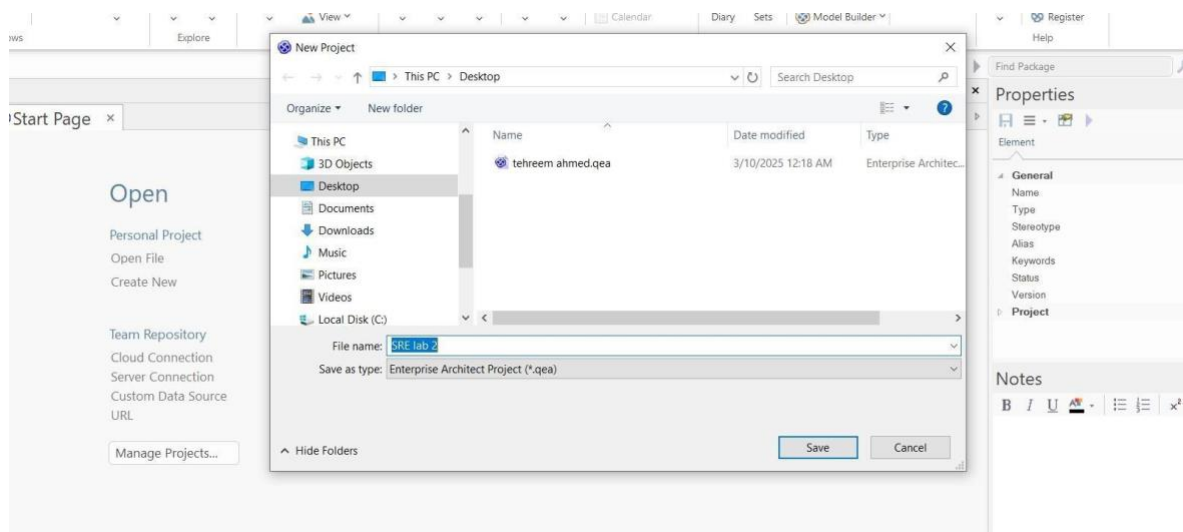
Enterprise Architect Tool :

Step 1: Open Enterprise Architect

Launch Enterprise Architect.



Open an existing project or create a new one.



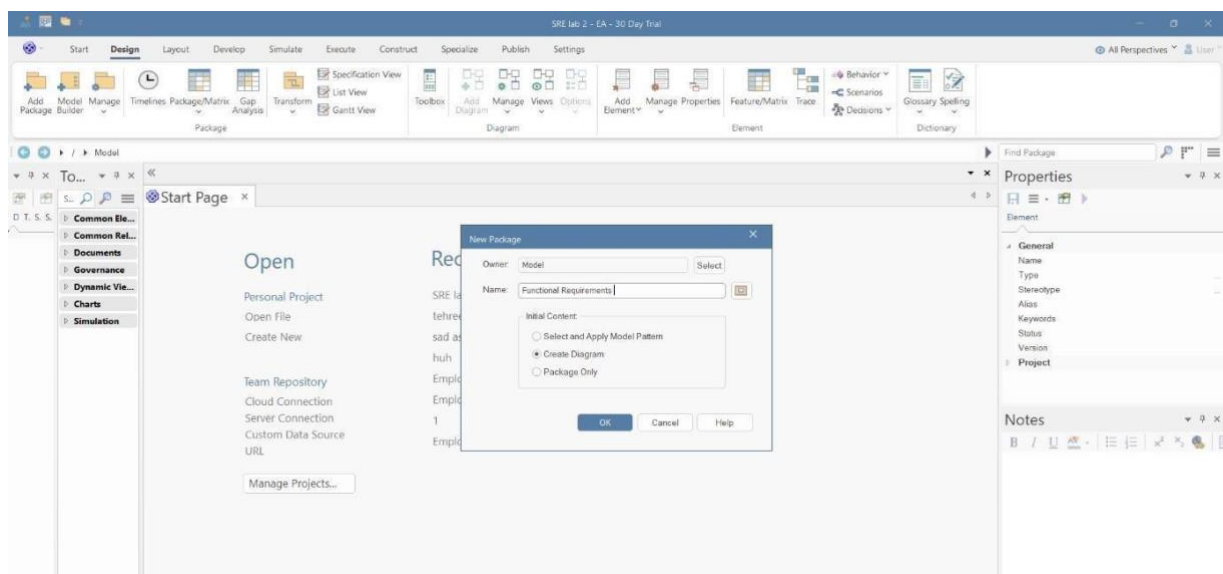
Step 2: Create a Requirements Package

In the **Project Browser** (left panel), right-click on the root node or an appropriate package.

Select **Add > Add a Package**.

Name the package (e.g., "Functional Requirements") and set its type to **Requirements**.

Click **OK**.



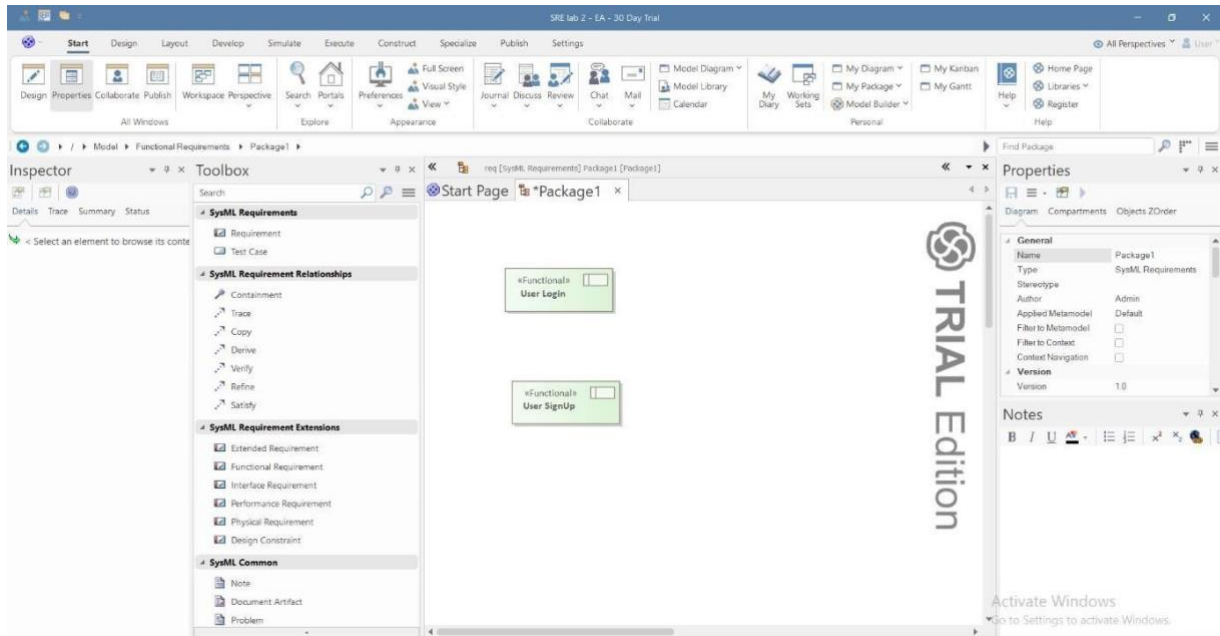
Step 3: Add Functional Requirements

Open the newly created **Requirements Package**.

Right-click inside the package and select **Add > Add Element**.

In the **New Element** dialog:

- Set the **Type** to **Requirement**.
- Provide a **Name** for the requirement (e.g., "User Login").
- Click **OK**.



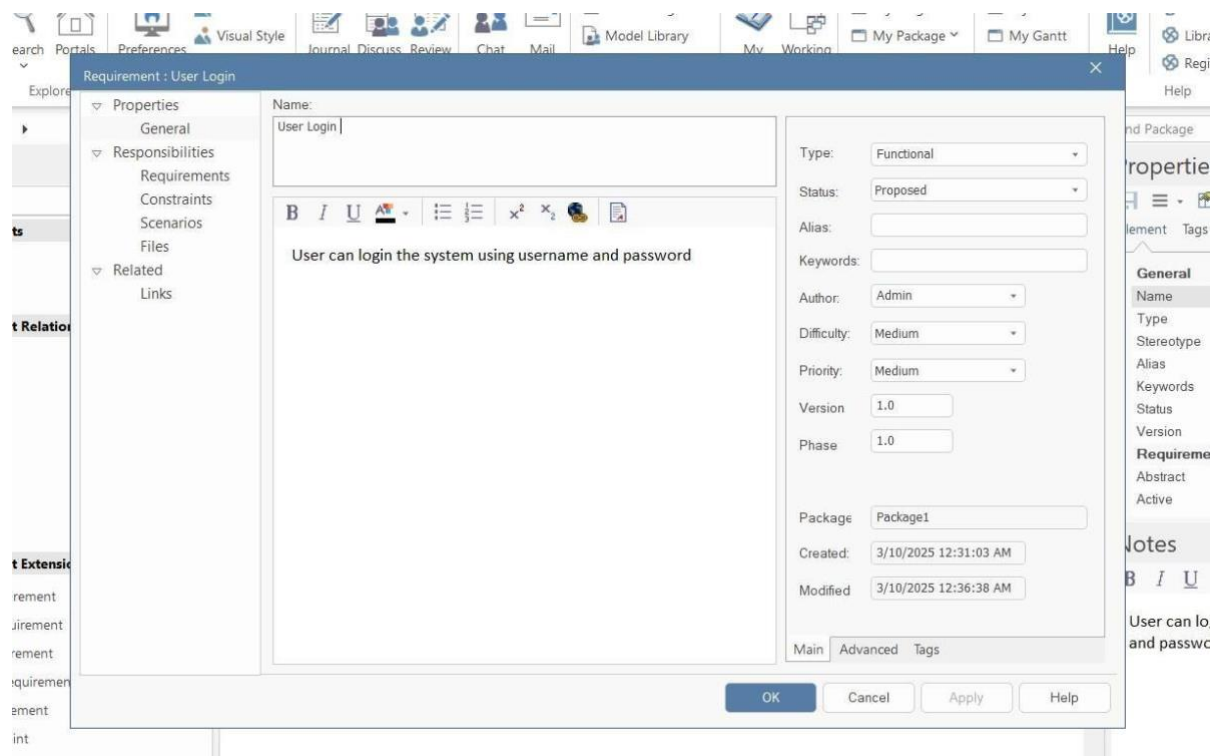
Step 4: Define the Requirement Details

Double-click on the newly created requirement element to open its **Properties** window.

Fill in the following fields:

- Name:** A short, descriptive name for the requirement (e.g., "User Login").
- Stereotype:** Optionally, set a stereotype (e.g., <<Functional>>).
- Notes:** Provide a detailed description of the requirement (e.g., "The system shall allow users to log in using their username and password").
- Priority:** Set the priority (e.g., High, Medium, Low).
- Status:** Set the status (e.g., Proposed, Approved, Implemented).
- Difficulty:** Optionally, set the difficulty level (e.g., Low, Medium, High).
- Version:** Assign a version number if needed.

Click **OK** to save the details.



req [SysML Requirements] Package1 [Package1]

Start Page *Package1

Toolbox

- SysML Requirements
 - Requirement
 - Test Case
- SysML Requirement Relationships
 - Containment
 - Trace
 - Copy
 - Derive
 - Verify
 - Refine
 - Satisfy
- SysML Requirement Extensions
 - Extended Requirement
 - Functional Requirement
 - Interface Requirement
 - Performance Requirement
 - Physical Requirement
 - Design Constraint
- SysML Common
 - Note
 - Document Artifact
 - Problem

«Functional» User Login

«Functional» User SignUp

Properties

General

Name	User Login
Type	Requirement
Stereotype	Functional
Alias	
Keywords	
Status	Proposed
Version	1.0

Requirement

Abstract	☐
Active	☐

Notes

User can login the system using username and password

Activate Windows
Go to Settings to activate Windows.

Start Design Layout Develop Simulate Execute Construct Specialize Publish Settings

Model Requirements

Inspector

Details Trace Summary Status

Select an element to browse its contents

Toolbox

- SysML Requirements
 - Requirement
 - Test Case
- SysML Requirement Relationships
 - Containment
 - Trace
 - Copy
 - Derive
 - Verify
 - Refine
 - Satisfy
- SysML Requirement Extensions
 - Extended Requirement
 - Functional Requirement
 - Interface Requirement
 - Performance Requirement
 - Physical Requirement
 - Design Constraint
- SysML Common
 - Note
 - Document Artifact
 - Problem

«Functional» Search for Product

«Functional» Browse the product

«Functional» View the Product Details

«Functional» View Customer Reviews

«Functional» User Signup

«Functional» User Login

«Functional» Add product to cart

«Functional» Remove product from cart

Properties

General

Name	Requirements
Type	SysML Requirements
Stereotype	functional
Author	Admin
Applied Metamodel	Default
Filter to Metamodel	☐
Filter to Context	☐
Context Navigation	☐

Version

Version	1.0
---------	-----

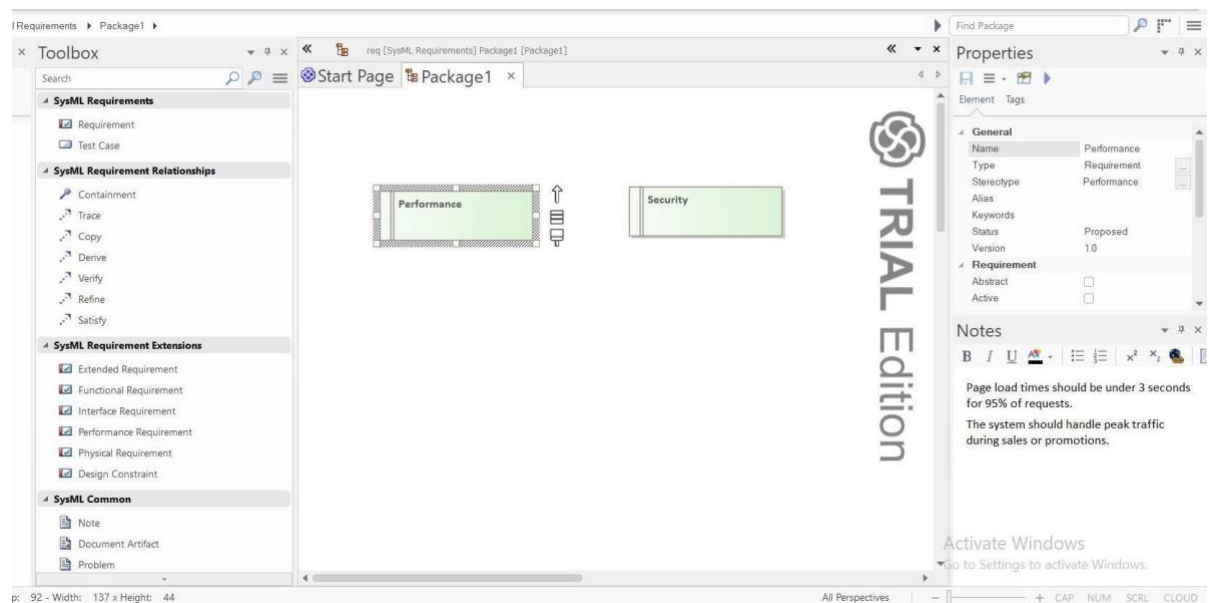
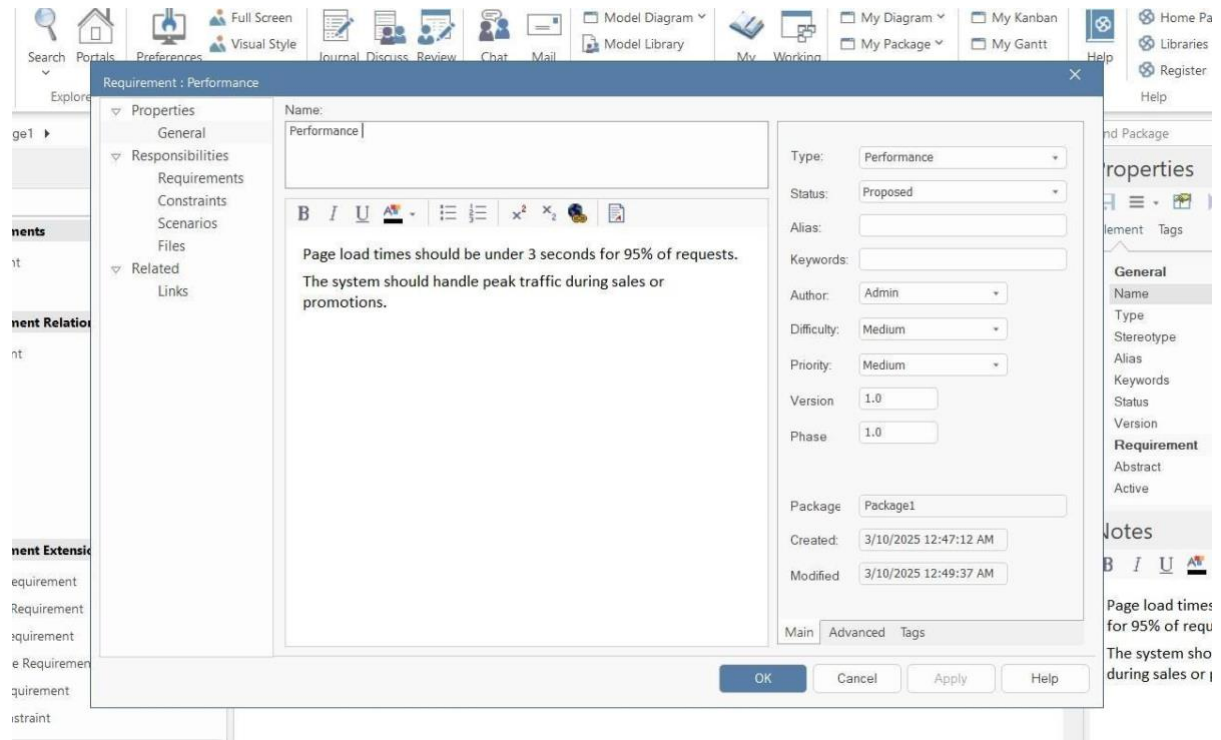
Notes

Activate Windows
Go to Settings to activate Windows.

Step 5: Organize Requirements

Group related functional requirements into **Packages** or **Modules** for better organization

Non-Functional Requirements:



Practice Task:

Practice task 01

[expected time 20 min]

Edu Learn is a global online learning platform that offers courses in various domains such as technology, business, and personal development. The platform allows students to register, enroll in courses, watch video lectures, participate in discussion forums, and take assessments.

The system must support high-definition video streaming with minimal buffering and ensure secure access to premium content. Additionally, it should track student progress and generate performance reports. Edu Learn must also integrate with third-party tools like Zoom for live sessions and provide multilingual support. Ensuring system availability and security is critical, as the platform caters to thousands of users worldwide.

Extract functional and non-functional requirements.

Practice task 02

[expected time 20 min]

Global Bank is launching a new digital banking platform that provides users with secure and convenient access to banking services. Customers can check account balances, transfer funds, pay bills, and apply for loans through the platform. The system should handle thousands of transactions per second while ensuring robust security measures such as multi-factor authentication and end-to-end encryption. The banking platform must also comply with financial regulations and provide 24/7 customer support through chat and phone services. Additionally, the mobile app should offer biometric authentication for enhanced security.

Extract functional and non-functional requirements.

Lab Manual for Software Requirement Engineering

LAB 03 : Non-Functional Requirements (NFRs)

1. Objective

- Understand the concept of **Non-Functional Requirements (NFRs)**
- Learn how to identify different types of NFRs
- Apply NFRs to real-world systems
- Differentiate between **functional** and **non-functional** requirements

2. Introduction

Non-Functional Requirements define **how a system should perform**, rather than what it should do. These requirements focus on system qualities such as performance, security, usability, and reliability.

Example:

- Functional: “User can log in”
- Non-functional: “Login should take less than 2 seconds”

3. Types of Non-Functional Requirements

- **Performance** → Speed, response time
- **Usability** → Ease of use
- **Reliability** → System stability
- **Security** → Data protection
- **Scalability** → Handling growth
- **Availability** → System uptime

4. Walkthrough Task (Step-by-Step)

Scenario: Online Food Delivery System

A company is developing an online food delivery system where users can browse restaurants, place orders, and track deliveries in real time.

Step 1: Understand the System

System features:

- User registration/login
- Browse restaurants
- Place orders
- Track delivery

Step 2: Identify Functional Requirements (for clarity)

- User can create account
- User can order food
- User can track order

Step 3: Identify Non-Functional Requirements

Now define **how well the system should perform**:

◆ Performance

- The system should load the homepage within **3 seconds**
- Order placement should complete within **2 seconds**

◆ Usability

- The interface should be **easy to navigate**
- Users should complete order in **maximum 3 steps**

◆ Security

- User data must be **encrypted**
- Payment transactions must be **secure**

◆ Reliability

- System should have **99% uptime**
- Orders should not be lost during processing

◆ Scalability

- System should handle **1000+ users simultaneously**

◆ Availability

System should be available **24/7**

Step 4: Write Final Answer (Well-Structured)

Students should present like this:

Non-Functional Requirements for Food Delivery System:

1. Performance: Pages load within 3 seconds
2. Security: Data encryption and secure payment
3. Usability: Easy navigation and quick ordering
4. Reliability: 99% uptime
5. Scalability: Supports 1000+ users
6. Availability: 24/7 service

5. Practice Tasks

Task 1: E-Learning Platform Case Study

An educational institute is developing an e-learning platform where students can watch lectures, download materials, take quizzes, and track their progress. The platform should support both live and recorded sessions. Students may access the system from different devices and locations. The system should handle multiple users attending live classes simultaneously without lag. It should also ensure that student data, including grades and personal information, is kept secure.

Student Task

Identify and write at least:

- 5 Non-Functional Requirements
- Cover performance, security, usability, and reliability

Task 2: Online Banking System Case Study

A bank wants to develop an online banking system where users can check balances, transfer money, and pay bills. The system must ensure high security due to sensitive financial data. It should process transactions quickly and accurately. The application must be available at all times, especially during peak hours. Any system failure could lead to financial loss and customer dissatisfaction.

Student Task

Write Non-Functional Requirements focusing on:

- Security
- Performance
- Availability
- Reliability

Task 3: Hospital Management System **Case Study**

A hospital is implementing a digital system to manage patient records, doctor schedules, appointments, and billing. Doctors and staff need quick access to patient data in emergency situations. The system should be highly reliable and must not crash during critical operations. It should also protect sensitive patient information and comply with data privacy standards. The system will be used by multiple departments simultaneously.

Student Task

Identify Non-Functional Requirements related to:

- Reliability
 - Security
 - Performance
 - Usability
-

Lab-04

SRE Processes and Players

Lab 3: SRE Processes and Players

1. Objective

The objective of this lab is to provide students with a comprehensive understanding of the Site Reliability Engineering (SRE) process and the roles of different stakeholders involved in it. By utilizing Click Up, students will collaboratively map out the SRE process and identify key players in a sample project. This exercise will enhance their understanding of how SRE practices contribute to the reliability and performance of software systems.

Relevant Lecture Material

a) Textbooks:

Managing Software Requirements: A Use Case Approach, Second Edition by Dean Leffingwell, Don Widrig, Addison Wesley

b) Reference Material:

Requirements Engineering: From System Goals to UML Models to Software Specifications, Axel V. Lamsweerde, (2009) ISBN-10: 0470012706

2. Activity Time Boxing:

ACTIVITY NAME	ACTIVITY TIME	TOTAL
Quiz	25 min	25 min
Setting Up and Setting	5min	5 min
Walkthrough Task	20 min	40 min
Practice Task	20 to 30 min for each task	60 min

3. Quiz [Expected time 25 min]

4. Concept Map:

Software Requirements Engineering (SRE) is a critical phase in the software development lifecycle that focuses on identifying, analyzing, documenting, and validating the requirements for a software system.

Key SRE Processes

The processes involved in Software Requirements Engineering typically include the following key activities:

1. Requirements Elicitation

- **Objective:** Gather requirements from stakeholders.
- **Activities:**
 - Identify stakeholders (e.g., users, customers, developers, business analysts).
 - Conduct interviews, workshops, surveys, or brainstorming sessions.
 - Use techniques like observation, prototyping, or focus groups.

- Gather both functional (what the system should do) and non-functional (constraints, performance, security) requirements.
- Output: Initial list of requirements.

2. Requirements Analysis

- **Objective:** Understand and refine the gathered requirements.
- **Activities:**
 - Clarify ambiguous requirements.
 - Prioritize requirements based on importance and feasibility.
 - Identify conflicts or inconsistencies in requirements.
 - Model requirements using techniques like use cases, user stories, or data flow diagrams.
- Output: Clear, detailed, and structured requirements.

3. Requirements Specification

- **Objective:** Document the requirements in a formal and structured manner.
- **Activities:**
 - Write a Software Requirements Specification (SRS) document.
 - Include functional, non-functional, and system requirements.
 - Use diagrams, tables, and models to support textual descriptions.
 - Ensure the document is clear, concise, and unambiguous.
- Output: SRS document.

4. Requirements Validation

- **Objective:** Ensure the requirements are accurate, complete, and aligned with stakeholder needs.
- **Activities:**
 - Conduct reviews, walkthroughs, or inspections of the SRS document.
 - Use prototyping or simulations to validate requirements.
 - Obtain stakeholder feedback and approval.
 - Check for consistency, feasibility, and testability.
- Output: Validated and approved requirements.

5. Requirements Management

- **Objective:** Manage changes to requirements throughout the project lifecycle.
 - **Activities:**
 - Track changes to requirements using tools like requirement traceability matrices.
 - Handle change requests and assess their impact on the project.
 - Maintain version control of the SRS document.
 - Ensure alignment between requirements and project goals.
 - Output: Updated and managed requirements.
-

Key Players in SRE

Stakeholders are individuals or groups who have an interest in the software system. They play a critical role in defining, validating, and managing requirements. Key players include:

Customers

- **Role:** Provide the business needs and fund the project.
- **Contribution:** Define high-level business requirements and constraints.
- **Example:** A company commissioning a new software system.

End Users

- **Role:** Use the software system to perform tasks.
- **Contribution:** Provide input on functional requirements, usability, and user experience.
- **Example:** Employees using an internal tool or customers using a public app.

Business Analysts

- **Role:** Act as intermediaries between stakeholders and the development team.
- **Contribution:** Elicit, analyze, and document requirements.
- **Example:** A professional who conducts workshops and creates the SRS document.

Developers

- **Role:** Design and build the software system.
- **Contribution:** Provide technical feasibility feedback and clarify implementation details.
- **Example:** Software engineers or programmers.

Testers (Quality Assurance)

- **Role:** Ensure the software meets the specified requirements.
- **Contribution:** Validate requirements for testability and create test cases.
- **Example:** QA engineers who perform system testing.

Project Managers

- **Role:** Oversee the project and ensure it is delivered on time and within budget.
- **Contribution:** Prioritize requirements and manage scope.
- **Example:** A project manager coordinating the development team.

System Architects

- **Role:** Design the overall structure of the system.

- **Contribution:** Ensure requirements align with the system architecture.
- **Example:** A technical architect defining the system's high-level design.

Suppliers/Vendors

- **Role:** Provide third-party components or services.
- **Contribution:** Define requirements for integration with external systems.
- **Example:** A payment gateway provider for an e-commerce system.

Maintenance Team

- **Role:** Maintain and update the software after deployment.
- **Contribution:** Provide input on maintainability and scalability requirements.
- **Example:** IT support team responsible for system updates.

4. Key Challenges in Involving Players

- **Conflicting Interests:** Different stakeholders may have conflicting requirements.
- **Communication Gaps:** Miscommunication between technical and non-technical stakeholders.
- **Changing Needs:** Stakeholders may change their requirements during the project.
- **Engagement Issues:** Some stakeholders may not be actively involved in the process.

Class Activity: Mapping Out the SRE Process

Sample Project:

Mapping the SRE Process and Identifying Key Players

This activity is designed to help participants understand the **Software Requirements Engineering (SRE)** process and the roles of different stakeholders. Using Click Up as a tool, participants will map out the SRE process for a sample project, identify key players, and assign responsibilities. Below is a detailed breakdown of the activity, including a **walkthrough** and **two practice tasks**.

Objective

1. Understand the **SRE process** and the roles of stakeholders.
2. Use Click Up to map out the SRE process for a sample project.
3. Identify **key players** and their responsibilities.

Tools and Resources

- **Tool:** Click Up (a project management tool for planning, tracking, and collaborating on projects).
- **Sample Project Scenario:** A simple project scenario for the group to work on (e.g., developing a mobile app for a library system, an e-commerce platform, or a student attendance system).

Step 1: Introduce the SRE Process

Explain the key stages of the SRE process:

1. **Requirements Elicitation:** Gathering requirements from stakeholders.
2. **Requirements Analysis:** Analyzing and prioritizing requirements.
3. **Requirements Specification:** Documenting requirements in a structured format.
4. **Requirements Validation:** Reviewing and validating requirements with stakeholders.
5. **Requirements Management:** Tracking and managing changes to requirements.

Step 2: Provide a Sample Project Scenario

Example: Library Management System

- The system should allow librarians to manage books, members, and borrowing records.
- Members should be able to search for books, check availability, and reserve books.
- The system should send reminders for overdue books.

Step 3: Identify Key Players (Stakeholders)

Ask the group to identify the key players involved in the SRE process for the sample project.

Examples:

- **Stakeholders:** Librarians, library members, library management.

- **Business Analysts:** Gather and document requirements.
- **Developers:** Provide technical feasibility feedback.
- **Product Owner:** Prioritize requirements (if using Agile).
- **QA Team:** Ensure requirements are testable.
- **Project Manager:** Oversee the process and ensure timelines are met.

Step 4: Map the SRE Process Using Click Up

Task: Use Click Up to create a workflow for the SRE process.

1. Create a Project:

- a. Set up a new project in Click Up for the **Library Management System**.

2. Define Work Packages:

- a. Create work packages for each stage of the SRE process:
 - i. **Elicitation**
 - ii. **Analysis**
 - iii. **Specification**
 - iv. **Validation**
 - v. **Management**

3. Assign Tasks to Players:

- a. Assign tasks to the identified stakeholders (e.g., Business Analysts for elicitation, QA team for validation).

4. Add Details:

- a. Include descriptions, deadlines, and dependencies for each task.

5. Visualize the Process:

- a. Use Click Up **Gantt chart** or **Kanban board** to visualize the workflow.

Step 5: Group Discussion and Presentation

- Each group presents their mapped SRE process and explains the roles of the key players.
- Discuss challenges they faced and how they addressed them.

Sample Output in Click Up

Here's an example of how the SRE process can be mapped in Click Up:

Work Packages (Stages of SRE Process):

1. Requirements Elicitation:

- a. Tasks: Conduct interviews with librarians and members, organize workshops.
- b. Assigned to: Business Analysts.
- c. Deadline: Week 1.

2. Requirements Analysis:

- a. Tasks: Analyze gathered requirements, identify conflicts, and prioritize.
- b. Assigned to: Business Analysts, Product Owner.
- c. Deadline: Week 2.

3. Requirements Specification:

- a. Tasks: Document requirements in an SRS, create use cases.
- b. Assigned to: Requirements Engineers.
- c. Deadline: Week 3.

4. Requirements Validation:

- a. Tasks: Review SRS with stakeholders, create prototypes.
- b. Assigned to: QA Team, Stakeholders.
- c. Deadline: Week 4.

5. Requirements Management:

- a. Tasks: Track changes, update requirements baseline.
- b. Assigned to: Project Manager, Requirements Engineers.
- c. Deadline: Ongoing.

Key Players and Their Roles

Player	Role
Librarians	Provide input on book management and member interactions.
Library Members	Provide feedback on search, reservation, and notification features.
Business Analysts	Elicit and document requirements.
Requirements Engineers	Analyze and specify requirements.
Product Owner	Prioritize requirements and manage the product backlog (if using Agile).
Developers	Provide technical feasibility feedback.
QA Team	Validate requirements and ensure testability.
Project Manager	Oversee the SRE process and ensure timely delivery.

Walkthrough Task:

Task 01

[expected time 30 min]

Sample project: Setting Up a New Deployment Pipeline

The project aims to create a new deployment pipeline to make software releases faster and more reliable. First, the SRE team will gather requirements by talking to developers and operations staff to understand what they need from the new system. This will help identify any problems with the current deployment process.

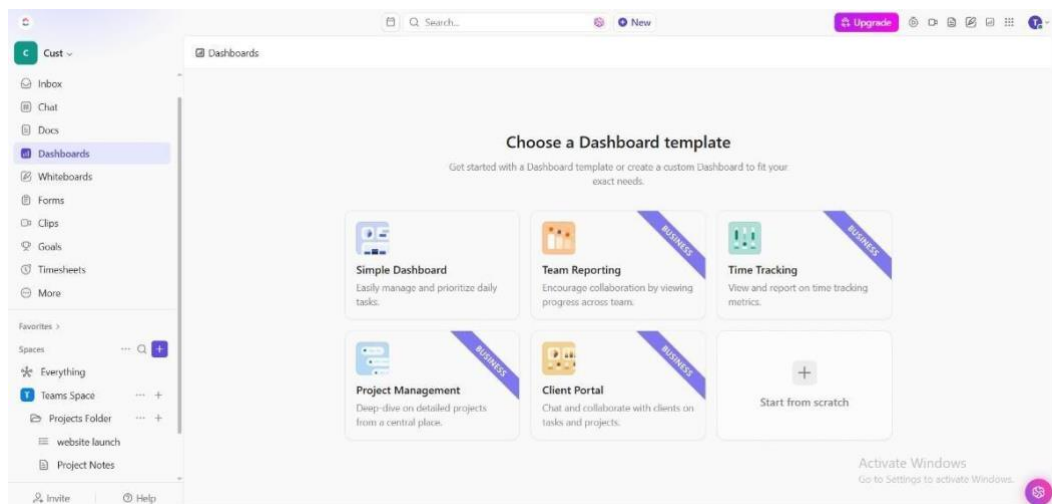
Next, the team will design the pipeline, ensuring it works well with existing systems and follows best practices for security and scalability. After the design is approved, the team will set up the necessary tools for the pipeline, including automation scripts to streamline the deployment process.

Once everything is set up, the team will test the pipeline to make sure it works as intended. They will run test deployments and gather feedback to fix any issues. Finally, the team will create user guides and conduct training sessions for developers and operations staff to help them use the new pipeline effectively. This project will help the organization release software more quickly and reliably.

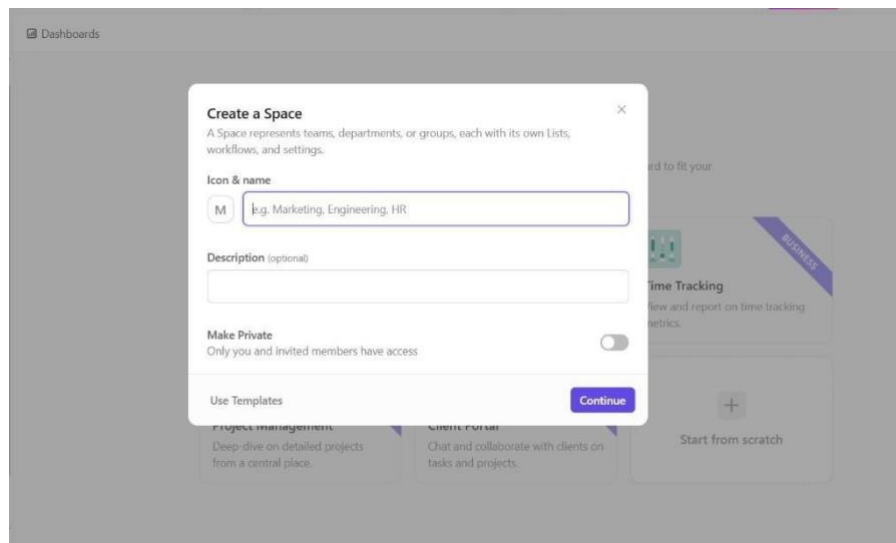
Step-by-Step Guide to Setting Up Your Project in ClickUp Create

a New Space (Optional):

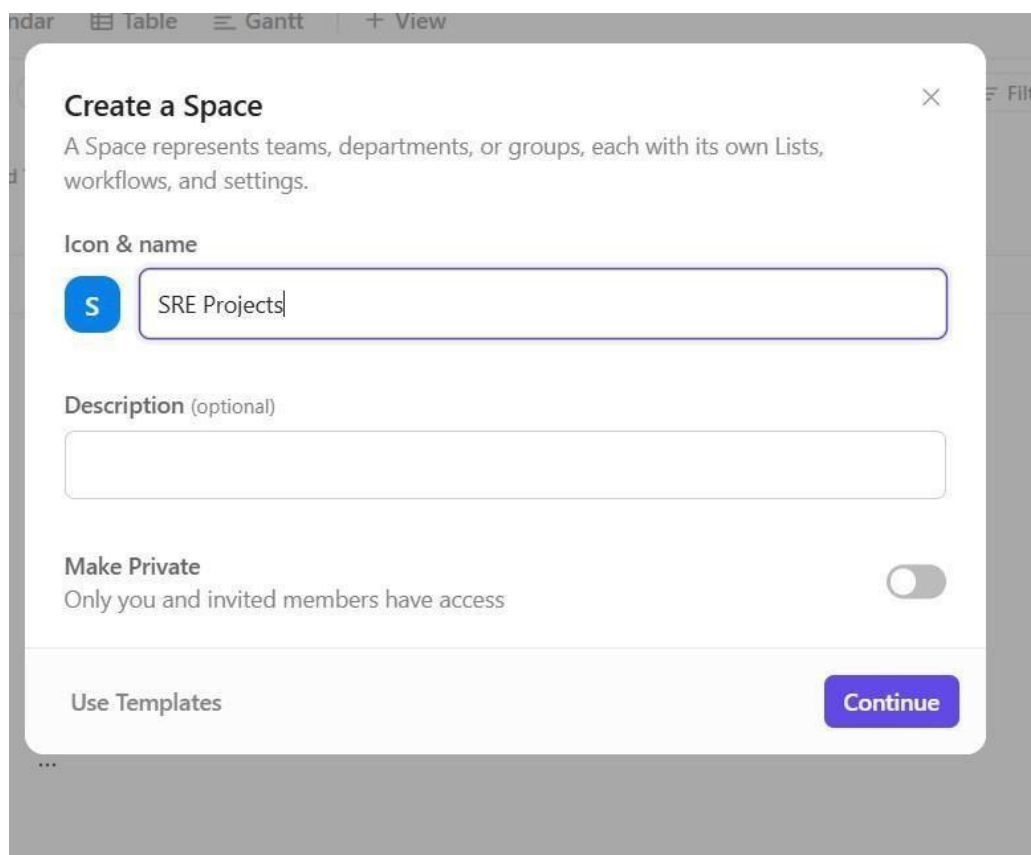
If you want to keep this project separate, you can create a new Space in ClickUp.

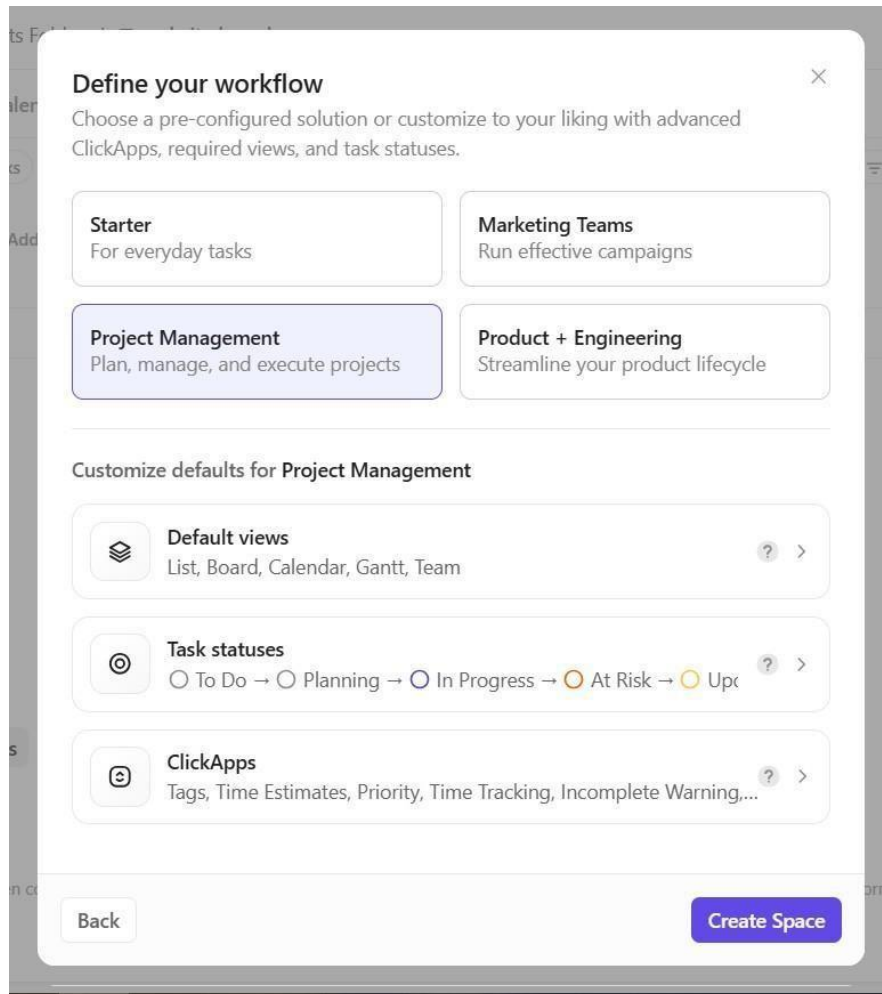


Click on the “Spaces” tab in the sidebar, then click “+ New Space.”



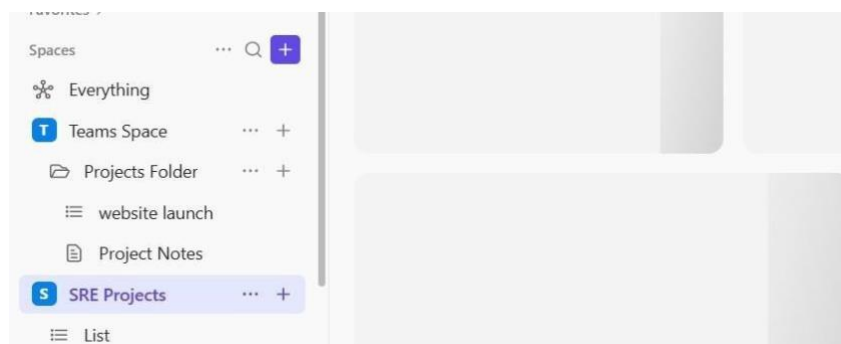
Name it something like "Deployment Projects" and customize it as needed.



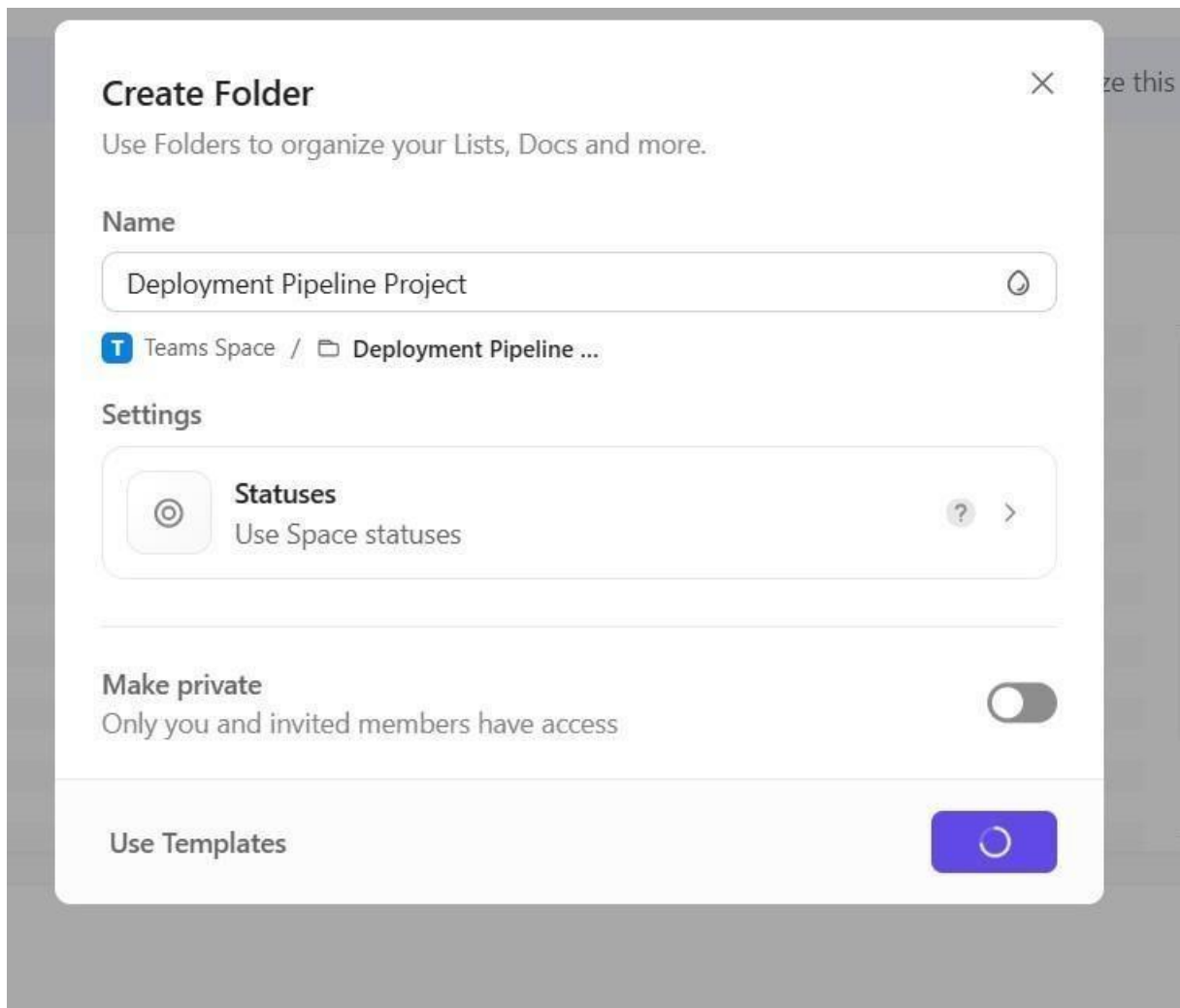


Create a New Folder (Optional):

Within your Space, you can create a Folder to organize related projects.

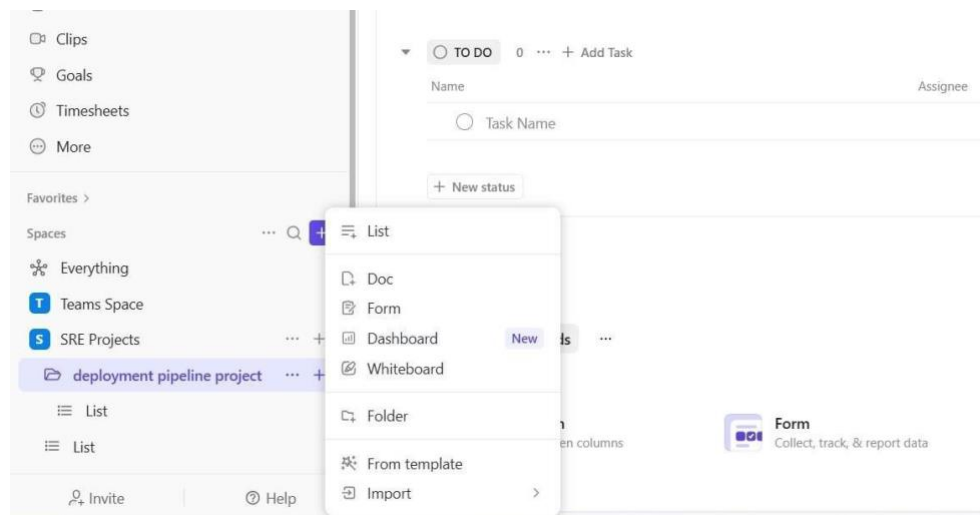


Click on the “+ New Folder” button and name it "Deployment Pipeline Project."

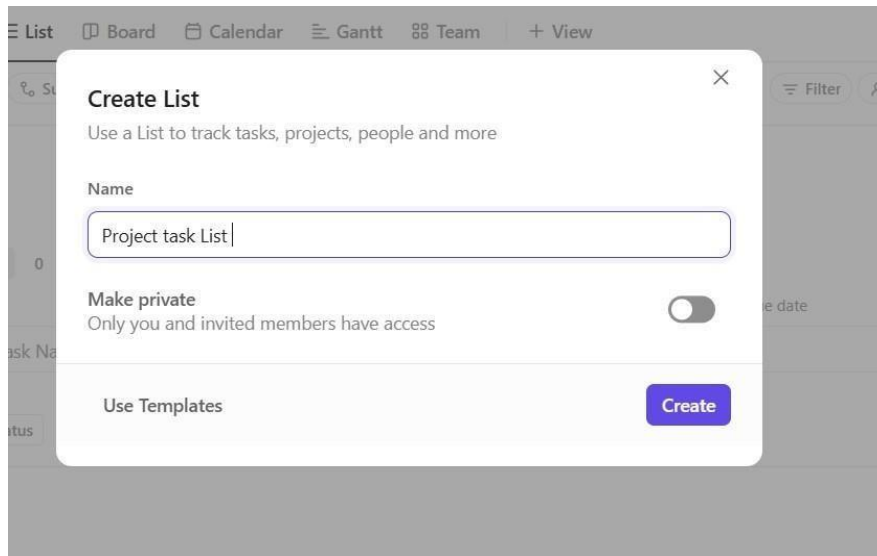


Create a New List:

Inside your Folder, create a new List for the project.



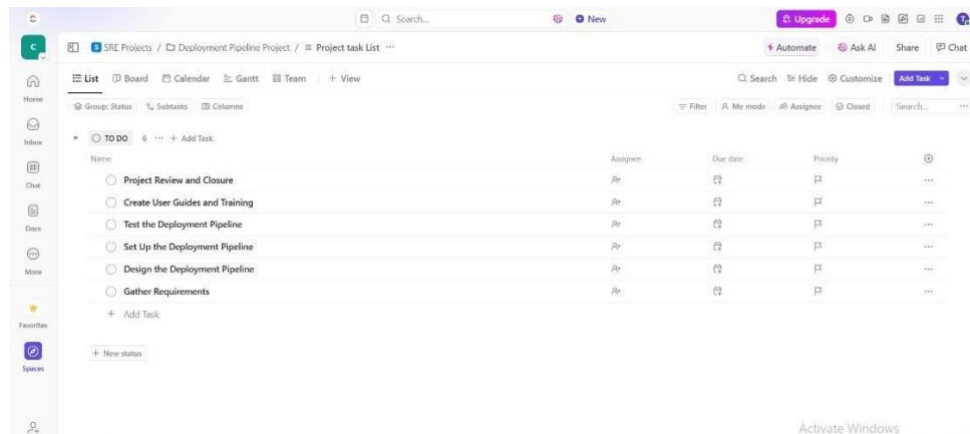
Click on "+ New List" and name it "Create a New Deployment Pipeline."



Add Tasks:

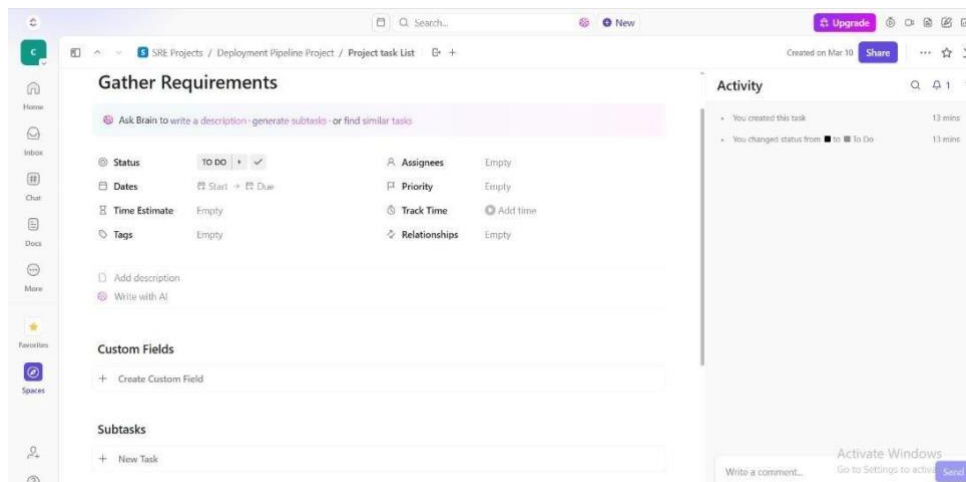
Click on “+ New Task” to create the main tasks outlined in the project.

For each main task (e.g., "Gather Requirements," "Design the Deployment Pipeline," etc.), create a separate task.

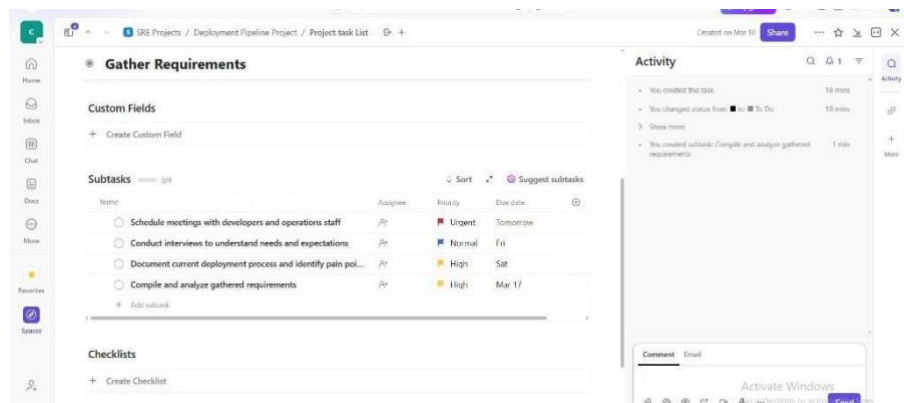


Add Subtasks:

Click on each main task to open it, then click on “+ Add Subtask” to create the subtasks.



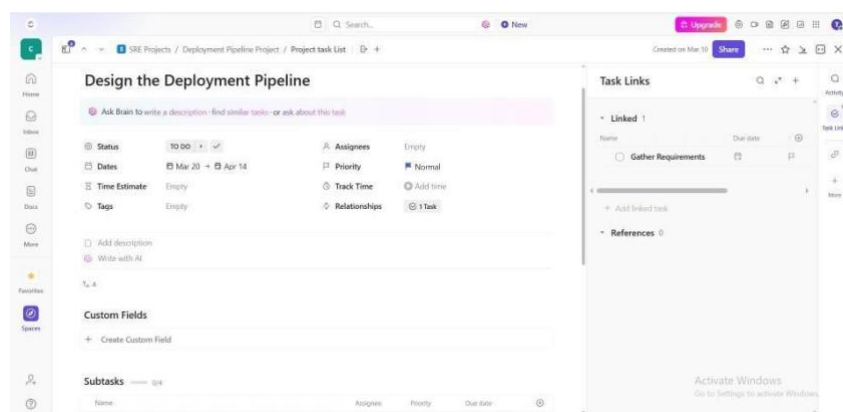
Input the subtasks as outlined in the previous response (e.g., for "Gather Requirements," add subtasks like "Schedule meetings with developers," etc.).

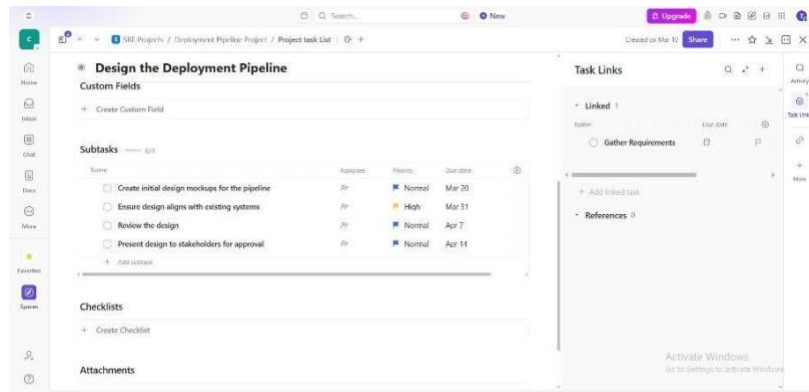


Set Due Dates:

For each task and subtask, set realistic due dates by clicking on the calendar icon in the task view.

Consider dependencies and the overall timeline of the project when assigning due dates.





Assign Team Members:

Assign tasks and subtasks to the appropriate team members by clicking on the “Assignee” field in each task.

Select the team members who will be responsible for each task.

Add Descriptions and Attachments(Optional):

For each task, you can add a description to provide more context or details.

You can also attach relevant documents or links that may help team members complete their tasks.

Use Custom Fields (Optional):

If you want to track additional information (like priority, status, or estimated time), you can create custom fields.

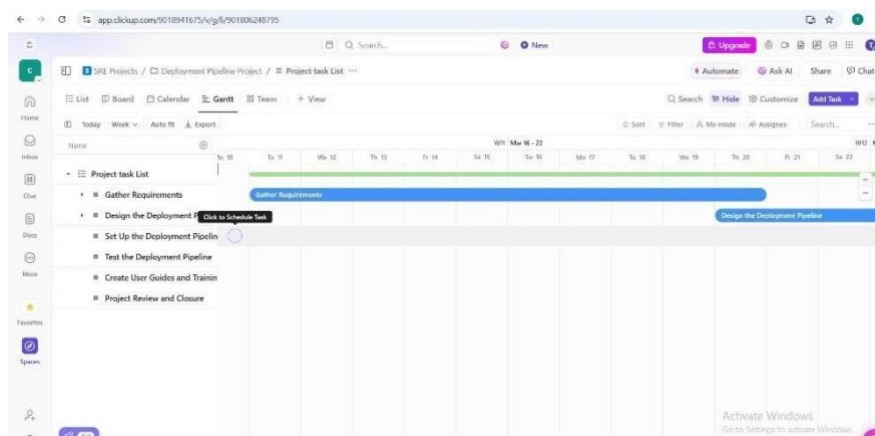
Click on the “Custom Fields” option in the task view to add fields that suit your project needs.

Switch to Gantt View:

To visualize the project timeline, switch to the Gantt chart view.

Click on the “Views” dropdown at the top of your List and select “Gantt.”

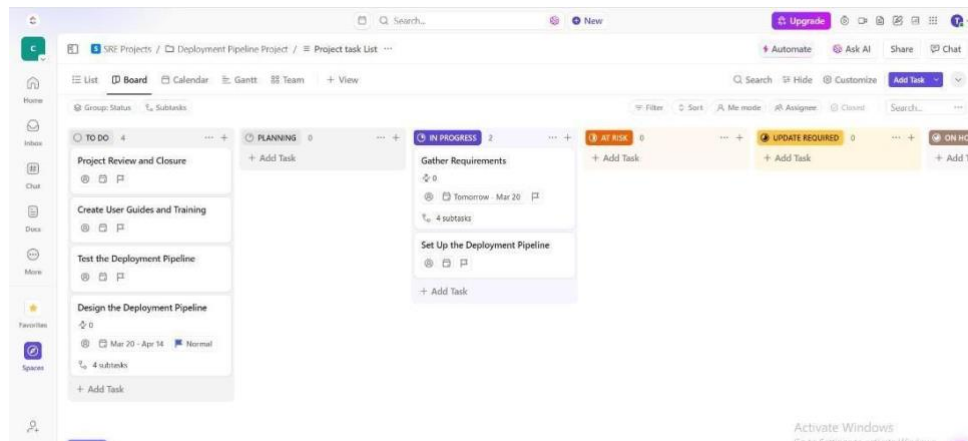
This will allow you to see the timeline of tasks, their dependencies, and how they overlap.



Monitor Progress:

Regularly check the progress of tasks and subtasks.

Use the “Status” field to update the progress (e.g., Not Started, In Progress, Completed).



5. Practice Task

Practice task 01

[expected time 20 min]

Task 1: E-Commerce Platform

Scenario: Develop an e-commerce platform that allows users to browse products, add them to a cart, and complete purchases. The platform should support multiple payment methods and provide personalized recommendations.

Steps:

1. Identify the **key players** (e.g., customers, product managers, developers, QA team).
2. Map the **SRE process** using Click Up:
 - a. Create work packages for elicitation, analysis, specification, validation, and management.
 - b. Assign tasks to stakeholders (e.g., Business Analysts for elicitation, QA team for validation).
3. Present your workflow and explain the roles of key players.

Practice task 02

[expected time 20 min]

Task 2: Student Attendance System

Scenario: Develop a system for tracking student attendance in a school. The system should allow teachers to mark attendance, generate reports, and send notifications to parents.

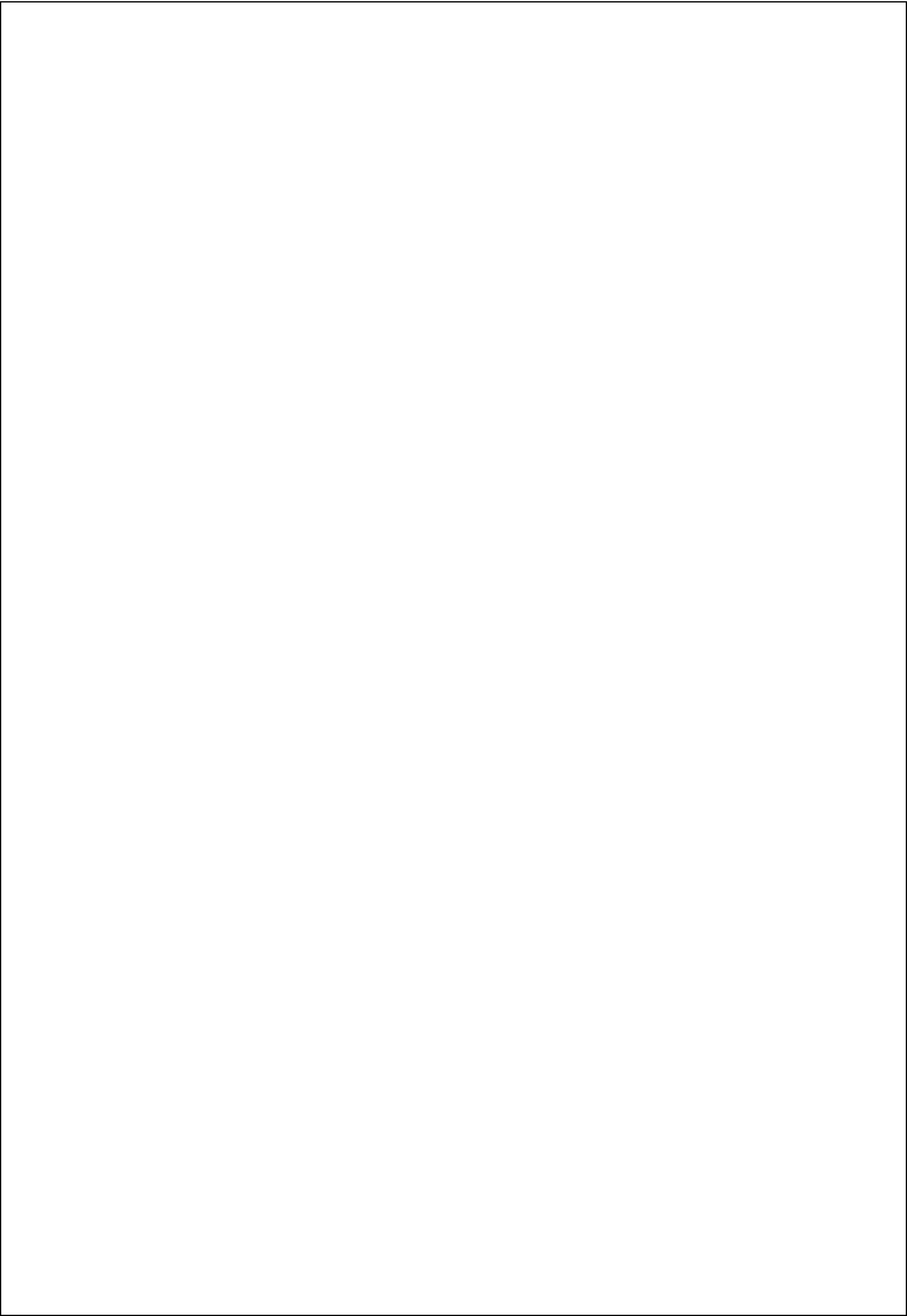
Steps:

1. Identify the **key players** (e.g., teachers, students, parents, school administration).
2. Map the **SRE process** using Click Up:
 - a. Create work packages for elicitation, analysis, specification, validation, and management.

- b. Assign tasks to stakeholders (e.g., Business Analysts for elicitation, QA team for validation).
3. Present your workflow and explain the roles of key players.

Reflection and Learning Outcomes

- Participants will understand the **SRE process** and the importance of each stage.
- They will learn how to use Click Up for project planning and tracking.
- They will gain experience in identifying **stakeholders** and assigning role.



Lab Manual for Software Requirement Engineering

Lab-05

Human social and cultural issues

Lab 5: Human, Social, and Cultural Factors in SRE

Objective

1. Learn how human behavior affects software requirements.
2. Explore social dynamics (e.g., teamwork, biases) in requirement gathering.
3. Study cultural differences that influence software design.
4. Practice creating inclusive and ethical software requirements.

Relevant Lecture Material:

a) Textbooks:

Managing Software Requirements: A Use Case Approach, Second Edition by Dean Leffingwell, Don Widrig, Addison Wesley

b) Reference Material:

Requirements Engineering: From System Goals to UML Models to Software Specifications, Axel V. Lamsweerde, (2009) ISBN-10: 0470012706

Activity Time Boxing:

ACTIVITY NAME	ACTIVITY TIME	TOTAL
Quiz	25 min	25 min
Setting Up and Setting	5min	5 min
Walkthrough Task	20 min	40 min
Practice Task	20 to 30 min for each task	60 min

Quiz

[Expected time 25 min]

Concept Map:

In SRE, reliability and performance are critical, but these cannot be achieved without considering the human, social, and cultural factors that influence how users interact with the system. This lab will:

1. Introduce key concepts related to human, social, and cultural factors in requirements engineering.
2. Provide a case study walkthrough to demonstrate how to analyze these factors.
3. Include two practice case studies for students to apply their learning.

Definitions

Human Factors

Definition:

Human factors are all about **how people interact with a system**. This includes making sure the system is easy to use, accessible to everyone, and matches how people naturally behave.

Examples:

1. **Usability:**
 - a. The system should be easy to use, even for people who aren't tech knowledge.
 - b. Example: A shopping app should have a simple layout so users can find products quickly.
2. **Accessibility:**
 - a. The system should work for everyone, including people with disabilities.
 - b. Example: Adding features like screen readers for visually impaired users or larger buttons for people with limited mobility.
3. **User Behavior:**
 - a. The system should match how people naturally behave.
 - b. Example: If users tend to abandon their shopping carts, the system could send reminders or offer discounts to encourage them to complete their purchase.

Social Factors

Definition:

Social factors are about **how society and groups of people influence what users expect from a system**. This includes things like social norms, trends, and how people interact with each other.

Examples:

1. Social Sharing:

- a. People like to share their experiences with others, so the system should support this.
- b. Example: An e-commerce platform could allow users to share products on social media or leave reviews.

2. Community Features:

- a. People often trust recommendations from others, so the system should include community-driven features.
- b. Example: A travel app could show reviews and ratings from other travelers to help users decide where to stay.

3. Group Behavior:

- a. People often follow trends or do what others are doing.
- b. Example: If a product becomes popular, the system could highlight it as a “trending item” to encourage more sales.

Cultural Factors

Definition:

Cultural factors are about **how differences in culture affect what users expect from a system**. This includes things like language, traditions, and local preferences.

Examples:

1. Language:

- a. The system should support multiple languages to cater to users from different regions.
- b. Example: A global e-commerce platform should offer the website in English, Spanish, French, etc.

2. Currency and Payment Methods:

- a. Different countries use different currencies and payment methods, so the system should adapt to these.
- b. Example: A shopping app should allow users in India to pay in rupees using UPI, while users in the U.S. can pay in dollars using credit cards.

3. Cultural Preferences:

- a. Different cultures have different preferences and traditions, so the system should respect these.

- b. Example: A food delivery app should offer halal or vegetarian options in regions where these are important.

Why Are These Factors Important?

- **Human Factors:** Ensure the system is **easy to use** and **accessible** for everyone.
- **Social Factors:** Make the system **socially engaging** and **trustworthy**.
- **Cultural Factors:** Ensure the system **respects and adapts** to different cultures.

Real-World Example: E-Commerce Platform

Let's say you're designing an e-commerce platform like Amazon. Here's how these factors come into play:

Human Factors:

- The website should be easy to navigate, with clear categories and a search bar.
- It should work well on both desktop and mobile devices.
- If a user adds items to their cart but doesn't check out, the system could send a reminder email.

Social Factors:

- Users should be able to leave reviews and ratings for products.
- The platform could show "most popular" or "trending" items based on what others are buying.
- Users should be able to share products on social media.

Cultural Factors:

- The platform should support multiple languages (e.g., English, Spanish, French).
- It should allow users to pay in their local currency and use popular payment methods (e.g., PayPal, UPI).
- It should offer products that are culturally relevant (e.g., traditional clothing, local foods).

Walkthrough Task:

Case Study: The Cross-Cultural Struggle to Launch a Disaster Alert App A tech startup, **SafeWorld**, aimed to create a disaster alert app to warn communities in flood-prone regions of Indonesia, cyclone-affected areas in the Caribbean, and earthquake zones in Nepal. The app would send real-time alerts, evacuation maps, and survival tips in local languages. However, the development team—based in the U.S., Japan, and France—faced unexpected hurdles. The U.S. team designed the app interface with a red "Danger" button, unaware that red symbolizes prosperity in Indonesia and was seen as inappropriate for warnings. Japanese developers, working in a highly formal corporate culture, strictly followed the project timeline but avoided questioning the U.S. team's assumption that all users could read maps, ignoring rural Nepali villages where literacy rates are low. The French project managers scheduled critical meetings during European working hours, making it impossible for Indonesian field testers to join due to the 12-hour time difference. When the app launched, users in the Caribbean rejected it because evacuation routes were based on paved roads, which many remote villages lacked. Meanwhile, religious leaders in Indonesia demanded the app avoid using loud alarm sounds during prayer times, a detail the team had overlooked. The app's AI chatbot, trained in formal Spanish, confused Caribbean users who relied on informal talks.

Assignment for Students

1. Identify the Issues: List the human (e.g., time zone conflicts), social (e.g., low literacy assumptions), and cultural (e.g., color symbolism) problems in the case study.
2. Propose Solutions: How would you redesign the app to address these issues?

Task Breakdown

Here's a breakdown of **human, cultural, and social issues** from the **Disaster Alert App case study**:

Human Issues

1. Time Zone Conflicts:

French project managers scheduled meetings during European hours, excluding Indonesian testers due to a 12-hour time difference.

2. Team Communication Gaps:

Japanese developers avoided questioning the U.S. team's flawed assumption about users' map-reading skills (common in hierarchical or overly formal workplace cultures).

3. Lack of Local Collaboration:

No involvement of local testers or community leaders during the design phase, leading to mismatched features.

Cultural Issues

1. Color Symbolism:

The U.S. team used a red "Danger" button, unaware that red symbolizes prosperity in Indonesia, making alerts culturally inappropriate.

2. Religious Sensitivities:

Indonesian religious leaders objected to loud alarm sounds during prayer times, which the team overlooked.

3. Language Nuances:

The AI chatbot used formal Spanish, The AI chatbot failed Caribbean users by ignoring their natural way of speaking., causing confusion.

Social Issues

1. Literacy Assumptions:

The app relied on map-based evacuation routes, ignoring low literacy rates in rural Nepal where many users couldn't read maps.

2. Infrastructure Inequality:

Evacuation routes were designed for paved roads, excluding remote Caribbean villages with unpaved or nonexistent roads.

3. Marginalization of Local Knowledge:

Developers prioritized global consistency over local needs (e.g., ignoring Caribbean dialects or Indonesian prayer customs).

Practice Task

Practice task 01 [expected time 20 min]

Identify the cultural, social, and human issues from the case study.

Case Study 1: The Health Tracking App That Missed the Mark in Rural Vietnam and Saudi Arabia

A health tech startup, *WellLife*, developed a fitness app to promote physical wellness in rural Vietnam and urban Saudi Arabia. The app included features like step counters, calorie trackers, and meditation

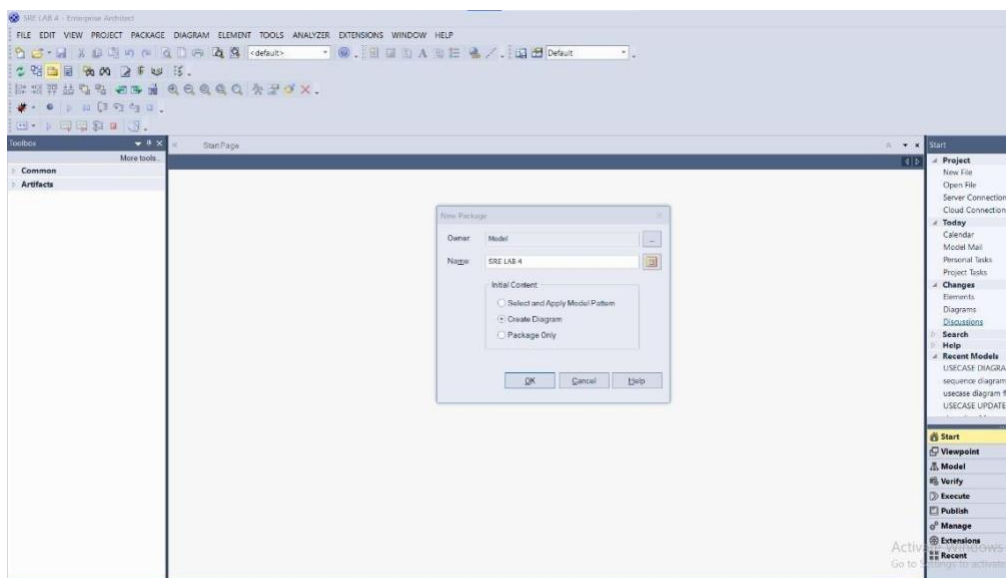
guides. The development team, based in the U.S. and South Korea, designed the app with vibrant animations and a gamified reward system. In rural Vietnam, farmers found the step-counting feature irrelevant, as their days were already physically demanding, and the app's calorie recommendations were based on Western diets, clashing with traditional Vietnamese meals like pho and rice-based dishes. Meanwhile, in Saudi Arabia, the app's female avatars wore workout clothing deemed culturally inappropriate, leading to low adoption among women. To encourage social sharing, the app displayed users' progress on public leaderboards, but Saudi users perceived this as intrusive to privacy. The team also overlooked Ramadan schedules, sending push notifications for midday workouts during fasting hours. After launch, rural Vietnamese users uninstalled the app within weeks, calling it "useless," while Saudi influencers criticized it for ignoring local norms.

Practice task 02

[expected time 20 min]

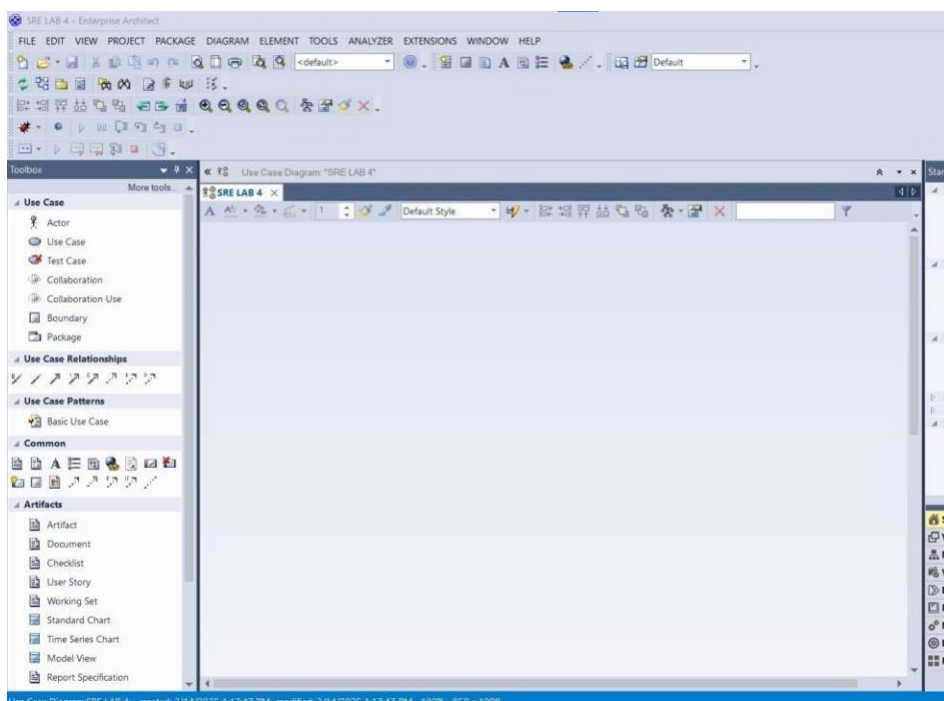
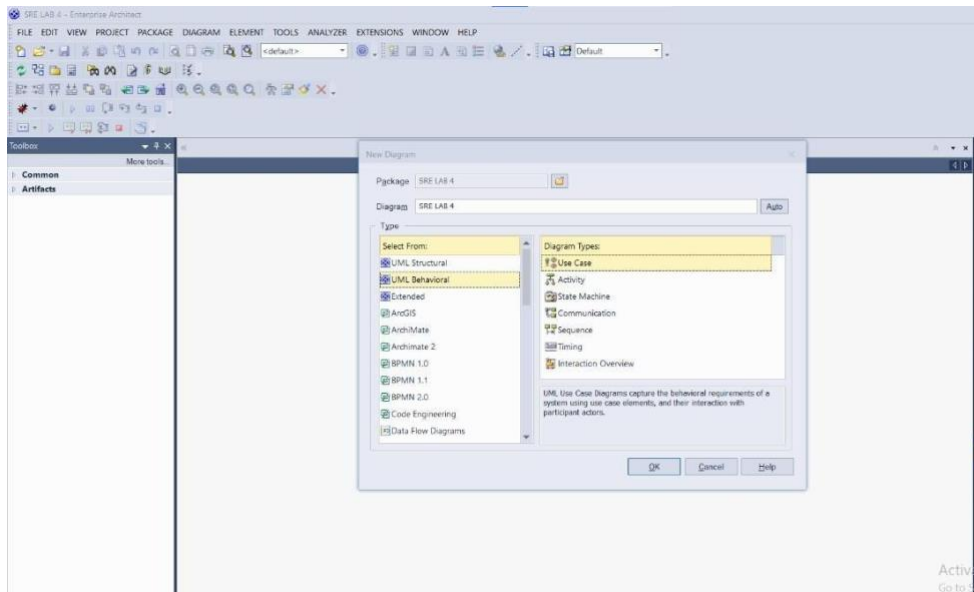
Case Study 2: The Language Learning App That Alienated Indigenous Communities in Mexico and New Zealand

LingoGlobal, an edtech company, launched a language-learning app to preserve indigenous languages like Nahuatl in Mexico and Māori in New Zealand. The app used AI tutors, interactive quizzes, and speech recognition. However, the content was designed by linguists in Spain and engineers in China who lacked input from native speakers. In Mexico, the app taught formal Nahuatl vocabulary used in academic settings, which differed sharply from the colloquial dialects spoken in rural communities. Māori elders in New Zealand protested when the AI mispronounced sacred words, calling it disrespectful to their culture. The app also required users to record themselves speaking, but many indigenous communities in Mexico shared devices, making private practice impossible. To promote engagement, LingoGlobal added a feature where users competed for "language mastery badges," but Māori users found the competitive tone contrary to their communal values. After backlash, the company faced petitions to remove the app, with critics calling it a "digital colonizer."



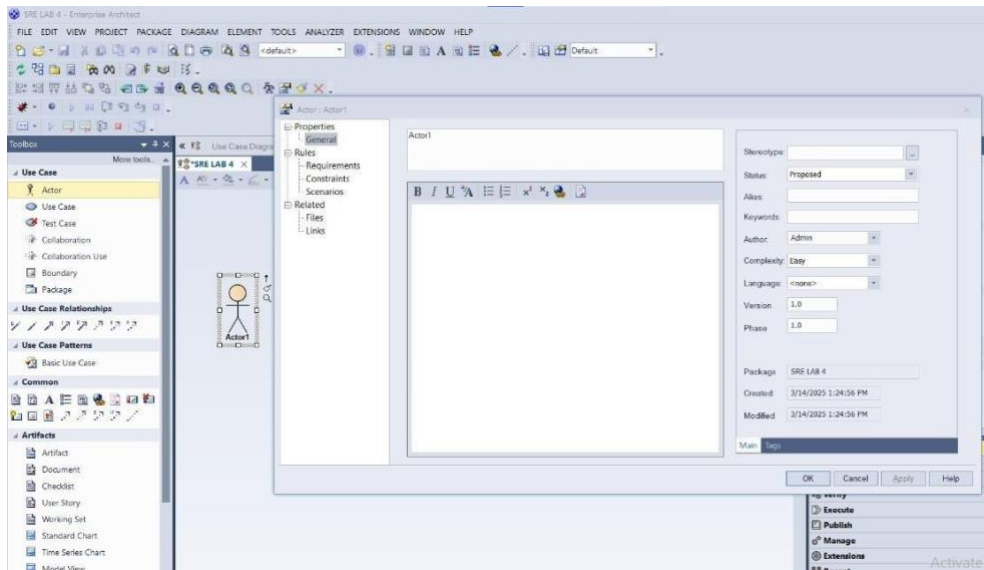
1. Add a Use Case Diagram:

- Right-click on the project → **Add Diagram** → **Use Case Diagram**.



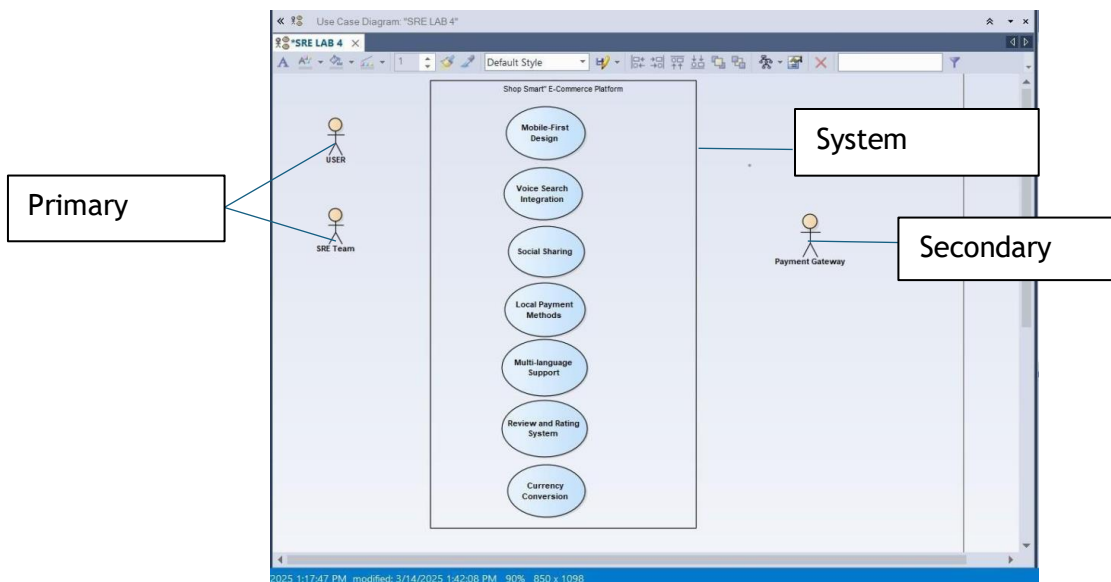
2. Add Actors:

- Drag and drop the **Actor** element onto the diagram.



b. Name the actors:

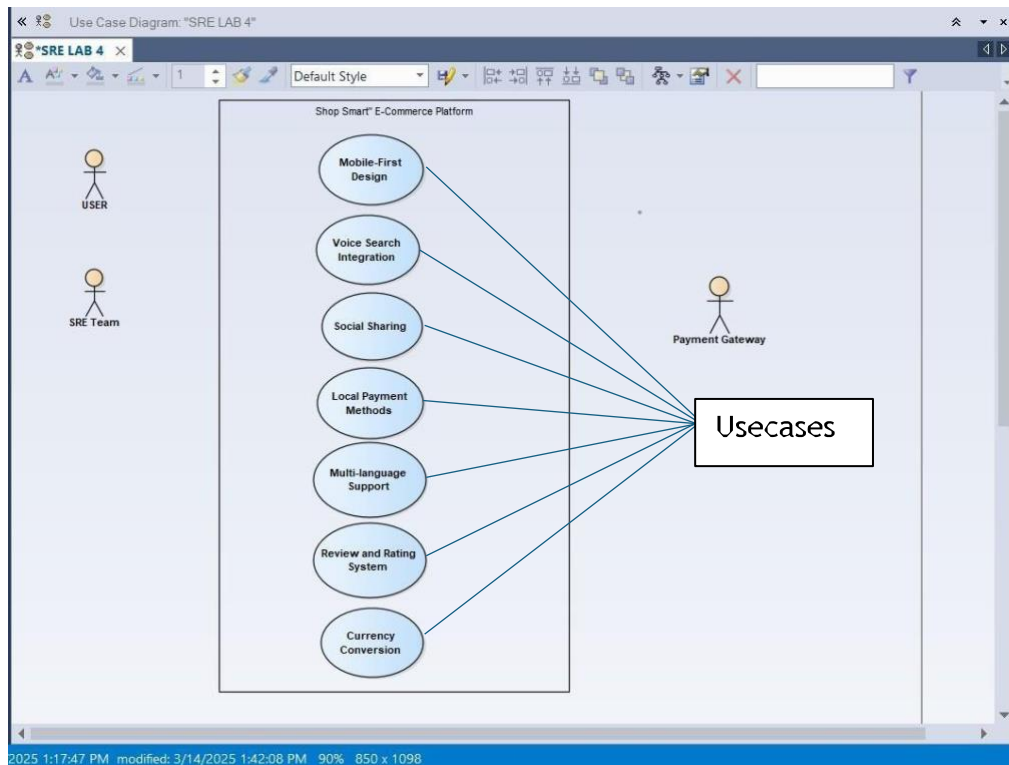
- i. User
- ii. SRE Team
- iii. Payment Gateway



3. Add Use Cases:

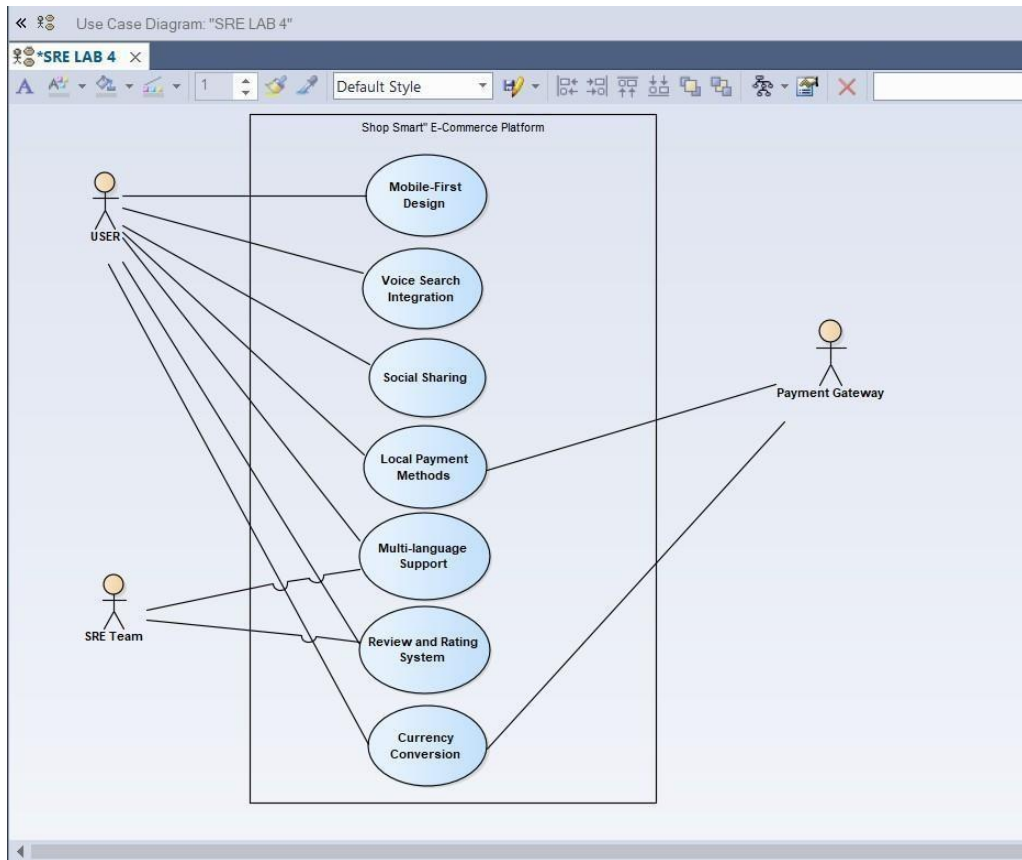
- a. Drag and drop the **Use Case** element onto the diagram.
- b. Name the use cases:
 - i. Mobile-First Design
 - ii. Voice Search Integration

- iii. Review and Rating System
- iv. Multi-language Support
- v. Local Payment Methods
- vi. Currency Conversion
- vii. Social Sharing



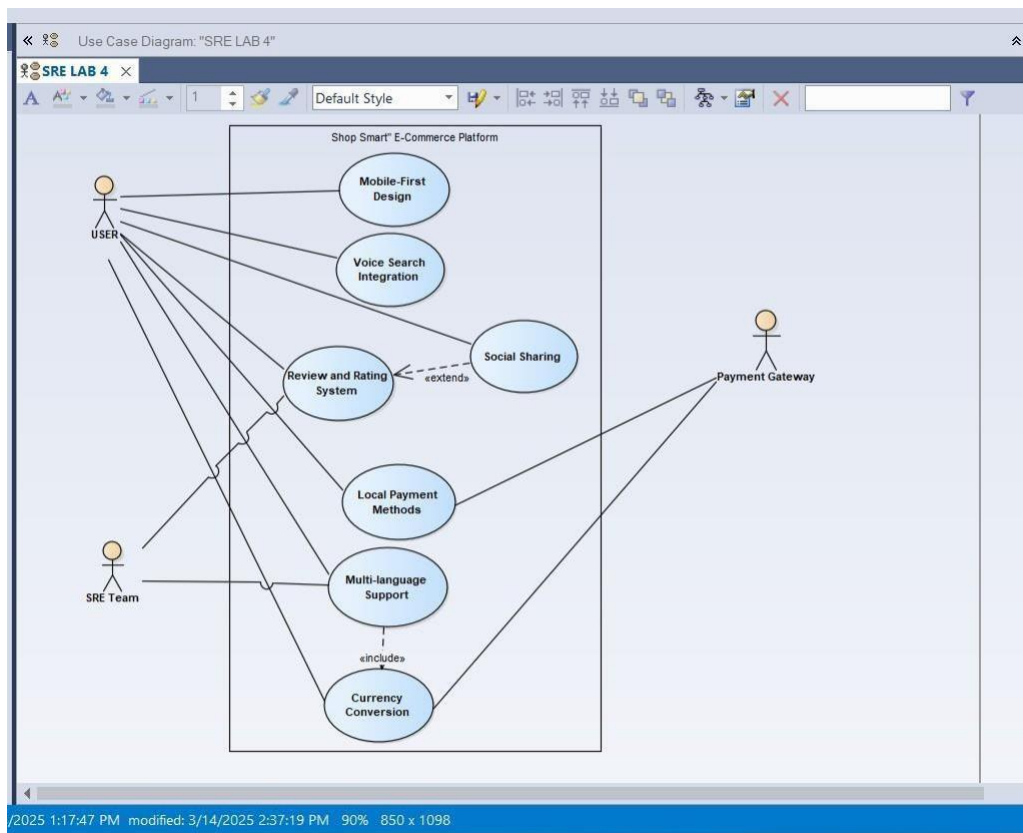
4. Connect Actors to Use Cases:

- a. Use the **Association** tool to connect actors to use cases.
- b. Example:
 - i. Connect **User** to all use cases.
 - ii. Connect **SRE Team** to **Review and Rating System** and **Multi-language Support**.
 - iii. Connect **Payment Gateway** to **Local Payment Methods** and **Currency Conversion**.



5. Add Relationships Between Use Cases:

- a. Use the **Include** or **Extend** relationships where applicable.
- b. Example:
 - i. **Multi-language Support** <<include>> **Currency Conversion** (since currency conversion is part of supporting multiple languages).
 - ii. **Review and Rating System** <<extend>> **Social Sharing** (since users can share reviews on social media).



Provide a brief explanation of how human, social, and cultural factors were incorporated into the use cases.

1. Mobile-First Design

- **Human Factor:** Added to address the preference for mobile usage, especially in regions like Asia where mobile devices are the primary way people access the internet.
- **Explanation:** The platform is optimized for smaller screens, with large buttons, easy navigation, and fast loading times to ensure a smooth mobile shopping experience.
- **Example:** A user in India can easily browse and purchase products on their smartphone without needing a desktop computer.

2. Voice Search Integration

- **Human Factor:** Added to improve accessibility and usability for all users, including those who prefer voice commands over typing.
- **Explanation:** Voice search allows users to find products quickly and efficiently, especially for users with disabilities or those who are multitasking.
- **Example:** A user can say, "Find wireless headphones," and the platform will display relevant results.

3. Review and Rating System

- **Social Factor:** Added to build trust and encourage social engagement by allowing users to share their opinions.
- **Explanation:** Users can leave reviews and ratings for products, which helps others make informed decisions and fosters a sense of community.
- **Example:** A user reads reviews before purchasing a laptop to ensure it meets their needs.

4. Multi-language Support

- **Cultural Factor:** Added to accommodate language diversity in regions like Europe and Asia.
- **Explanation:** The platform supports multiple languages (e.g., English, Spanish, French) to make it accessible to a global audience.
- **Example:** A user in France can browse the platform in French, while a user in Japan can use the platform in Japanese.

5. Local Payment Methods

- **Cultural Factor:** Added to meet regional payment preferences and ensure a seamless checkout experience.
- **Explanation:** The platform integrates local payment options to make transactions easier and more familiar for users.
- **Example:** A user in India can pay using UPI, while a user in the U.S. can use PayPal or credit cards.

6. Currency Conversion

- **Cultural Factor:** Added to provide a seamless shopping experience for international users by displaying prices in their local currency.
- **Explanation:** The platform automatically converts prices into the user's local currency, eliminating confusion and making it easier to shop.
- **Example:** A user in Germany sees prices in euros, while a user in Japan sees prices in yen.

7. Social Sharing

- **Social Factor:** Added to encourage users to share products and reviews on social media, increasing engagement and visibility.
- **Explanation:** Users can share product links or reviews on platforms like Facebook, Twitter, or Instagram, helping to promote the platform and its products.
- **Example:** A user shares a product they love on Instagram, and their followers click

the link to visit the platform.

Practice Task

Practice task 01

[expected time 20 min]

Practice Case Studies

Case Study 1: Healthcare App for Elderly Users

Scenario: A healthcare app is being developed to help elderly users manage their medications. The SRE team must ensure the app is reliable, easy to use, and accessible.

Task: Analyze the human, social, and cultural factors that could impact the app's requirements.

Steps to Analyze the Case Study

Identify Human Factors:

- a. How do elderly users interact with the app?
- b. Example: Large fonts, simple navigation, and voice commands.

Identify Social Factors:

- c. What social behaviors influence user expectations?
- d. Example: Family members may need access to monitor medication schedules.

Identify Cultural Factors:

- e. How do cultural differences impact system design?
- f. Example: Adapting the app for different languages and healthcare practices.

Model Requirements in Enterprise Architect:

- g. Create a **Use Case Diagram** that incorporates these factors.
- h. Example: Add use cases like "Medication Reminders," "Family Access," and "Multi-language Support."

Practice task 02

[expected time 20 min]

Case Study 2: Ride-Sharing App for a Diverse City

Scenario: A ride-sharing app is being launched in a diverse city with users from various cultural backgrounds. The SRE team must ensure the app is reliable, inclusive, and culturally sensitive.

Task: Analyze the human, social, and cultural factors that could impact the app's requirements.

Steps to Analyze the Case Study

1. **Identify Human Factors:**

- a. How do users interact with the app?
- b. Example: Users may prefer different payment methods or ride options.

2. **Identify Social Factors:**

- a. What social behaviors influence user expectations?
- b. Example: Users may expect safety features or driver ratings.

3. **Identify Cultural Factors:**

- a. How do cultural differences impact system design?
- b. Example: Adapting the app for different languages, cultural norms, and local regulations.

4. **Model Requirements in Enterprise Architect:**

- a. Create a **Use Case Diagram** that incorporates these factors.
- b. Example: Add use cases like “Multi-language Support,” “Safety Features,” and “Local Payment Methods.”

Deliverables

1. **Case Study Walkthrough:**

- a. Submit the Use Case Diagram for the global e-commerce platform.
- b. Include a brief explanation of how human, social, and cultural factors were incorporated.

2. **Practice Case Study 1:**

- a. Submit the Use Case Diagram for the healthcare app.
- b. Include a brief explanation of the factors analyzed.

3. **Practice Case Study 2:**

- a. Submit the Use Case Diagram for the ride-sharing app.
- b. Include a brief explanation of the factors analyzed.

Lab Manual for Software Requirement Engineering

Lab-06

Requirements Elicitation Process and Techniques:

Interviews, Surveys/Questionnaires

Lab Title: Requirements Elicitation Process and Techniques

Objective:

By the end of this lab, students will:

- Understand the importance of requirements elicitation in software development.
- Learn different elicitation techniques such as interviews, surveys, and workshops.
- Use Visual Paradigm to document the elicited requirements.
- Conduct a mock requirements elicitation session to apply their learning in a real-world

scenario.

- Encourage students to reflect on which elicitation technique was most effective.
- Discuss real-world challenges in gathering requirements.
- Highlight the importance of clear communication in the elicitation process.

● Relevant Lecture Material

a) Textbooks:

Managing Software Requirements: A Use Case Approach, Second Edition by
Dean Leffingwell, Don Widrig, Addison Wesley

b) Reference Material:

Requirements Engineering: From System Goals to UML Models to Software
Specifications, Axel V. Lamsweerde, (2009) ISBN-10: 0470012706

1. Activity Time Boxing:

ACTIVITY NAME	ACTIVITY TIME	TOTAL
Quiz	25 min	25 min
Setting Up and Setting	5min	5 min
Walkthrough Task	20 min	40 min
Practice Task	20 to 30 min for each task	60 min

2. Quiz

[Expected time 25 min]

3. Concept Map:

Requirements elicitation is a critical phase in software engineering that focuses on gathering and understanding the needs of stakeholders to define what the system must do. The process begins with recognizing the **importance of elicitation**, as it ensures that the final product aligns with user expectations and business goals. To achieve this, various **elicitation techniques** are employed, such as **interviews** (direct conversations with stakeholders to extract detailed requirements), **brainstorming** (collaborative sessions to generate ideas), **surveys/questionnaires** (structured tools to gather input from a large group), **observation** (watching users in their environment to understand workflows), and **prototyping** (creating early versions of the system to gather feedback). However, the elicitation process is not without challenges, including **ambiguous requirements** (unclear or vague needs), **stakeholder conflicts** (differing opinions among stakeholders), and scope creep (uncontrolled expansion of project scope). Once requirements are gathered, they must be carefully documented using tools like **Software Requirements Specification (SRS)** documents, use cases (descriptions of system interactions), and user stories (short, simple descriptions of features from the user's perspective). By mastering these concepts and techniques, students can effectively bridge the gap between stakeholder needs and system functionality.

Techniques for Requirements Elicitation

Introduce the following techniques with brief explanations and real-world examples:

- **Interviews:** One-on-one discussions with stakeholders to understand their needs.
- **Surveys/Questionnaires:** Structured forms to collect data from a large audience.
- **Workshops:** Collaborative sessions where multiple stakeholders discuss and refine requirements.
- **Observation:** Studying users' interactions with existing systems.
- **Prototyping:** Creating preliminary versions of the software to get early feedback.

Walk Through Task

Walkthrough task: E-commerce Website Case Study [expected time 30 min]

Case Study: Online Food Delivery System

You are a requirements engineer tasked with gathering requirements for a new Online Food Delivery System. The system will allow customers to order food from local restaurants, track their orders, and make payments online. The stakeholders include customers, restaurant owners, delivery personnel, and system administrators. Stakeholders and Their Roles

1. Customers:

- Primary users of the system who will place orders, track deliveries, and make payments.
- Key needs: Easy-to-use interface, multiple payment options, real-time order tracking, and personalized recommendations.

2. Restaurant Owners:

- Business owners who will manage their menus, receive orders, and update order statuses.
- Key needs: Efficient order management, notification system for new orders, and analytics to track sales performance.

3. Delivery Personnel:

- Individuals responsible for picking up orders from restaurants and delivering them to customers.
- Key needs: Route optimization, delivery notifications, and a way to communicate with customers or restaurants.

4. System Administrators:

- Technical staff responsible for maintaining the system, ensuring security, and managing user accounts.
- Key needs: System performance monitoring, user management, and data security features.

Task Breakdown

Step 1: Understand the Scenario (10 minutes)

- Read the scenario carefully and identify the key stakeholders and their roles.
- Discuss within your group the specific needs and challenges each stakeholder might have.

Step 2: Choose an Elicitation Technique (10 minutes)

- Each group will be assigned a stakeholder role and must select an appropriate elicitation technique:
 - **Group 1 (Customers):** Use brainstorming to generate ideas for customer-facing features.
 - **Group 2 (Restaurant Owners):** Create a survey to gather requirements from restaurant owners.
 - **Group 3 (Delivery Personnel):** Conduct a simulated interview to understand delivery personnel needs. (optional)
 - **Group 4 (System Administrators):** Use observation to identify system-level requirements.

Step 3: Conduct the Elicitation Activity (40 minutes)

- **Group 1 (Customers):**
 - Brainstorm features like order tracking, payment options, and restaurant filtering.
 - Document functional requirements (e.g., "The system must allow customers to filter restaurants by cuisine.").
- **Group 2 (Restaurant Owners):**
 - Design a survey with questions like:
 - *"What features would help you manage orders more efficiently?"*
 - *"How would you like to be notified of new orders?"*
 - Analyze the survey results to identify key requirements.
- **Group 3 (Delivery Personnel):**
 - Simulate an interview with questions like:

- *"What challenges do you face while delivering orders?"*

- *"What tools or features would make your job easier?"*

- Document requirements such as route optimization and delivery notifications.

- **Group 4 (System Administrators):**

- Observe a hypothetical system administration workflow to identify needs like:

- User account management.

- System performance monitoring.

- Data security measures.

Step 4: Document Requirements (30 minutes)

- Each group will organize their findings into:

- **Functional Requirements:** Specific features the system must have (e.g., "The system must allow customers to track their orders in real-time.").

- **Non-Functional Requirements:** Quality attributes of the system (e.g., "The system must

handle 1,000 concurrent users without performance degradation.").

- **Use Cases or User Stories:** Descriptions of how users will interact with the system

(e.g., *"As a customer, I want to filter restaurants by cuisine so that I can find food I like."*).

Expected Outcomes

1. A comprehensive list of requirements for **the Online Food Delivery System**, categorized by stakeholder.

2. A use case diagram or user stories that describe how the system will function.

3. A short report summarizing the elicitation process, techniques used, and challenges encountered.

Practice Task:

Practice task 01: Eliciting Requirements for a Library Management System [expected time 20 min]

Practice Task 1: Eliciting Requirements for a Library Management System

Scenario:

A local library wants to modernize its operations by developing a Library Management System. The system will help librarians manage books, members, and borrowing activities more efficiently. Students will act as requirements engineers and use elicitation techniques to gather requirements from stakeholders.

Practice Task 2: Eliciting Requirements for a Fitness Tracking App
[expected time 20 min]**Practice Task 2: Eliciting Requirements for a Fitness Tracking App****Scenario:**

A startup is developing a Fitness Tracking App to help users monitor their workouts, track progress, and stay motivated. Students will act as requirements engineers and gather requirements from potential users and other stakeholders.

Lab Manual for Software Requirement Engineering

Lab-07

Requirements Elicitation Process and Techniques: Workshops, Observation, Prototyping

Lab Title: Requirements Elicitation Process and

Techniques: Workshops, Observation, Prototyping

Objective:

By the end of this lab, students will:

- Understand the importance of requirements elicitation in software development.
- Learn different elicitation techniques such as interviews, surveys, and workshops.
- Use Visual Paradigm to document the elicited requirements.
- Conduct a mock requirements elicitation session to apply their learning in a real-world

Scenario.

- Encourage students to reflect on which elicitation technique was most effective.
- Discuss real-world challenges in gathering requirements.
- Highlight the importance of clear communication in the elicitation process.

• Relevant Lecture Material

a) Textbooks:

Managing Software Requirements: A Use Case Approach, Second Edition by
Dean Leffingwell, Don Widrig, Addison Wesley

b) Reference Material:

Requirements Engineering: From System Goals to UML Models to Software
Specifications, Axel V. Lamsweerde, (2009) ISBN-10: 0470012706

1. Activity Time Boxing:

ACTIVITY NAME	ACTIVITY TIME	TOTAL
Quiz	25 min	25 min
Setting Up and Setting	5min	5 min
Walkthrough Task	20 min	40 min
Practice Task	20 to 30 min for each task	60 min

2. Quiz

[Expected time 25 min]

3. Concept Map:

Techniques for Requirements Elicitation

1. • **Interviews:** One-on-one discussions with stakeholders to understand their needs.
2. • **Surveys/Questionnaires:** Structured forms to collect data from a large audience.
3. • **Workshops:** Collaborative sessions where multiple stakeholders discuss and refine requirements.

(discussed in the previous lab)

4. Observation: Studying User Interactions with Existing Systems

Description:

Observing users in their natural environment helps uncover implicit needs and pain points that may not be expressed in interviews or surveys.

Key Activities:

- Passive Observation: Silently watching users interact with a system.
- Active Observation: Asking users to think aloud while performing tasks.
- Contextual Inquiry: Combining observation with short interviews.

Lab Exercise:

- Assign students to observe a real or simulated user interaction (e.g., a student using a library portal).
- Record usability issues, workarounds, and inefficiencies.
- Compare observed behavior with stated requirements (if available).
- Document findings and suggest improvements.

5. Prototyping: Early Feedback through Preliminary Versions

Description:

Creating a simplified version of the software (e.g., wireframes, mockups, or clickable prototypes) to validate requirements before full development.

Key Activities:

- Low-Fidelity Prototypes: Sketches or paper prototypes for quick feedback.
- High-Fidelity Prototypes: Interactive digital mockups (e.g., using Figma, Balsamiq).
- User Testing: Presenting the prototype to stakeholders for feedback.

Lab Exercise:

- Provide a scenario (e.g., "Design a food delivery app").
- Students create a low-fidelity prototype (paper sketches or digital wireframes).
- Conduct peer reviews where students play the role of end-users and provide feedback.
- Refine the prototype based on feedback and document changes.

Walk Through Task

Walkthrough task: E-commerce Website Case Study [expected time 30 min]

Case Study: Online Food Delivery System (Continue)

Steps to Draw the UseCase Diagram:**Identify Actors:**

1. **Customer** – Places orders, tracks deliveries, and makes payments.
2. **Restaurant Staff** – Manages orders, updates menus, and confirms orders.
3. **Delivery Agent** – Picks up and delivers orders to customers.
4. **Admin** – Manages users, restaurants, and system settings.

Key Use Cases:

Actor	Use Cases
Customer	- Register/Login
	- Browse Restaurants & Menu
	- Add Items to Cart
	- Place Order
	- Make Payment (Online/Cash)
	- Track Order Status
	- Rate & Review Order
	- Restaurant Staff
	- Update Menu
Restaurant Staff	- Accept/Reject Orders
	- Notify Order Status
	- Delivery Agent
Delivery Agent	- Accept Delivery Request
	- Update Delivery Status
	- Mark Order as Delivered
Admin	- Manage Users (Customers, Restaurants, Delivery Agents)
	- Handle Complaints & Refunds
	- Generate Reports

Steps to Create a Use Case Diagram in Visio / Draw.io

1. Open Visio or Draw.io

- **Visio:** Launch Microsoft Visio → Select **Software & Database** → Choose **UML Model Diagram**.
- **Draw.io:** Go to app.diagrams.net → Select **Blank Diagram** → Choose **UML** under the "Shapes" panel.

2. Add Actors

- In the **Shapes** panel, find **UML Use Case** (Visio) or **UML** (Draw.io).
- Drag the **Actor** symbol (stick figure) onto the canvas for each role:
 - Customer
 - Restaurant Staff
 - Delivery Agent
 - Admin

3. Define Use Cases

- Drag the **Use Case** oval shape onto the canvas for each action:
 - **Customer:** Register, Browse Menu, Place Order, Track Order, etc.
 - **Restaurant Staff:** Update Menu, Accept Order, etc.
 - **Delivery Agent:** Accept Delivery, Update Status, etc.
 - **Admin:** Manage Users, Generate Reports, etc.

4. Connect Actors to Use Cases

- Use the **Association** line (solid line) to link actors to their use cases.
 - Example:
 - **Customer → Place Order**
 - **Restaurant Staff → Accept Order**

5. Add System Boundaries (Optional)

- Draw a **rectangle** around related use cases to group them (e.g., "Order Management System").

6. Include Relationships Between Use Cases

- **Extend/Include Relationships** (for complex flows):

- **Include** (dashed line + <<include>>): Mandatory dependency (e.g., "Place Order" **includes** "Make Payment").
- **Extend** (dashed line + <<extend>>): Optional flow (e.g., "Rate Delivery" **extends** "Track Order").

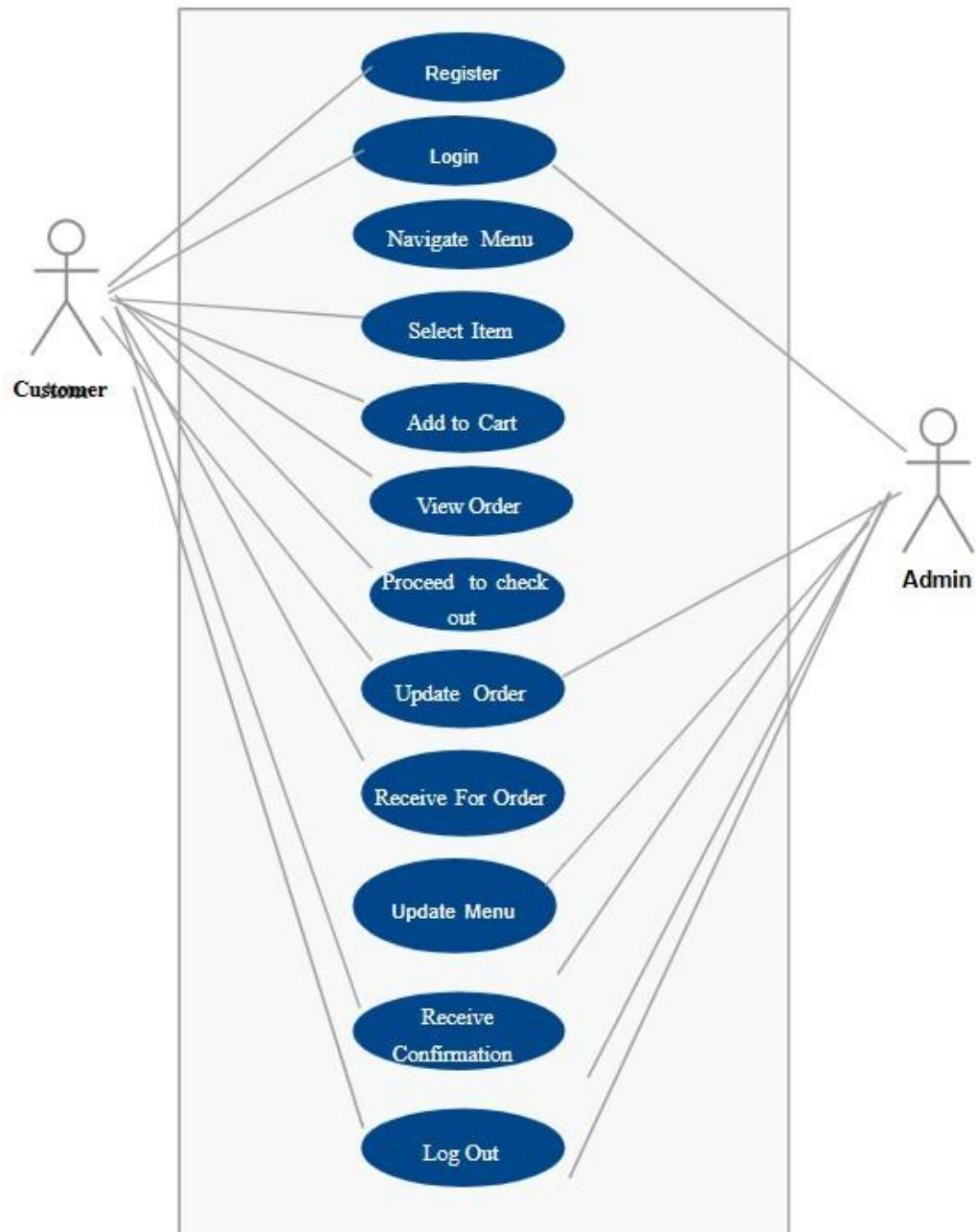
7. Label and Format

- Right-click elements to add **text labels**.
- Adjust colors/fonts via the **Format Panel** (Visio) or **Style Tab** (Draw.io).

8. Save & Export

- **Visio**: Save as .vsdx or export as .png/.pdf.
- **Draw.io**: Export via **File** → **Export as** → **PNG/PDF**.

Online Food Ordering System



(Document can be modified further)

Practice Task:

Lab Task1: Prototyping for "FeastExpress" – A Hyper-Local Food Delivery App

Scenario:

A new startup, **FeastExpress**, wants to launch a food delivery app targeting **small local restaurants** (not chains). Their unique selling point is "**10-minute prep time guarantee**" for partner restaurants.

However, after initial interviews, the team realizes:

- **Customers** complain about **unclear delivery ETAs**.
- **Restaurants** struggle with **last-minute order cancellations**.
- **Delivery riders** find navigation inefficient.

Design a **low-fidelity prototype** to address these issues and validate it with users.

Expected Deliverables

1. **Hand-drawn or digital prototype** (before/after testing).
2. **Short report** (200 words) summarizing:
 - The pain point addressed.
 - User feedback received.
 - Changes made to the design.

Lab Task 2 :Eliciting Requirements for "FitTrack+" – A Smart Fitness Companion

Scenario:

A startup is developing **FitTrack+**, a fitness app targeting **casual gym-goers and home exercisers**. The app aims to:

- Track workouts, heart rate, and calories.
- Suggest personalized routines.
- Integrate with wearables (e.g., smartwatches).

After initial interviews, key issues emerge:

- **Users** forget to log workouts manually.
- **Trainers** want to monitor client progress.
- **Gyms** struggle with app compatibility.

Task: Use **two elicitation techniques** (workshop + prototyping) to refine requirements.

Lab Manual for Software Requirement Engineering

Lab-08

Requirements Analysis & Negotiation

Lab Title: Requirements Analysis & Negotiation

Objective:

By the end of this lab, students will:

- Understand the importance of requirements analysis & negotiation in software development.
- Learn how to identify, analyze, and refine software requirements.
- Gain hands-on experience in negotiating conflicting requirements among stakeholders.
- Use Visual Paradigm to document analyzed and negotiated requirements.
- Encourage students to reflect on the challenges of negotiating requirements.
- Discuss how real-world software projects handle conflicting stakeholder needs.
- Emphasize that compromise & clear documentation are key in requirement analysis.
- Relevant Lecture Material

a) Textbooks:

Managing Software Requirements: A Use Case Approach, Second Edition by
Dean Leffingwell, Don Widrig, Addison Wesley

b) Reference Material:

Requirements Engineering: From System Goals to UML Models to Software
Specifications, Axel V. Lamsweerde, (2009) ISBN-10: 0470012706

1. Activity Time Boxing:

ACTIVITY NAME	ACTIVITY TIME	TOTAL
Quiz	25 min	25 min
Setting Up and Setting	5min	5 min
Walkthrough Task	20 min	40 min
Practice Task	20 to 30 min for each task	60 min

2. Quiz [Expected time 25 min]

3. Concept Map:

1. Importance of Requirements Analysis & Negotiation

- Ensures that requirements are clear, complete, and consistent.
- Helps resolve conflicts between stakeholder needs.
- Prioritizes requirements to align with project goals and constraints.
- Provides a solid foundation for system design and development.

2. Requirements Analysis

- Clarity:
 - Ensures requirements are unambiguous and easy to understand.
 - Example: Avoid vague statements like "The system should be fast" and replace them with specific metrics like "The system must respond within 2 seconds."
- Completeness:
 - Verifies that all necessary requirements have been captured.
 - Example: Ensure all user roles (customers, restaurant owners, delivery personnel) have their needs addressed.
- Consistency:
 - Checks for conflicts or contradictions between requirements.
 - Example: Resolve conflicts like "Customers want low delivery fees" vs. "Restaurant owners want higher profits."

3. Requirements Negotiation

- Conflict Resolution:
 - Identifies and resolves conflicts between stakeholder requirements.
 - Techniques: Win-Win negotiation, voting, or compromise.
- Prioritization:
 - Ranks requirements based on importance and feasibility.
 - Techniques: MoSCoW Method (Must Have, Should Have, Could Have, Won't Have).
- Negotiation Techniques:
 - Win-Win: Finding solutions that satisfy all stakeholders.

- Voting: Using majority decisions to resolve conflicts.
- MoSCoW: Categorizing requirements to focus on critical needs first.

4. Documentation

- Finalized Requirements:
 - A clear, organized document listing all agreed-upon requirements.
 - Includes functional and non-functional requirements.
- Traceability Matrix:
 - Maps requirements back to their sources (stakeholders).
 - Ensures accountability and tracks the origin of each requirement.

Example Requirement:

"The system should provide quick and easy access to user accounts."

Issues:

- What does "quick and easy" mean? (Subjective)
- How will access be provided? (Unclear method)
- Is there a time limit for access speed?

Revised Requirement:

"The system should allow users to log in within 2 seconds using a username and password authentication."

Walk Through Task

Walkthrough task: Online Food Delivery System [expected time 30 min]

Case Study: Online Food Delivery System

You are requirements engineer tasked with gathering requirements for a new Online Food Delivery System. The system will allow customers to order food from local restaurants, track their orders, and make payments online. The stakeholders include customers, restaurant owners, delivery personnel, and system administrators. Stakeholders and Their Roles

Case Study: Eliciting Requirements for a Library Management System

Scenario:

A local library wants to modernize its operations by developing a Library Management System. The system will help librarians manage books, members, and borrowing activities more efficiently. Students will act as requirements engineers and use elicitation techniques to gather requirements from stakeholders.

Task Breakdown

Task 1: Introduction to Requirements Analysis & Negotiation (30 minutes)

1. Lecture/Discussion (15 minutes):

- Explain the importance of requirements analysis and negotiation.
- Discuss the key aspects of requirements analysis: clarity, completeness, and consistency.
- Introduce negotiation techniques like Win-Win, Voting, and the MoSCoW

Method.

2. Concept Map Walkthrough (15 minutes):

- Present the concept map to students.
- Discuss how analysis and negotiation fit into the overall requirements engineering process.

Task 2: Scenario-Based Analysis Activity (1 hour)

Step 1: Review Gathered Requirements (15 minutes)

- Review the requirements gathered in the previous lab (Lab 5: Requirements Elicitation).
- Divide students into the same groups (Customers, Restaurant Owners, Delivery Personnel, System Administrators).

Step 2: Analyze Requirements (30 minutes)

- Each group will analyze their requirements for:
 - Clarity: Are the requirements clear and unambiguous?
 - Completeness: Are all necessary requirements captured?
 - Consistency: Are there any conflicts within the requirements?
- Document any issues or conflicts identified during the analysis.

Step 3: Identify Conflicts (15 minutes)

- Groups share their findings with the class.
- Highlight any conflicts between stakeholder requirements (e.g., customers want low delivery fees, while restaurant owners want higher profits).

Task 3: Requirements Negotiation Activity (1 hour)

Step 1: Prioritize Requirements (30 minutes)

- Use the MoSCoW Method to prioritize requirements:
 - Must Have: Critical requirements without which the system cannot function.
 - Should Have: Important but not critical requirements.
 - Could Have: Nice-to-have requirements that can be postponed.
 - Won't Have: Requirements that are out of scope for the current project.
- Each group prioritizes their requirements and presents their findings.

Step 2: Resolve Conflicts (20 minutes)

- Facilitate a negotiation session where groups discuss and resolve conflicts.
- Use techniques like Win-Win Negotiation (finding solutions that satisfy all stakeholders) or Voting (to decide on contentious issues).

Step 3: Finalize Requirements (10 minutes)

- Combine the prioritized and conflict-free requirements into a single list.

Task 4: Documentation and Presentation (30 minutes)

Step 1: Document Finalized Requirements (15 minutes)

- Each group contributes to a Finalized Requirements Document for the Online Food Delivery System.
- Include:
 - Functional and non-functional requirements.
 - Use cases or user stories.

Step 2: Present Findings (15 minutes)

- Groups present their finalized requirements and explain how conflicts were resolved.
- Discuss the prioritization process and the rationale behind the final decisions.

Lab Deliverables

1. A Finalized Requirements Document for the Online Food Delivery System.
(prepare document according to the given template)
2. A short report summarizing the analysis, negotiation process, and final decisions.

Practice Task 1

Activity 1: Identifying Issues in a Sample Requirement Specification

- Provide students with a sample Software Requirements Specification (SRS) containing:
 - Ambiguous statements
 - Conflicting requirements
 - Incomplete details
- Students work in pairs to identify problems and suggest improvements.

Example Requirement:

"The system should provide quick and easy access to user accounts."

Issues:

- What does "quick and easy" mean? (Subjective)
- How will access be provided? (Unclear method)
- Is there a time limit for access speed?

Revised Requirement:

"The system should allow users to log in within 2 seconds using a username and password authentication."

Practice Task 2: Stakeholder Role-Play & Requirement Negotiation

1. Form groups of 3–4 students.
2. Assign each student a stakeholder role (e.g., Client, Developer, Project Manager, End-User).
3. Provide a scenario:
 - A university wants an online learning management system (LMS) with live classes, offline downloads, and student progress tracking.
4. Each stakeholder presents their requirements & concerns:
 - Client: Wants all features quickly.
 - Developer: Says live classes require more development time.
 - Project Manager: Concerned about budget constraints.
 - End-User: Wants a simple, fast interface.
5. Students discuss & negotiate which requirements are feasible, prioritized, or postponed.

Practice Task 3: Documenting Finalized Requirements in Visual Paradigm

1. Open Visual Paradigm and create a Requirement List.
2. Use Requirement Attributes to:
 - Categorize requirements (Functional, Non-Functional).
 - Assign priorities (High, Medium, Low),
3. Create a Use Case Diagram showing how users interact with the system.

Capital University of Science & Technology
Islamabad
Department of Software Engineering,
Faculty of Computing

**Lab Manual for Software Requirement
Engineering**

Lab-11: Identifying and Correcting Requirement Errors

Lab Title: Identifying and Correcting Requirement Errors

Objective:

The objective of this lab is to:

- Develop the ability to identify common types of requirement errors in software requirement specifications (SRS).
- Understand the impact of requirement errors on software development.
- Practice techniques to correct and refine requirements to ensure clarity, completeness, and consistency.
- Enhance critical thinking and analytical skills in evaluating software requirements.

● Relevant Lecture Material

a) Textbooks:

Managing Software Requirements: A Use Case Approach, Second Edition by Dean Leffingwell, Don Widrig, Addison Wesley

b) Reference Material:

Requirements Engineering: From System Goals to UML Models to Software Specifications, Axel V. Lamsweerde, (2009) ISBN-10: 0470012706

1. Activity Time Boxing:

ACTIVITY NAME	ACTIVITY TIME	TOTAL
Quiz	25 min	25 min
Setting Up and Setting	5min	5 min
Walkthrough Task	20 min	40 min
Practice Task	20 to 30 min for each task	60 min

2. Quiz

[Expected time 25 min]

3. Concept Map:

The concept map for "Identifying and Correcting Requirement Errors" revolves around understanding the types of requirement errors, their impact on software development, and the techniques to identify and correct them. Requirement errors can manifest in various forms, such

as ambiguity, incompleteness, inconsistency, incorrect assumptions, and unverifiable requirements. These errors can have significant consequences, including increased development costs, project delays, and poor software quality. To address these issues, several techniques can be employed, such as peer reviews, prototyping, requirement traceability, and checklists, to identify errors effectively. Once identified, errors can be corrected by refining requirements, clarifying ambiguities, resolving conflicts, and validating the corrected requirements with stakeholders. This structured approach ensures that the software requirements are clear, complete, and consistent, ultimately leading to a successful software development process.

● Requirement Errors:

○ Types of Errors:

- **Ambiguity:** Unclear or vague requirements that can be interpreted in multiple ways.
- **Incompleteness:** Missing details or information in the requirements.
- **Inconsistency:** Contradictory or conflicting requirements.
- **Incorrect Assumptions:** Requirements based on incorrect or unrealistic assumptions.
- **Unverifiable Requirements:** Requirements that cannot be tested or validated.

○ Impact of Errors:

- **Increased Development Costs:** Errors lead to rework and additional resources.
- **Project Delays:** Time spent fixing errors can delay the project timeline.
- **Poor Software Quality:** Errors result in a product that does not meet user needs.

● Techniques to Identify Errors:

- **Peer Reviews:** Team members review requirements to identify errors.
- **Prototyping:** Creating a mock-up to validate requirements.
- **Requirement Traceability:** Tracking requirements to ensure they are met throughout the development process.
- **Checklists:** Using predefined lists to ensure all aspects of requirements are covered.

● Techniques to Correct Errors:

- **Refining Requirements:** Improving the clarity and detail of requirements.
- **Clarifying Ambiguities:** Resolving vague or unclear requirements.
- **Resolving Conflicts:** Addressing contradictory requirements.

- **Validating with Stakeholders:** Ensuring corrected requirements align with stakeholder needs.

Walk Through Task

Walkthrough task: Online Food Delivery System

[expected time 30 min]

You are provided with a sample Software Requirements Specification (SRS) document for an **Online Food Delivery System**. The document contains several requirement errors. Your task is to:

1. **Identify** the errors in the requirements.
2. **Classify** the errors based on their type (ambiguity, incompleteness, inconsistency, etc.).
3. **Correct** the errors by rewriting the requirements to make them clear, complete, and consistent.

Sample SRS Document (with Errors):

Below is an excerpt from the SRS document for the **Online Food Delivery System**:

1. **Functional Requirement 1:**
"Users should be able to order food."
2. **Functional Requirement 2:**
"The system should show restaurants."
3. **Non-Functional Requirement 1:**
"The system should be user-friendly."
4. **Functional Requirement 3:**
"The system should allow users to pay for their orders."
5. **Non-Functional Requirement 2:**
"The system should be reliable."

Task Steps:

1. **Identify Errors:**
Analyze the sample SRS document and list the errors in each requirement. For example:
 - **Ambiguity:** What does "user-friendly" mean in Non-Functional Requirement 1?
 - **Incompleteness:** Functional Requirement 1 does not specify how users can order food (e.g., selecting items, specifying delivery address, etc.).
 - **Inconsistency:** Functional Requirement 2 does not specify how restaurants are displayed (e.g., by location, cuisine, ratings, etc.).
 - **Unverifiable:** Non-Functional Requirement 2 does not define what "reliable" means or how it will be measured.
2. **Classify Errors:**
Categorize each error based on its type (ambiguity, incompleteness, inconsistency, etc.).

3. **Correct Errors:**

Rewrite the requirements to eliminate errors and make them clear, complete, and consistent.

Expected outcome:

1. **Corrected Functional Requirement 1:**

"The system should allow users to browse restaurants, select food items, specify delivery address and time, and place an order."

2. **Corrected Functional Requirement 2:**

"The system should display a list of restaurants based on the user's location, sorted by cuisine type, ratings, and delivery time."

3. **Corrected Non-Functional Requirement 1:**

"The system should have an intuitive user interface with clear navigation, minimal clicks to place an order, and accessibility features for users with disabilities."

4. **Corrected Functional Requirement 3:**

"The system should allow users to pay for their orders using multiple payment methods, including credit/debit cards, digital wallets, and cash on delivery."

5. **Corrected Non-Functional Requirement 2:**

"The system should have an uptime of 99.9%, with a maximum response time of 2 seconds for 95% of requests under normal load conditions."

Practice task 01: Case Study – Healthcare System

[expected time 30 min]

Background:

Health Track Pro is a new software system being developed for a chain of fitness centers. The system aims to manage member registrations, class bookings, trainer schedules, and health metrics tracking. The project team has gathered initial requirements from stakeholders, but during the early stages of development, inconsistencies and ambiguities have been identified, leading to rework and delays.

The following is an excerpt from the requirements document:

1. **Member Registration:**

- The system shall allow new members to register by providing personal details.
- Members should receive a confirmation email upon registration.

2. **Class Booking:**

- Members can book fitness classes online.
- The system shall prevent double bookings.

3. **Trainer Schedule Management:**

- Trainers can set their available time slots.
- The system shall notify trainers when a class is booked in their schedule.

4. **Health Metrics Tracking:**

- Members can log their weight, heart rate, and workout duration.
- The system shall display progress reports.

5. **Performance & Security:**

- The system should be fast.
- User data must be secure.

Task:

Identify Requirement Errors:

Classify Errors:

Categorize the errors based on their type (ambiguity, incompleteness, inconsistency, etc.).

Propose Corrective Measures:

Rewrite the requirements to address the errors and ensure the system meets user needs

Practice Task 2:

[expected time 20 min]

Case Study – Learning Management System

Background:

EduManage360 is a Learning Management System (LMS) being developed for a university to streamline course management, student enrollment, assignment submissions, and grading. The

initial requirements were gathered from faculty, students, and administrators, but the development team is encountering inconsistencies, missing details, and conflicting expectations.

The following is an excerpt from the requirements document:

1. User Authentication:

- Users (students, instructors, admins) must log in to access the system.
- Passwords should be secure.

2. Course Enrollment:

- Students can enroll in courses online.
- The system should prevent over-enrollment in courses.

3. Assignment Submission:

- Students can submit assignments before the deadline.
- Instructors can grade assignments.

4. Notification System:

- Users should receive alerts for important updates.

5. Reporting & Analytics:

- Admins should be able to generate reports.

Task:

Identify Requirement Errors:

Classify Errors:

Categorize the errors based on their type (ambiguity, incompleteness, inconsistency, etc.).

Propose Corrective Measures:

Rewrite the requirements to address the errors and ensure the system meets user needs

Practice Task 3:

[expected time 20 min]

Case Study– Smart Home Automation System

Background

You are a **business analyst** working on a **Smart Home Automation System**. The system will allow users to control lights, thermostats, and security cameras via a mobile app. Stakeholders have provided the following initial requirements:

1. **R1:** The system shall allow users to turn lights on/off remotely.
2. **R2:** The system shall adjust the thermostat based on the user's preferences.
3. **R3:** The system shall send alerts if a security camera detects motion.
4. **R4:** The system shall support voice commands.
5. **R5:** The system shall be accessible from anywhere.
6. **R6:** The system shall be secure against unauthorized access.

Tasks:

Identify Requirement Errors:

Classify Errors:

Categorize the errors based on their type (ambiguity, incompleteness, inconsistency, etc.).

Propose Corrective Measures:

Rewrite the requirements to address the errors and ensure the system meets user needs

Capital University of Science & Technology
Islamabad
Department of Software Engineering,
Faculty of Computing

Lab 12 : Software Requirement Specification

Lab: Software Requirement Specification

Objective

This lab is designed to:

1. Develop students' ability to write **clear, detailed, and structured software requirement specifications** using industry best practices.
2. Provide hands-on experience with **Visual Paradigm** for modeling and documenting requirements in a professional SRS format.
3. Teach students to distinguish between **functional** and **non-functional requirements** and represent them effectively.
4. Emphasize the importance of **traceability, completeness, and ambiguity-free specifications** to prevent project failures.
5. Prepare students for real-world software engineering by simulating a collaborative requirements analysis and documentation process.

Key Focus Areas

- **Precision:** Eliminate vague language in requirements.
- **Organization:** Structure requirements logically (e.g., by modules or user roles).
- **Tool Proficiency:** Master Visual Paradigm for diagrams (use cases, workflows) and documentation.
- **Validation:** Learn peer-review techniques to refine requirements.

This objective aligns with **IEEE SRS standards** and equips students with skills directly applicable to industry workflows.

Tools and Resources

- **Tool:** Visual Paradigm (for requirements modeling and documentation)
- **Sample Project:** Choose one of the following:
 - **Online Bookstore System**
 - **Student Attendance Management System**
 - **Healthcare Appointment Booking App**

Lab Tasks

Task 1: Introduction to Requirement Specifications

1. Understand the key components of a **Software Requirements Specification (SRS)**:
 - a. **Functional Requirements** (what the system should do)
 - b. **Non-Functional Requirements** (performance, security, usability)
 - c. **Use Cases & User Stories** (interactions between users and the system)

- d. **Constraints & Assumptions** (limitations and dependencies)
2. Review **IEEE SRS Template** (or a simplified version) for structure.

Task 2: Create a Requirements Specification Document

Using **Visual Paradigm**, follow these steps:

Step 1: Define Project Scope

- **Example (Online Bookstore):**
 - *"The system shall allow users to browse, search, and purchase books online. Admins shall manage inventory and process orders."*

Step 2: List Functional Requirements

- Use **Visual Paradigm's Requirement Diagram** to document:
 - User registration/login
 - Book search and filtering
 - Shopping cart and checkout
 - Order tracking

Step 3: Define Non-Functional Requirements

- **Performance:** *"The system shall load search results in under 2 seconds."*
- **Security:** *"User passwords shall be encrypted using SHA-256."*
- **Usability:** *"The UI shall comply with WCAG accessibility standards."*

Step 4: Model Use Cases

- Create a **Use Case Diagram** in Visual Paradigm with:
 - **Actors** (Customer, Admin, Payment Gateway)
 - **Use Cases** (e.g., "Place Order," "Manage Inventory")
 - **Relationships** (<<include>>, <<extend>>)

Step 5: Add Constraints & Assumptions

- *"The system shall support only credit/debit card payments initially."*
- *"Assumes users have basic internet literacy."*

Task 3: Validate & Review

1. **Peer Review:** Exchange documents with another team and check for:
 - a. Clarity
 - b. Completeness

- c. Ambiguities
2. **Update SRS:** Refine based on feedback.

Deliverables

1. **SRS Document** (Exported from Visual Paradigm as PDF/Word)
 - a. Must include:
 - i. Project scope
 - ii. Functional & non-functional requirements
 - iii. Use case diagrams
 - iv. Constraints & assumptions
2. **Visual Paradigm Project File** (.vpp)

Assessment Criteria

Criteria	Marks
Clarity & Completeness	30%
Correct Use of Visual Paradigm	25%
Logical Requirement Grouping	20%
Peer Review Participation	15%
Adherence to SRS Structure	10%

Sample Output

Example: Online Bookstore SRS (Excerpt)

Functional Requirements

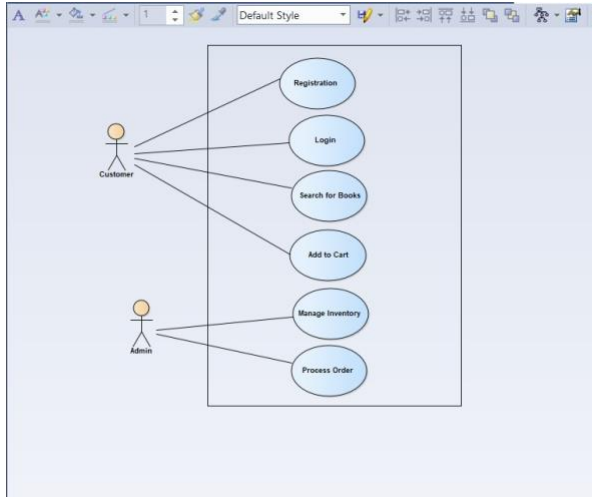
- **FR1:** The system shall allow users to search books by title, author, or genre.
- **FR2:** The system shall display book details (price, ratings, stock status).

Non-Functional Requirements

- **NFR1:** The system shall handle 1,000 concurrent users.

- **NFR2:** All transactions shall use HTTPS encryption.

Use Case Diagram



Practice Tasks

Practice task 01:

Case Study: Smart Campus Parking System

The University of Techville is facing significant parking challenges across its growing campus. With over 15,000 students, 2,000 faculty/staff members, and daily visitors, the current parking system leads to congestion, frustration, and wasted time. The administration wants to implement a Smart Campus Parking System that will leverage IoT technology and mobile applications to transform parking management. The system needs to provide real-time parking availability through sensors in all 25 campus lots, support digital payments and reservations, and integrate with the university's ID card system. Key challenges include reducing the average 12-minute search time for parking spots during peak hours, eliminating paper parking passes, preventing unauthorized parking in restricted zones, and ensuring accessibility for disabled users. The solution must serve diverse user groups including time-pressed students who need affordable options, faculty requiring guaranteed spots, visitors needing temporary access, and administrators who

require usage analytics and fraud detection capabilities. Security is a major concern, as the system must protect user data while detecting parking violations automatically. The university wants a phased implementation, starting with the main academic lot before expanding campus-wide, with particular attention to user-friendly design for non-tech-savvy users.

Generate the Software Requirement Specification Document for the Given Case Study

Practice task 02:

Case Study: Smart Retail Inventory Management System

A mid-sized retail chain with 35 stores across the region is struggling with outdated inventory management processes that lead to frequent stockouts, overstocking, and lost sales. The current manual system causes discrepancies between physical stock and digital records, resulting in inaccurate reordering and frustrated customers who often find advertised products unavailable. During peak seasons, these issues become particularly severe, with some stores losing up to 15% of potential sales due to inventory mismanagement. The company wants to implement an AI-powered Smart Inventory System that can automatically track stock levels in real-time using RFID technology, predict demand patterns, and generate optimized restocking alerts. The solution needs to integrate with their existing point-of-sale systems while being simple enough for store associates with varying technical skills to operate. Special consideration must be given to high-value items that are frequently stolen and seasonal products with unpredictable demand fluctuations. The system should also provide regional managers with consolidated inventory views across multiple locations to enable smarter distribution of goods between stores. With thin retail margins, the company emphasizes that the solution must demonstrate clear ROI through reduced shrinkage and improved sales conversion rates.

Capital University of Science & Technology
Islamabad
Department of Software Engineering,
Faculty of Computing

Lab 13 : Requirement Validation

Week 13 : Requirement Validation

Task No.	Activity Name	Activity time	Total Time
1	Quiz	30 min	30 min
2	Walkthrough Tasks	60 min	90min
3	Practice Tasks	50 As per time specified for each task	140 min
4	Evaluation Task	40 min for each assigned task	180 min

Requirement Validation

Requirements validation techniques are essential processes used to ensure that software requirements are complete, consistent, and accurately reflect what the customer wants. These techniques help identify and fix issues early in the development process, reducing the risk of costly errors later on. By thoroughly validating requirements, teams can ensure that the final product meets user needs and expectations. This article focuses on discussing the requirement validation technique in detail.

What is Requirements Validation?

Requirements validation is the process of **checking** that requirements defined for development, define the system that the customer wants. To check issues related to requirements, we perform requirements validation. We typically use requirements validation to check errors at the initial phase of development as the error may increase excessive rework when detected later in the development process. In the requirements validation process, we perform a different type of test to check the requirements mentioned in the Software Requirements Specification (SRS), these checks include:

1. **Completeness checks**
2. **Consistency checks**
3. **Validity checks**
4. **Realism checks**
5. **Ambiguity checks**
6. **Variability**

The output of requirements validation is the **list of problems and agreed-on actions of detected problems**. The lists of problems indicate the problem detected during the process of requirement validation. The list of agreed actions states the corrective action that should be taken to fix the detected problem.

Requirement Validation Techniques

There are several techniques that are used either individually or in conjunction with other techniques to check entire or part of the system:

1. Test Case Generation
2. Prototyping
3. Requirements Reviews
4. Automated Consistency Analysis
5. Walk-through
6. Simulation
7. Checklists for Validation

Advantages	Disadvantages
Improved Requirement Quality: Ensures requirements are clear, consistent, and unambiguous.	Time-Consuming Process: Can delay the start of development, especially for large projects.
Reduced Development Costs: Early error detection prevents costly rework later.	Increased Initial Costs: Requires resources (time, effort, tools) for thorough validation.
Enhanced Stakeholder Satisfaction: Ensures the final product meets stakeholder needs.	Stakeholder Availability: Stakeholders may not always be available or willing to participate.
Minimized Risk of Project Failure: Reduces the risk of building an irrelevant or flawed system.	Subjectivity in Validation: Conflicting stakeholder views can lead to disagreements or delays.
Better Communication: Fosters collaboration between stakeholders and developers.	Over-Reliance on Documentation: May not capture dynamic or evolving stakeholder needs.
Higher Accuracy in Estimates: Enables better project planning and resource allocation.	Potential for Over-Validation: Can lead to "analysis paralysis," delaying progress.
Compliance with Standards: Ensures requirements meet industry and regulatory standards.	Difficulty in Validating Non-Functional Requirements: Harder to validate (e.g., performance, scalability).
Early Detection of Issues: Identifies conflicts, gaps, or inconsistencies early.	Resistance to Change: Stakeholders or team members may resist necessary changes.

Walk through (Requirement Validation Using the Testcase generation)

Creating a simple SysML requirement diagram

1. Select **Project > New** from the application toolbar.
2. In the **New Project** window, enter *Tutorial* as **Name**.

New Project

Create New Project

Create a new project by filling in the following information.

Name: Tutorial

Author: Peter

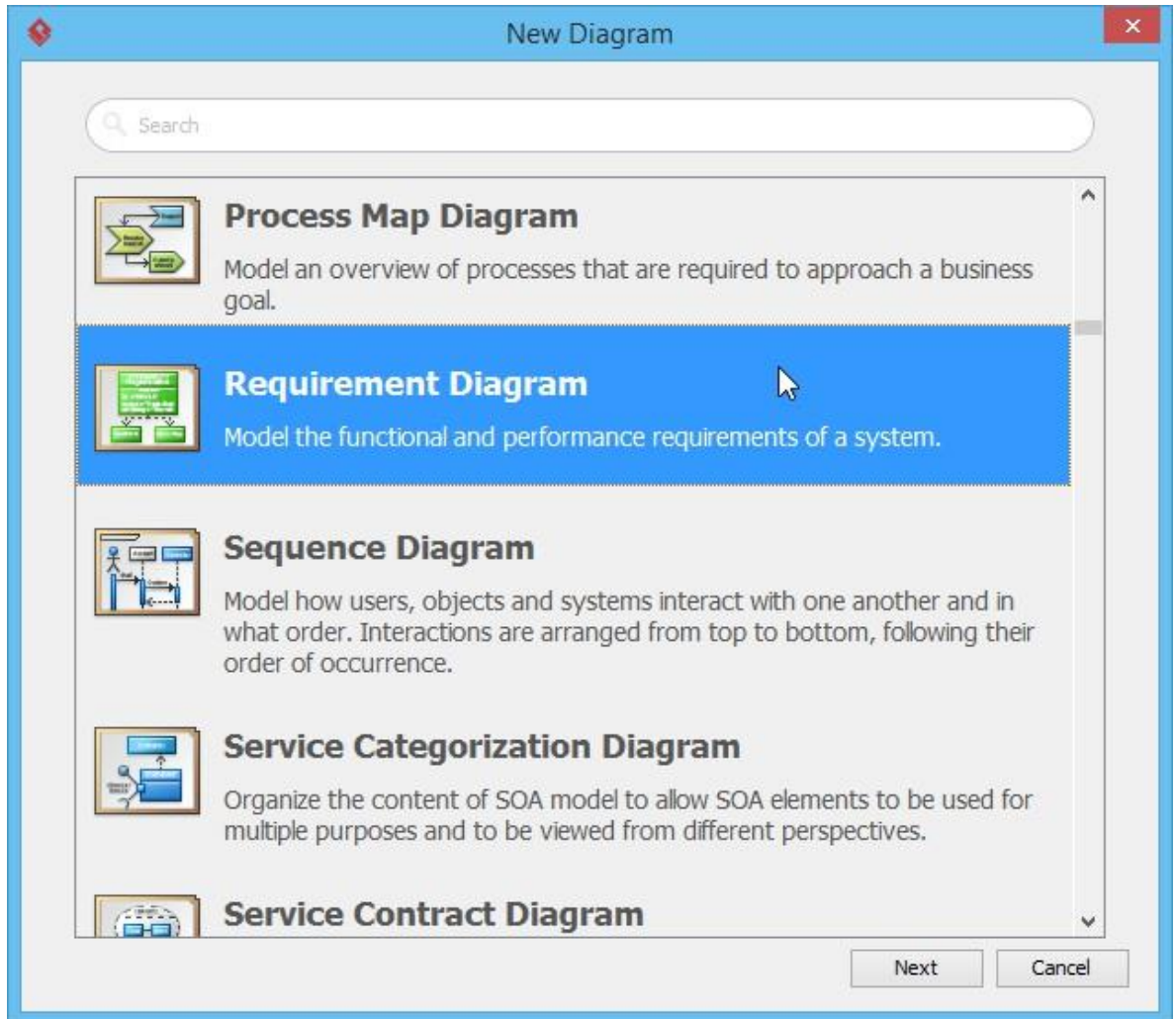
Data type set: UML

Description:

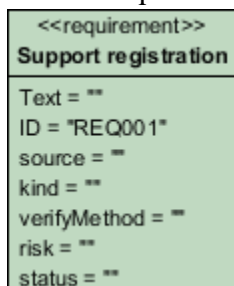
[Advanced Options](#)

Create Blank Project Cancel

3. Click **Create Blank Project**.
4. Create a SysML requirement diagram first. You can create a requirement diagram by selecting **Diagram > New** from the application toolbar. Select **Requirement Diagram** in the **New Diagram** window and then click **Next**. Click **OK** again to create the diagram.



5. Click a requirement *Support registration*.

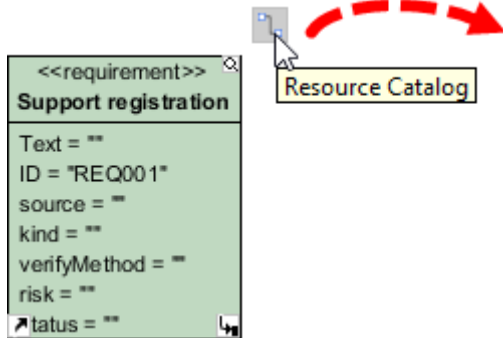


Creating test cases

Let's create some test cases for the requirement *Support registration*.

1. Move your mouse pointer over the requirement.

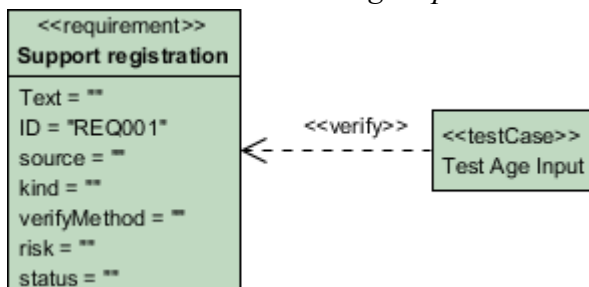
2. Press on the **Resource Catalog** icon and drag it out.



3. Release the mouse button. Select **Verify <- Test Case** from Resource Catalog.



4. Name the test case as *Test Age Input*



5. In addition to having a named test case shape, you can define the steps to test as well. Open the specification by right clicking on test case *Test Age Input* and selecting **Open Specification...** from the popup menu.
6. Open the tab **Test Plans**. There are three columns. The **Steps** column defines a set of steps to test the requirement in a more general angle. The **Procedures** column defines also the steps, but in a more concrete manner by providing also test data. The **Expected Results** column is the result expected to achieve by walking through the defined procedures.

7.

The screenshot shows a 'Test Case Specification' window with a 'Test Plans' tab. The window contains a toolbar with various icons and a table with three columns: 'Steps', 'Procedures', and 'Expected Results'. The 'Steps' column has a single entry '1.'. The bottom of the window has buttons for 'Reset', 'OK', 'Cancel', 'Apply', and 'Help'.

8. Enter the steps as shown below:

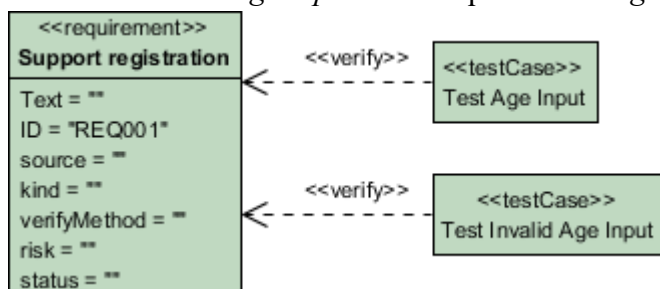
Step	Procedure	Expected Result
Enter user information	Enter: - User name: john - Age: 20	Entered data appear in form
Submit the form	Click the submit button	Prompt: Application is submitted

9. The screen should look like this:

Steps	Procedures	Expected Results
1. Enter user information	Enter: - User name: john - Age: 20	Entered data appear in form
2. Submit the form	Click the submit button	Prompt: Application is submitted

10. Click **OK** to confirm.

11. Create another test case to test invalid age input. Repeat the previous steps to create test case *Test Invalid Age Input* from requirement *Registration Form*.



12. Enter the test plan as shown below:

Step	Procedure	Expected Result
Enter user information	Enter: - User name: john - Age: -1	Entered data appear in form

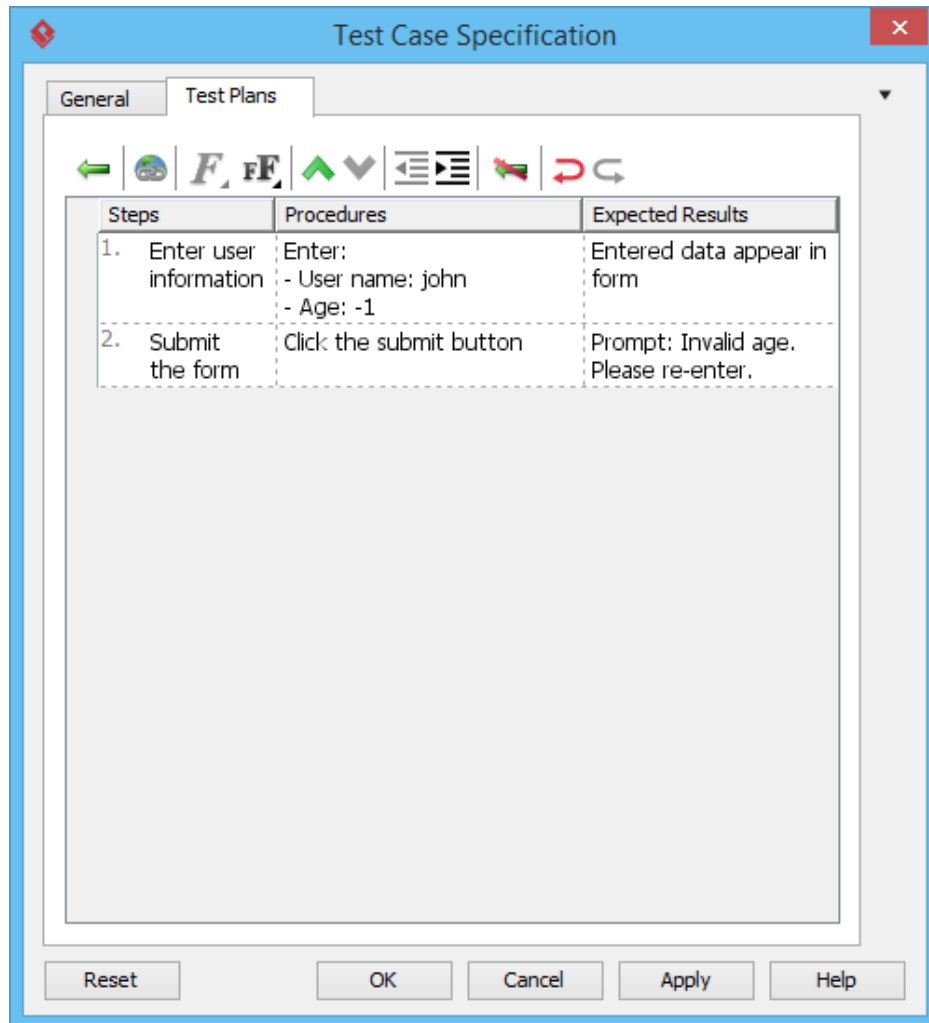
Submit the form

Click the submit button

Prompt: Invalid age.
Please re-enter.

13.

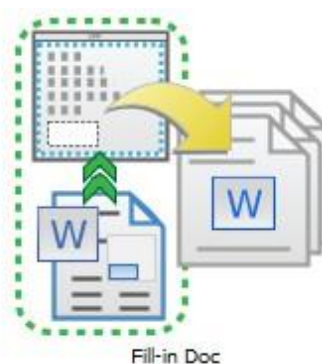
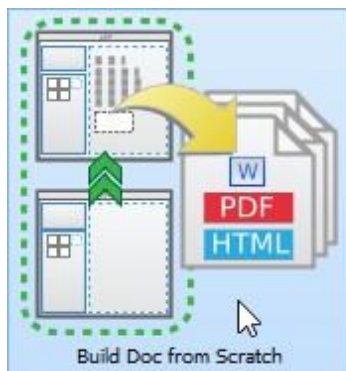
The screen should look like this:



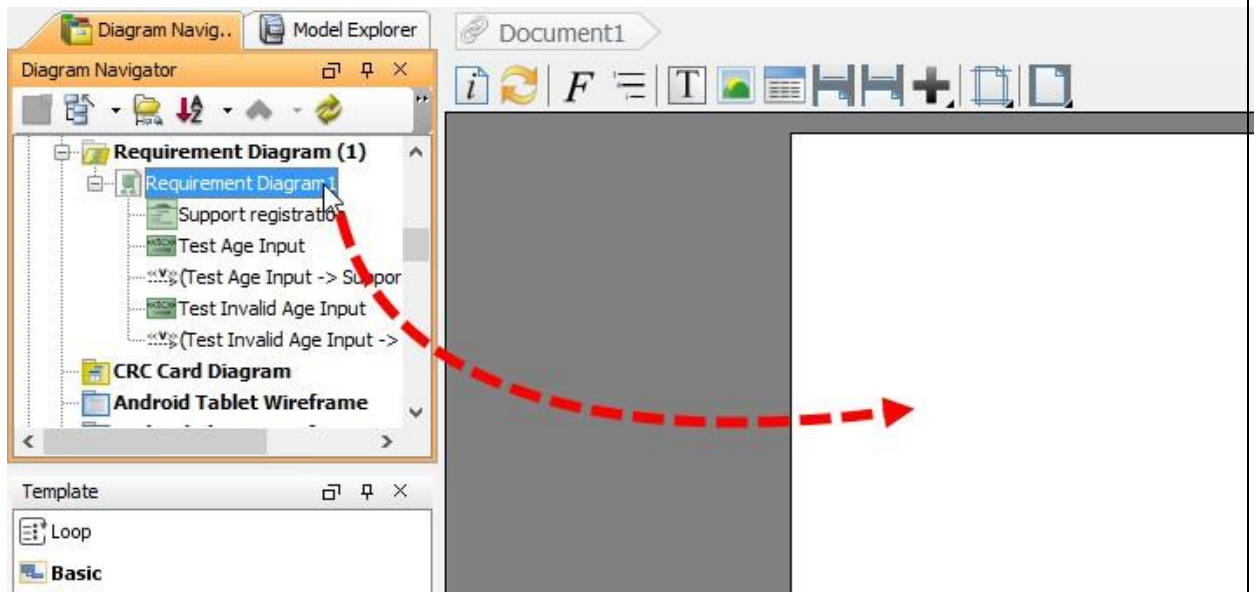
Producing a document of requirements and test cases

In this section you are going to produce a Word document that contains the details of requirements and test cases.

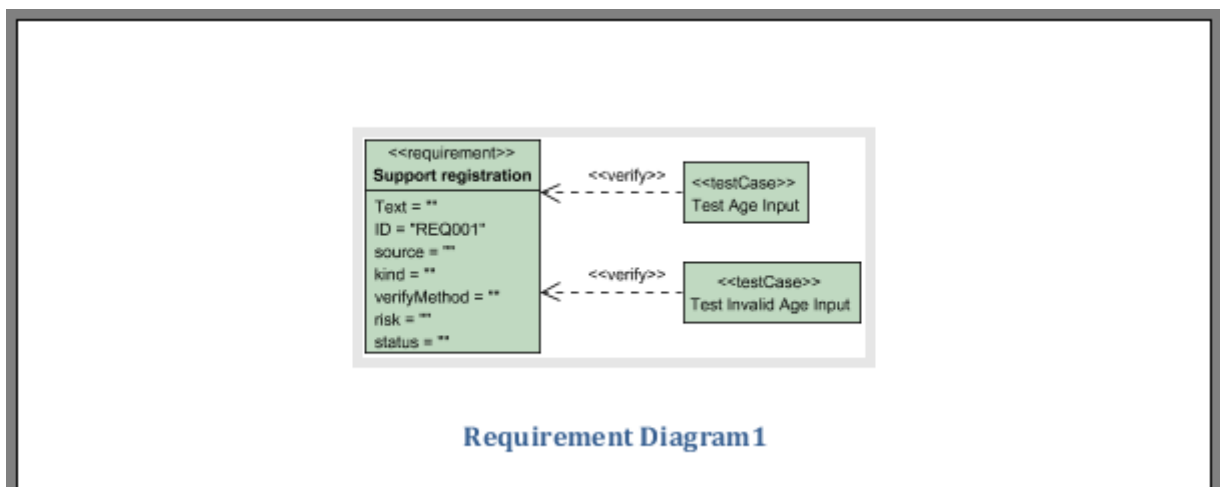
1. Select **Tools > Doc. Composer** from the application toolbar.
2. Select **Build Doc from Scratch**.



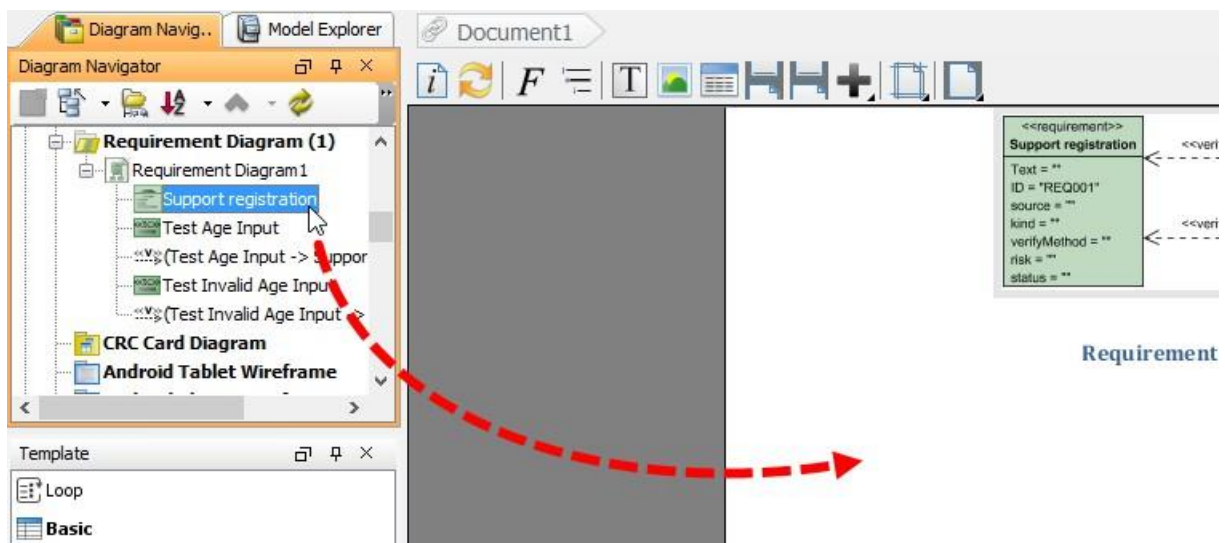
- Find the requirement diagram in **Diagram Navigator**. Press on the diagram and drag it onto the document.



Your document should look like this:



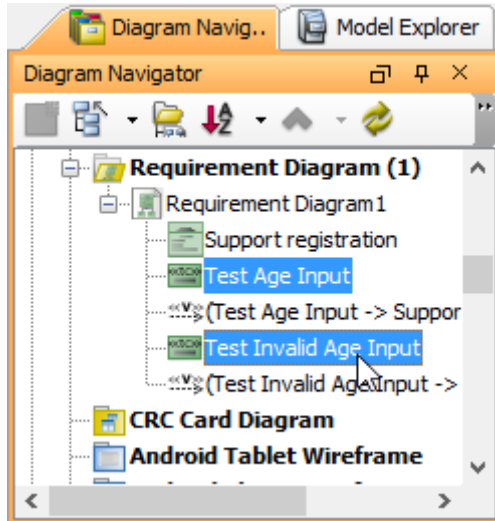
- Press on the requirement *Support registration* in **Diagram Navigator** and then drag it onto the document.



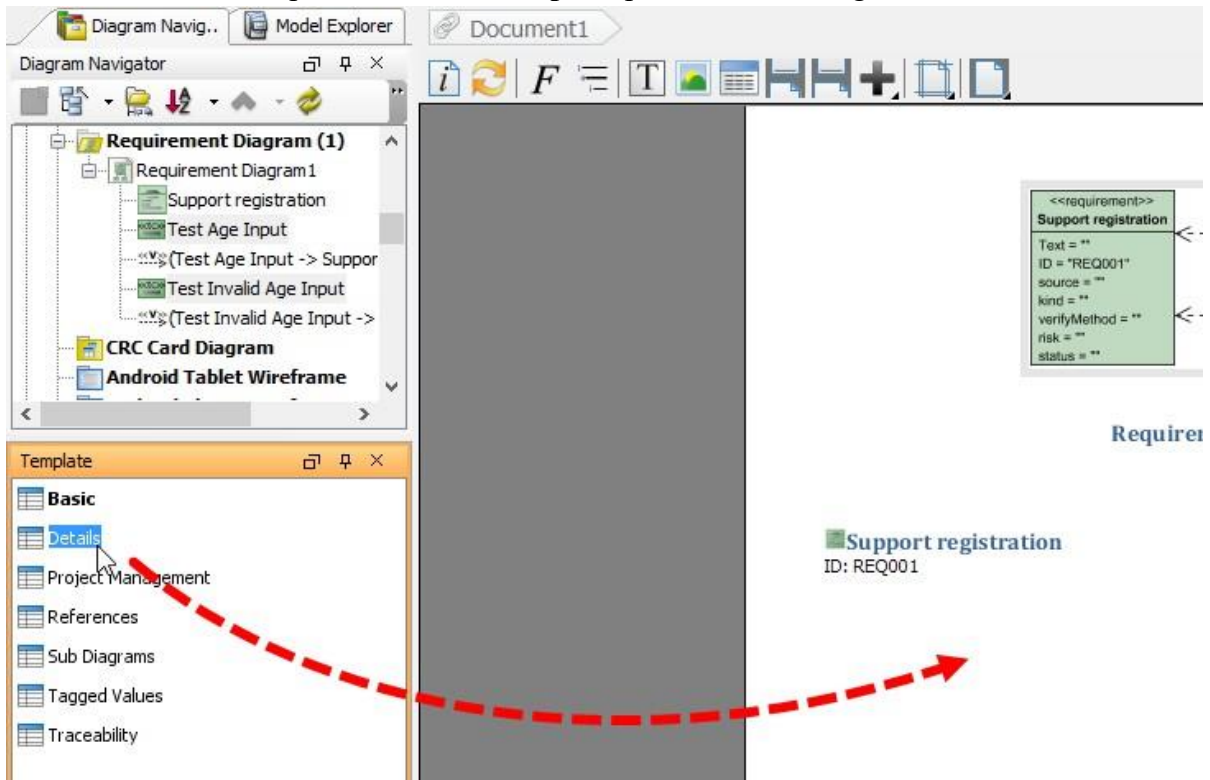
Your document should look like this:

Support registration
ID: REQ001

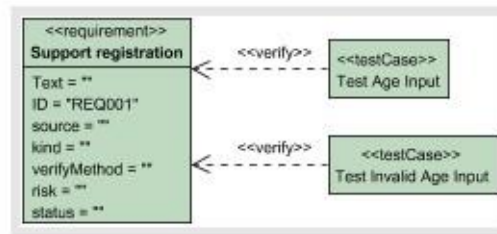
5. Select the two test cases in **Diagram Navigator**.



6. Select the **Details** template under the **Template** pane and then drag it onto the document.



Finally, your document should look like this:



Requirement Diagram 1

Support registration

ID: REQ001

Test Age Input

Test Plans

Steps	Procedures	Expected Results
1. Enter user information	Enter: - User name: john - Age: 20	Entered data appear in form
2. Submit the form	Click the submit button	Prompt: Application is submitted

Relationships

Relationship	From	To
unnamed	Test Age Input	Support registration

Test Invalid Age Input

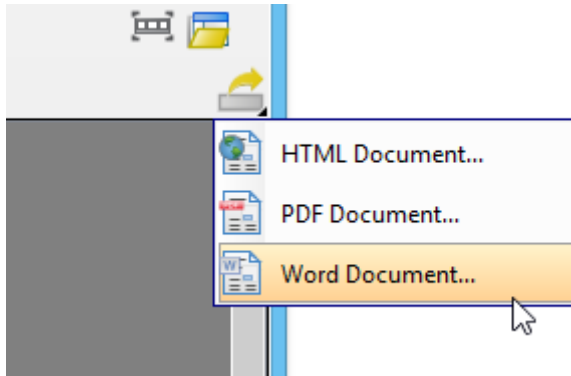
Test Plans

Steps	Procedures	Expected Results
1. Enter user information	Enter: - User name: john - Age: -1	Entered data appear in form
2. Submit the form	Click the submit button	Prompt: Invalid age. Please re-enter.

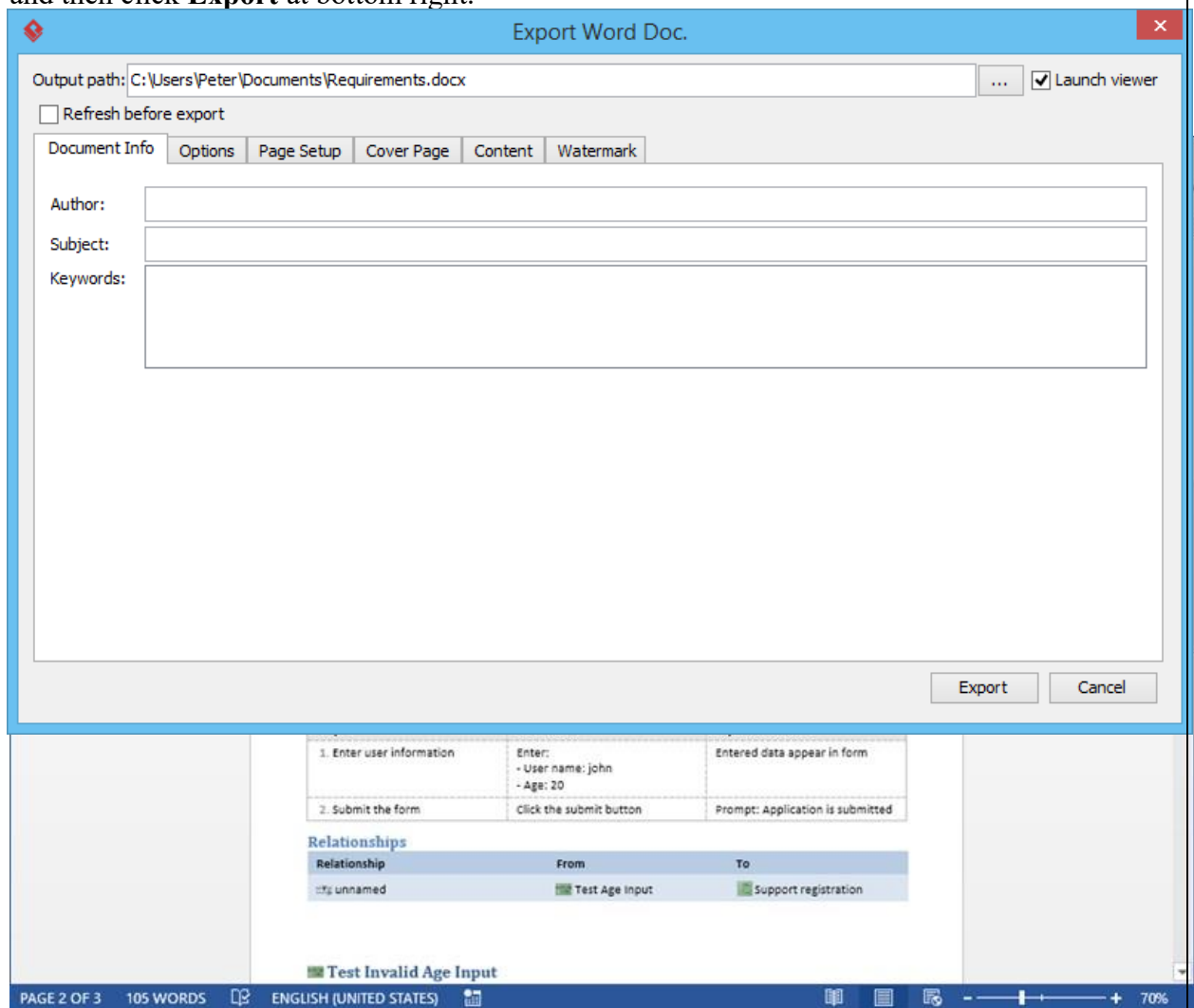
Relationships

Relationship	From	To
unnamed	Test Invalid Age Input	Support registration

7. Click on **Export > Word Document...** at the top right of Doc. Composer.



8. In the **Export Word Doc.** window, provide the output path of the .docx file to be exported and then click **Export** at bottom right.



Save the project. This is the end of the tutorial.

Practice Task: Test Case for the Library Management System

Objective:

Create a test case for the "**Borrow Book**" use case in the Library Management System. This test case will validate the functionality of a member borrowing a book from the library.

Test Case Specification

Test Case ID: TC_BORROW_BOOK_001

Test Case Title: Member Borrows a Book

Description:

This test case verifies that a member can successfully borrow a book from the library, and the system updates the borrowing record accordingly.

Preconditions:

1. The member is registered in the library system.
2. The book is available in the library catalog.
3. The member has not exceeded their borrowing limit.

Test Steps:

Step	Action	Expected Result
1	The member logs into the library system.	The member is successfully logged in.
2	The member searches for a book by title, author, or category.	The system displays a list of books matching the search criteria.
3	The member selects a book from the search results.	The system displays the book details (title, author, availability status).
4	The member clicks the "Borrow" button for the selected book.	The system checks the member's borrowing limit and the book's availability.
5	If the book is available and the member has not exceeded their borrowing limit, the system updates the borrowing record.	The system confirms the book has been borrowed and updates the member's borrowing history.
6	The system reduces the available quantity of the book by 1.	The book's availability status is updated in the catalog.
7	The system sets a due date for the book (e.g., 14 days from the borrowing date).	The due date is displayed to the member.
8	The system logs the transaction in the member's borrowing history.	The borrowing history is updated with the new transaction.

Test Data:

- **Member Details:**
 - Member ID: MEM123
 - Name: John Doe
 - Borrowing Limit: 5 books
 - Currently Borrowed: 2 books

- **Book Details:**

- Book ID: BK456
- Title: "Introduction to Software Engineering"
- Author: Jane Smith
- Available Quantity: 3

Postconditions:

1. The book is marked as borrowed in the system.
2. The member's borrowing history is updated.
3. The available quantity of the book is reduced by 1.
4. The due date for the book is set and displayed.

Expected Results:

1. The member can successfully borrow the book.
2. The system updates the borrowing record and reduces the book's available quantity.
3. The member's borrowing history reflects the new transaction.
4. The due date is correctly calculated and displayed.

Capital University of Science & Technology
Islamabad
Department of Software Engineering,
Faculty of Computing

Lab 14 : Requirement Modeling (Usecase)

Requirement Modeling

- Introduction to requirement modeling techniques (use case diagrams)
Exercise: Create use case and class diagrams for a sample project

1. Activity Time Boxing

Task No.	Activity Name	Activity time	Total Time
1	Quiz	30 min	30 min
2	Walkthrough Tasks	60 min	90min
3	Practice Tasks	50 As per time specified for each task	140 min
4	Evaluation Task	40 min for each assigned task	180 min

Objective of the Experiment

The core objective of this experiment is to understand the fundamental concepts of **Requirement Modeling** and to learn about what is a use case, its relationships, and how to create this in **Visual Paradigm**.

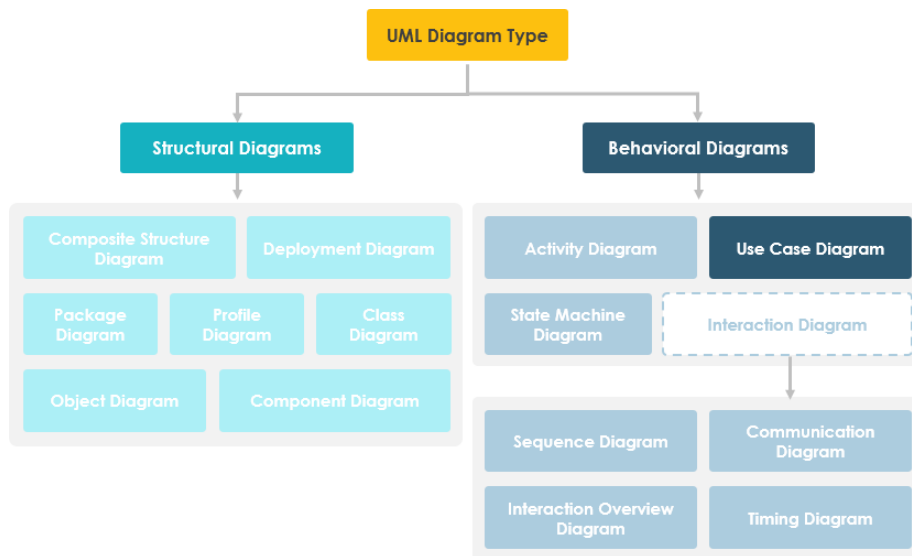
What is Use Case Diagram?

A [UML](#) use case diagram is the primary form of system/software requirements for a new software program underdeveloped. Use cases specify the expected behavior (what), and not the exact method of making it happen (how). Use cases once specified can be denoted both textual and visual representation (i.e. use case diagram). A key concept of use case modeling is that it helps us design a system from the end user's perspective.

A use case diagram is usually simple. It does not show the detail of the use cases:

- It only summarizes **some of the relationships** between use cases, actors, and systems.
- It does **not show the order** in which steps are performed to achieve the goals of each use case.

The figure below shows the UML diagram hierarchy and the positioning of the UML Use Case Diagram. As you can see, use case diagrams belong to the family of behavioral diagrams.



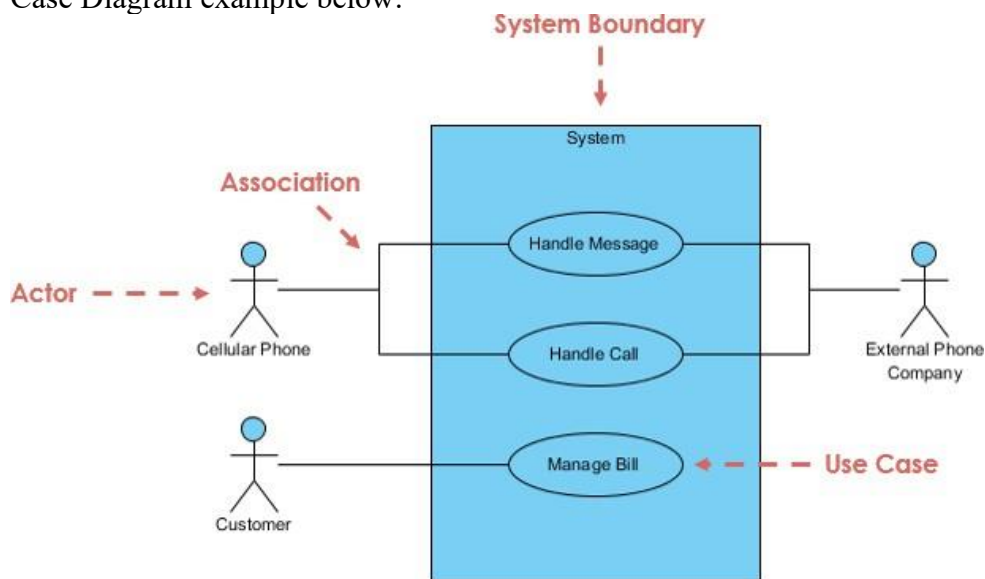
Purpose of Use Case Diagram

Use case diagrams are typically developed in the early stage of development and people often apply use case modeling for the following purposes:

- Specify the context of a system
- Capture the requirements of a system
- Validate a systems architecture
- Drive implementation and generate test cases
- Developed by analysts together with domain experts

Use Case Diagram at a Glance

A standard form of use case diagram is defined in the Unified Modeling Language as shown in the Use Case Diagram example below:



Actor

- Someone interacts with use case (system function).
- Named by noun.
- Actor plays a role in the business
- Similar to the concept of user, but a user can play different roles
- For example:
 - A prof. can be instructor and also researcher



- plays 2 roles with two systems
- Actor triggers use case(s).
- Actor has a responsibility toward the system (inputs), and Actor has expectations from the system (outputs).

Use Case

- System function (process - automated or manual)
- Named by verb + Noun (or Noun Phrase).
- i.e. Do something
- Each Actor must be linked to a use case, while some use cases may not be linked to actors.



Communication Link

- The participation of an actor in a use case is shown by connecting an actor to a use case by a solid link.
- Actors may be connected to use cases by associations, indicating that the actor and the use case communicate with one another using messages.

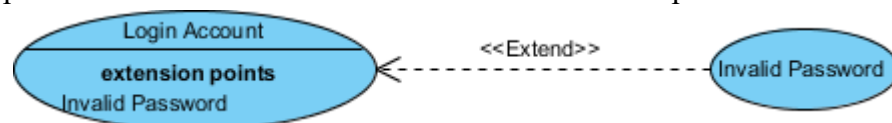
Boundary of system

- The system boundary is potentially the entire system as defined in the requirements document.
- For large and complex systems, each module may be the system boundary.
- For example, for an ERP system for an organization, each of the modules such as personnel, payroll, accounting, etc.
- can form a system boundary for use cases specific to each of these business functions.
- The entire system can span all of these modules depicting the overall system boundary

Use Case Relationship

Extends

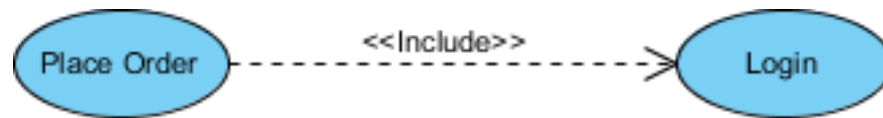
- Indicates that an **"Invalid Password"** use case may include (subject to specified in the extension) the behavior specified by base use case **"Login Account"**.
- Depict with a directed arrow having a dotted line. The tip of arrowhead points to the base use case and the child use case is connected at the base of the arrow.
- The stereotype "<<extends>>" identifies as an extend relationship



Include

- When a use case is depicted as using the functionality of another use case, the relationship between the use cases is named as include or uses relationship.
- A use case includes the functionality described in another use case as a part of its business process flow.
- A uses relationship from base use case to child use case indicates that an instance of the base use case will include the behavior as specified in the child use case.
- An include relationship is depicted with a directed arrow having a dotted line. The tip of arrowhead points to the child use case and the parent use case connected at the base of the arrow.

- The stereotype "<<include>>" identifies the relationship as an include relationship.



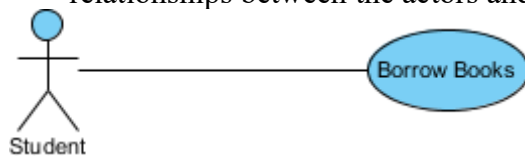
Generalization

- A generalization relationship is a parent-child relationship between use cases.
- The child use case is an enhancement of the parent use case.
- Generalization is shown as a directed arrow with a triangle arrowhead.
- The child use case is connected at the base of the arrow. The tip of the arrow is connected to the parent use case.



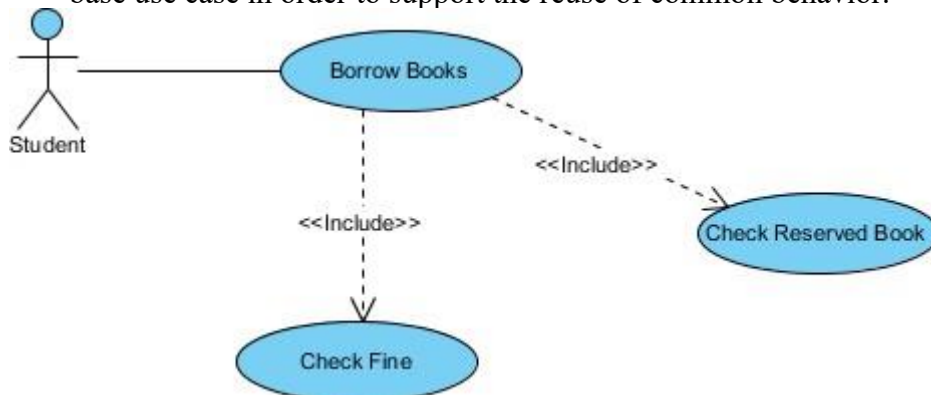
Use Case Example - Association Link

A Use Case diagram illustrates a set of use cases for a system, i.e. the actors and the relationships between the actors and use cases.



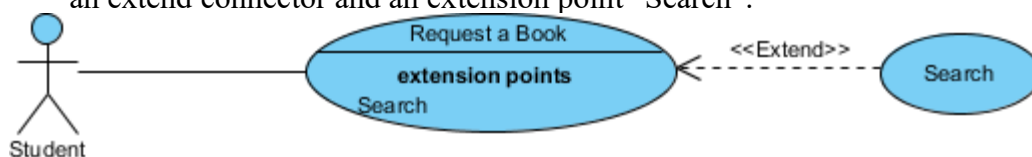
Use Case Example - Include Relationship

The include relationship adds additional functionality not specified in the base use case. The <<Include>> relationship is used to include common behavior from an included use case into a base use case in order to support the reuse of common behavior.



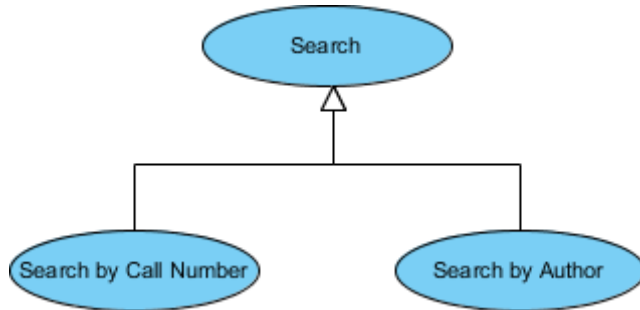
Use Case Example - Extend Relationship

The extend relationships are important because they show optional functionality or system behavior. The <<extend>> relationship is used to include optional behavior from an extending use case in an extended use case. Take a look at the use case diagram example below. It shows an extend connector and an extension point "Search".



Use Case Example - Generalization Relationship

A generalization relationship means that a child use case inherits the behavior and meaning of the parent use case. The child may add or override the behavior of the parent. The figure below provides a use case example by showing two generalization connectors that connect between the three use cases.



How to Draw Use Case Diagram?

[Use case diagram](#) is a kind of [UML diagram](#) that enables you to model system functions (i.e. goals) as well as the actors that interact with those functions. You can draw use case diagrams in Visual Paradigm as well as to document the use case scenario of use cases using the flow-of-events editor. In this page, you will see how to draw use case diagram with the [UML tool](#).

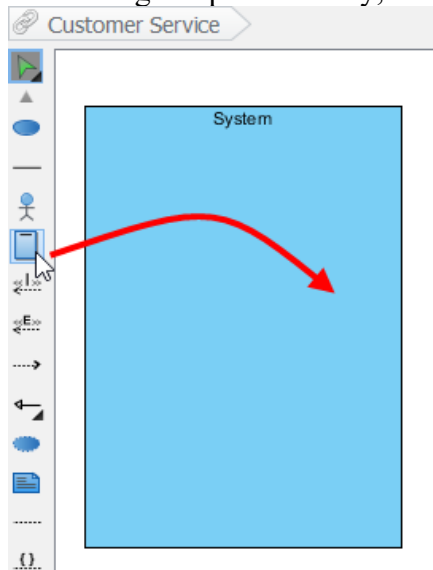
Creating a use case diagram

Perform the steps below to create a UML use case diagram in Visual Paradigm.

1. Select **Diagram > New** from the application toolbar.
2. In the **New Diagram** window, select **Use Case Diagram**.
3. Click **Next**.
4. Enter the diagram name and description. The **Location** field enables you to select a model to store the diagram.
5. Click **OK**.

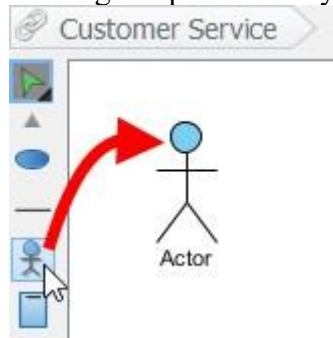
Drawing a system

To create a system in use case diagram, select **System** on the diagram toolbar and then click it on the diagram pane. Finally, name the newly created system when it is created.



Drawing an actor

To draw an actor in use case diagram, select **Actor** on the diagram toolbar and then click it on the diagram pane. Finally, name the newly created actor when it is created.

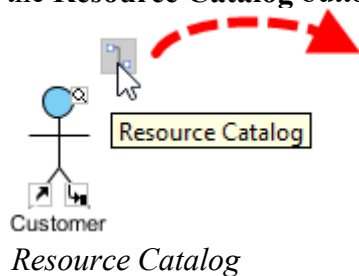


Create an actor

Drawing a use case

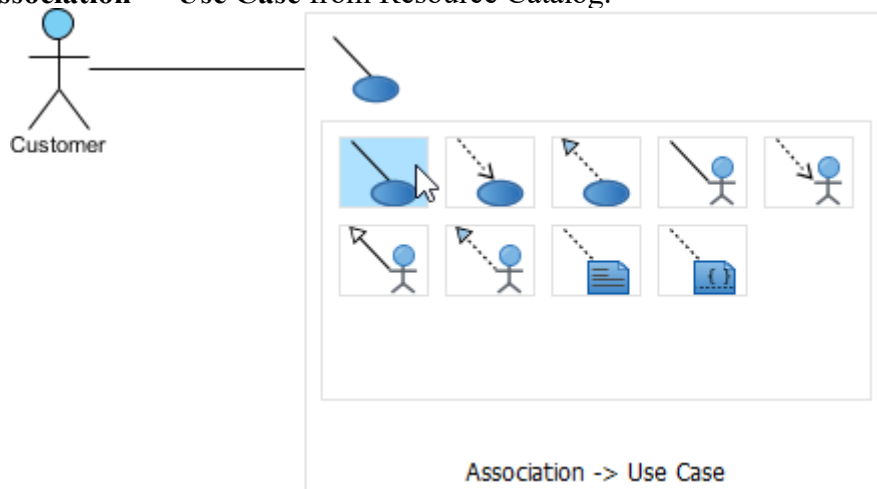
Besides creating a use case through diagram toolbar, you can also create it through Resource Catalog:

1. Move the mouse over a source shape (e.g. an actor).
2. Press on the **Resource Catalog** button and drag it out.



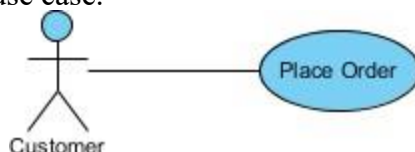
Resource Catalog

3. Release the mouse button until it reaches your preferred place.
4. Select **Association -> Use Case** from Resource Catalog.



To create a use case

5. The source shape and the newly created use case are connected. Finally, name the newly created use case.



Use Case created

Line wrapping use case name

If a use case is too wide, you may resize it by dragging the filled selectors for a better outlook. As a result, the name of use case will be line-wrapped automatically.

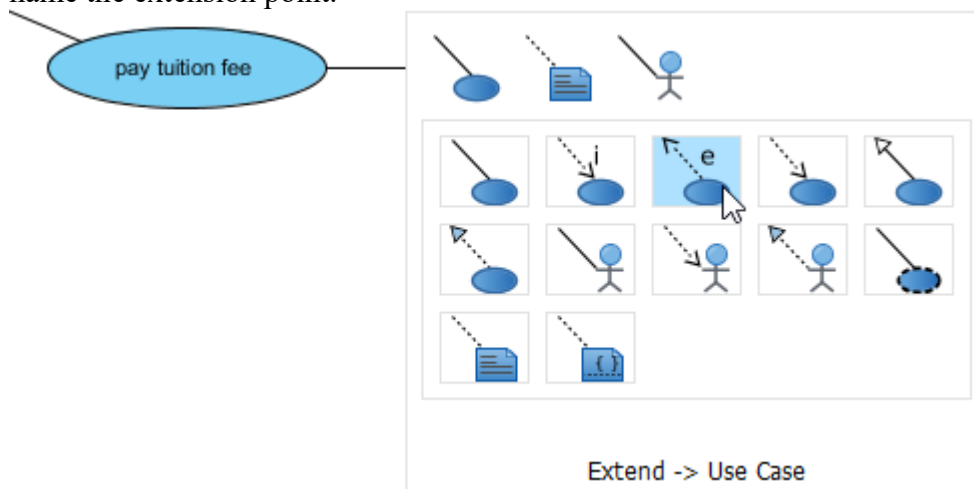


Resize a use case

NOTE: Alternatively, you can press **Alt + Enter** to force a new line.

Drawing <<Extend>> relationship

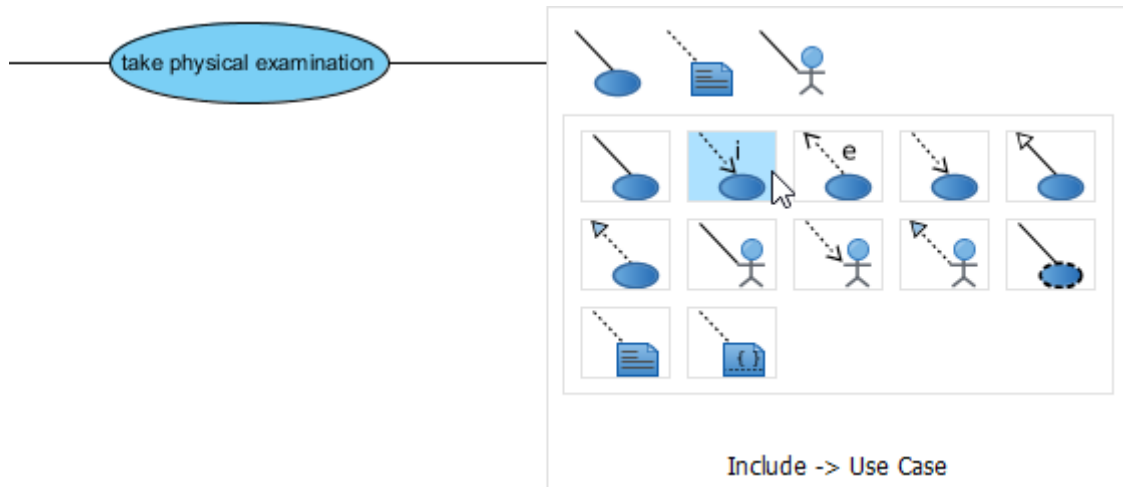
To create an extend relationship, move the mouse over a use case, press and drag out its **Resource Catalog** button. Then, release the mouse button at the preferred place and select **Extend -> Use Case** from Resource Catalog. The use case with extension points and a newly created use case are connected. After you name the newly created use case, you can name the extension point.



Create an extend relationship

Drawing <<Include>> relationship

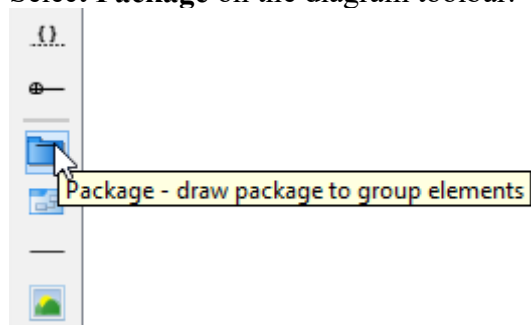
To create an include relationship, move the mouse over a use case, press and drag out its **Resource Catalog** button. Then, release the mouse button at the preferred place and select **Include -> Use Case** from Resource Catalog. A new use case together with an include relationship is created. Finally, name the newly created use case.



Include relationship is created

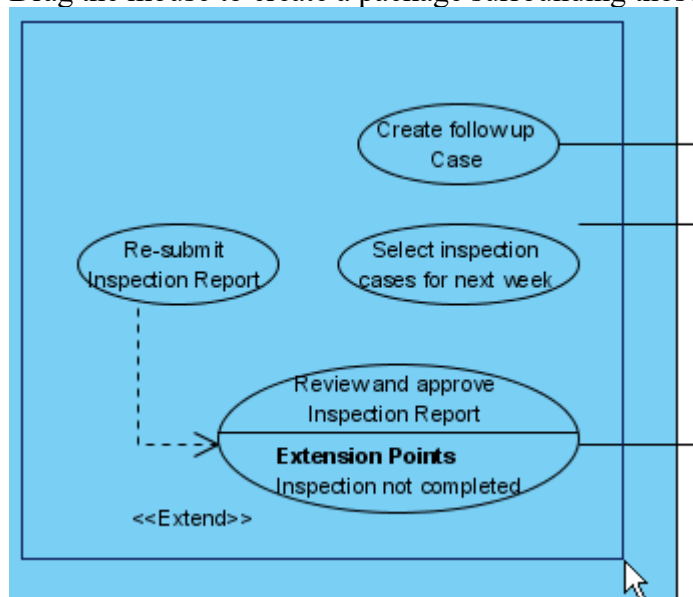
Structuring use cases with package

You can organize use cases with package when there are many of them on the diagram. Select **Package** on the diagram toolbar.



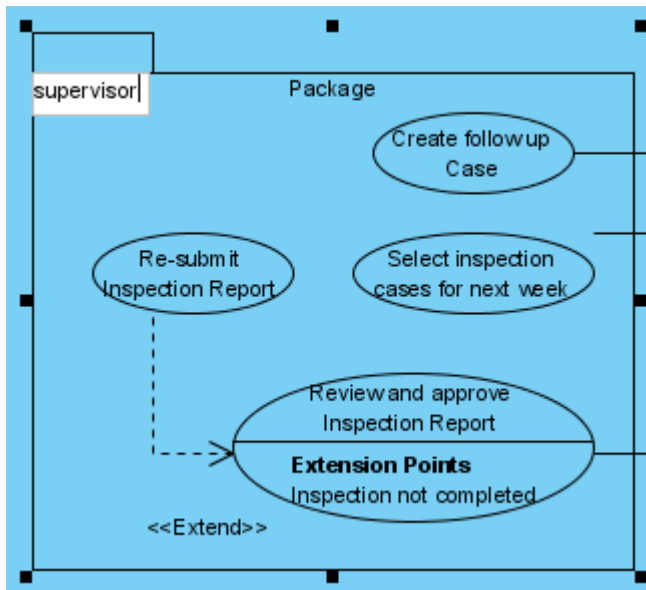
Create a package

Drag the mouse to create a package surrounding those use cases.



Surround use cases with package

Finally, name the package.

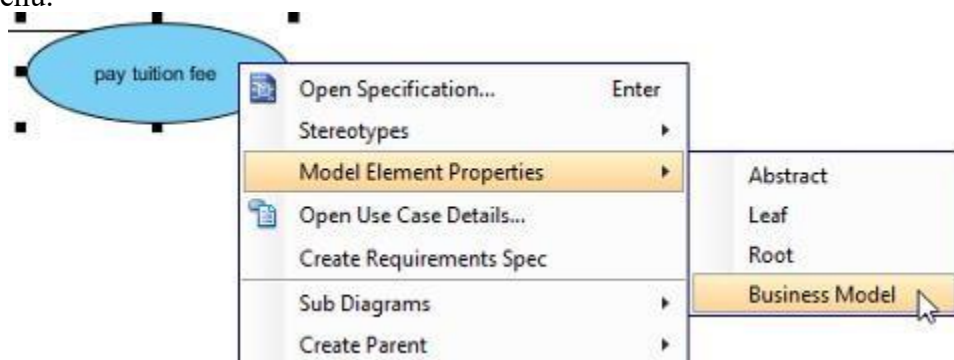


Name the package

Drawing business use case

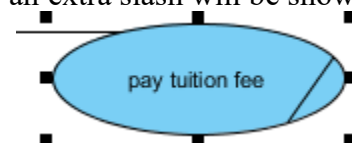
The uml diagram tool also supports the representation of business actor and use case. To show an ordinary use case as business use case:

1. Right click on a use case and select **Model Element Properties > Business Model** from the pop-up menu.



Click Business Model

2. After selected, an extra slash will be shown on the left edge of the use case.



Business Model

How to Identify Actor

Often, people find it easiest to start the requirements elicitation process by identifying the actors. The following questions can help you identify the actors of your system (Schneider and Winters - 1998):

Who uses the system?
Who installs the system?
Who starts up the system?
Who maintains the system?
Who shuts down the system?
What other systems use this system?
Who gets information from this system?
Who provides information to the system?
Does anything happen automatically at a present time?

How to Identify Use Cases?

Identifying the Use Cases, and then the scenario-based elicitation process carries on by asking what externally visible, observable value that each actor desires. The following questions can be asked to identify use cases, once your actors have been identified (Schneider and Winters - 1998):

- What functions will the actor want from the system?
- Does the system store information? What actors will create, read, update or delete this information?
- Does the system need to notify an actor about changes in the internal state?
- Are there any external events the system must know about? What actor informs the system of those events?

Use Case Diagram Tips

Now, check the tips below to see how to apply use case effectively in your software project.

- Always structure and organize the use case diagram from the perspective of actors.
- Use cases should start off simple and at the highest view possible. Only then can they be refined and detailed further.
- Use case diagrams are based upon functionality and thus should focus on the "what" and not the "how".

Use Case Levels of Details

Use case granularity refers to the way in which information is organized within use case specifications, and to some extent, the level of detail at which they are written. Achieving the right level of use case granularity eases communication between stakeholders and developers and improves project planning.

Alastair Cockburn in Writing Effective Use Cases gives us an easy way to visualize different levels of goal level by thinking in terms of the sea:

Practice Tasks for Drawing Use Cases

1. **Online Shopping System:**

Draw a use case diagram for an online shopping system. Include actors like "Customer," "Admin," and "Payment Gateway." Represent use cases such as "Browse Products," "Add to Cart," "Checkout," and "Process Payment." Use the **include** relationship to show that "Checkout" must include "Process Payment," and use the **extends** relationship to indicate that "Apply Discount" extends "Checkout" under certain conditions.

2. **Library Management System:**

Create a use case diagram for a library management system. Identify actors like "Librarian" and "Member." Include use cases such as "Search Book," "Issue Book," "Return Book," and "Manage Inventory." Use the **include** relationship to show that "Issue Book" must include "Validate Membership," and use the **extends** relationship to indicate that "Renew Book" extends "Return Book" if the member chooses to renew.

3. **Hospital Management System:**

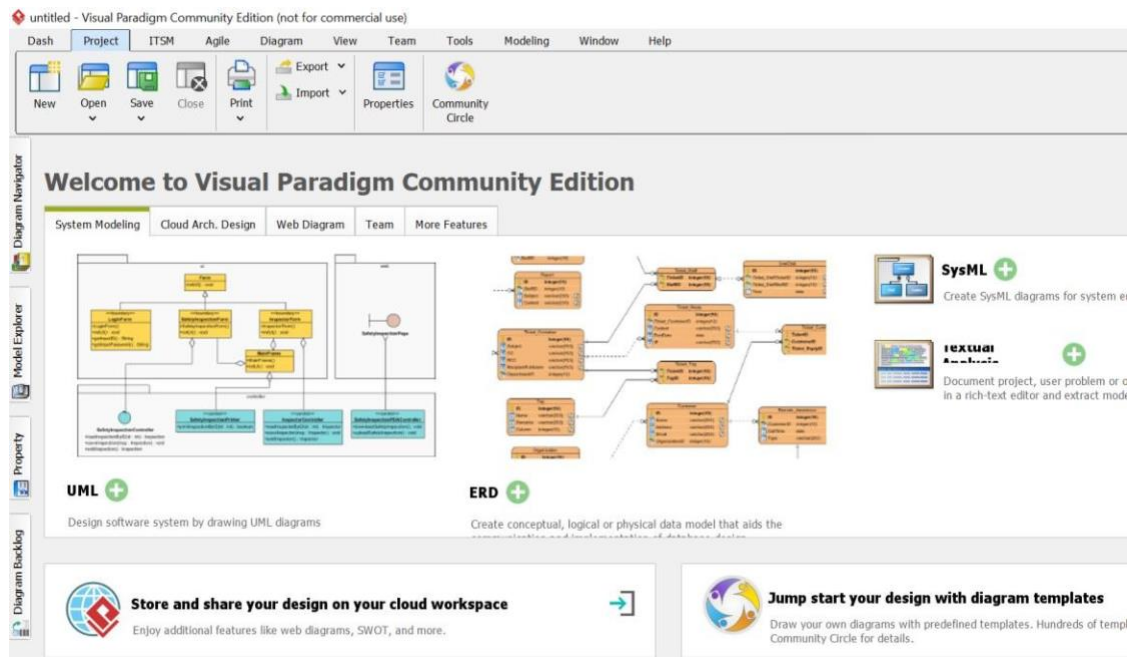
Design a use case diagram for a hospital management system. Include actors like "Doctor," "Patient," and "Receptionist." Represent use cases such as "Schedule Appointment," "Diagnose Patient," "Generate Bill," and "Manage Records." Use the **include** relationship to show that "Generate Bill" must include "Calculate Charges," and use the **extends** relationship to indicate that "Emergency Appointment" extends "Schedule Appointment" in urgent cases.

Level	Example	Summary
 Cloud	Product	High Summary
 Kite	Sell Products	Summary
 Sea	Sell Furniture	User goal
 Fish	Browse Catalog	Sub-function
 Clam	Insert Orderline	Low level

Walkthrough Task

1. Open Visual Paradigm

- Launch Visual Paradigm and open your project or create a new one.



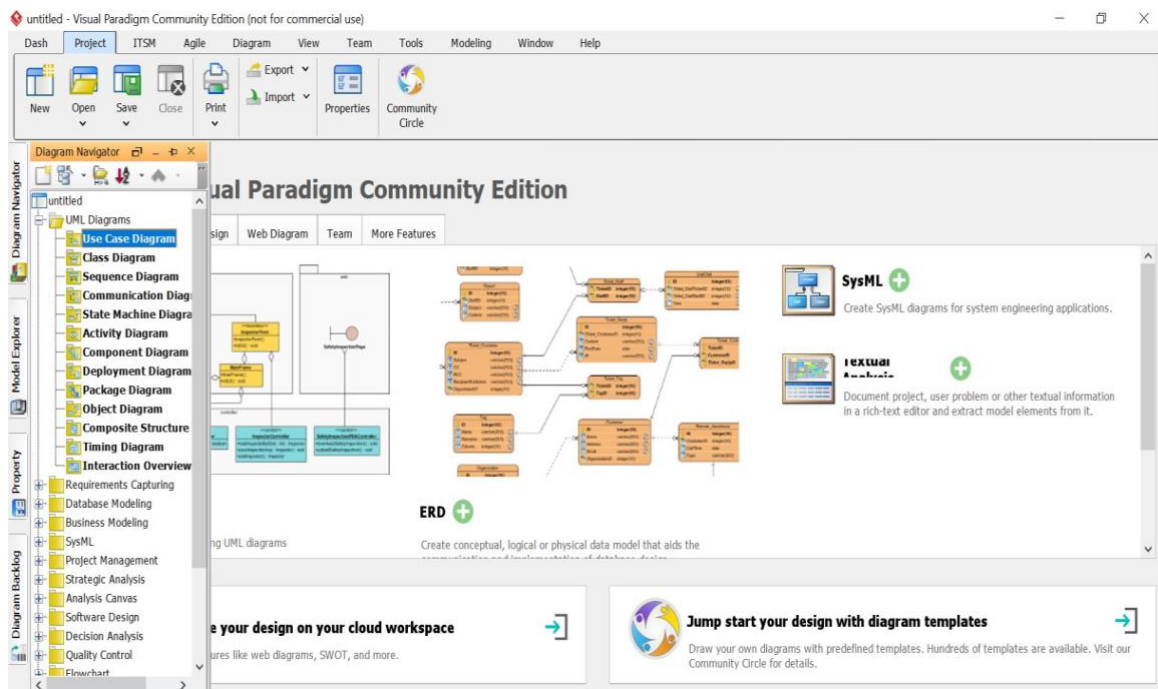
2. Create a New Use Case Diagram

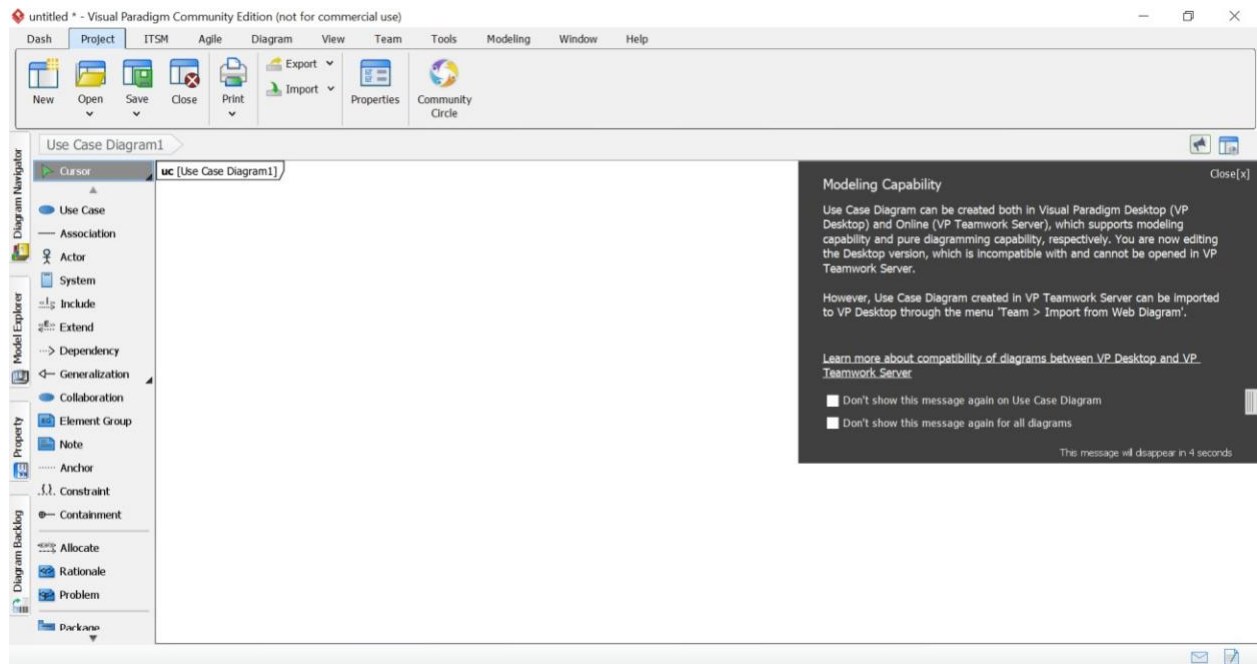
Go to the Diagram Navigator.

Click on New Diagram or Create Diagram.

Search for "Use Case Diagram" and select it.

Name the diagram (e.g., "Online Therapy Platform Use Case Diagram").





3. Add Actors

- Identify the actors from the file:
 - **User:** The person using the platform to book therapy sessions.
 - **Therapist:** The professional providing therapy services.
- Drag and drop **Actor** elements from the toolbar onto the diagram and label them as "User" and "Therapist."

4. Add Use Cases

- Add the use cases based on the file content:
 - **User Use Cases:**
 - Sign Up
 - Log in
 - Search for Therapist
 - View Therapist Profile
 - Book Therapy Session
 - Confirm Booking
 - Attend Therapy Session
 - **Therapist Use Cases:**
 - Update Profile
 - View Bookings
 - Decline Booking
- Drag and drop **Use Case** elements from the toolbar onto the diagram and label them with the above names.

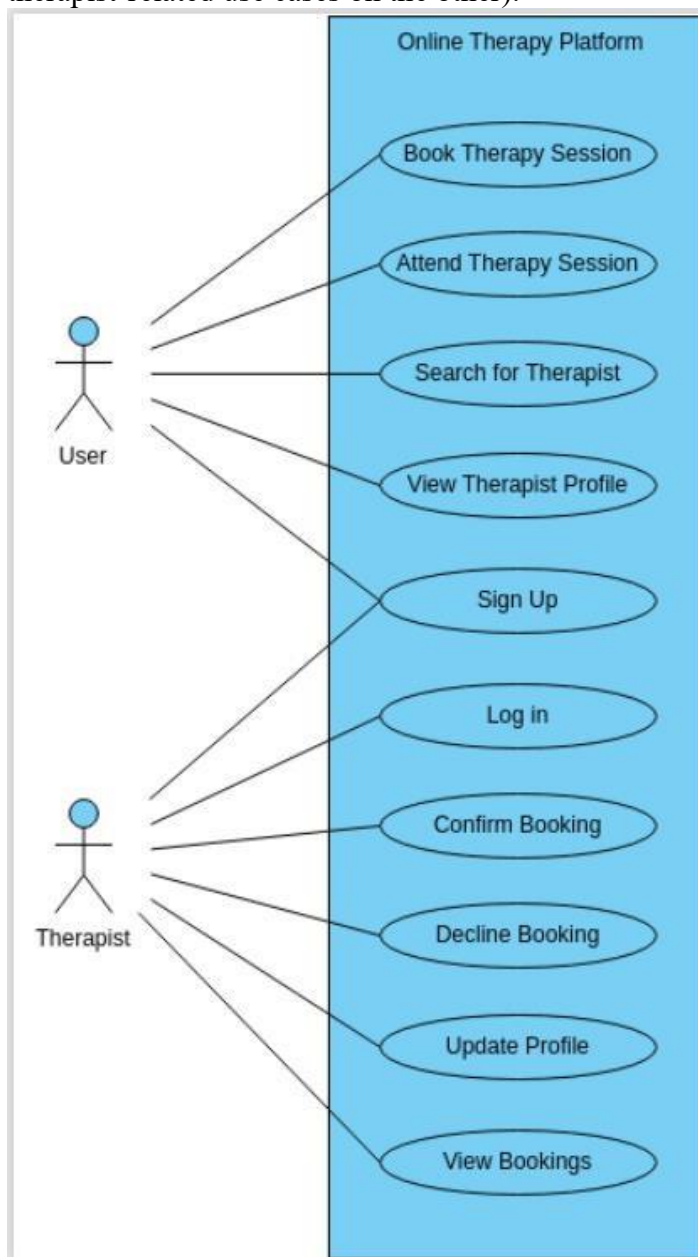
5. Connect Actors to Use Cases

- Use the **Association** tool to connect actors to their respective use cases:
 - **User** should be connected to:
 - Sign Up
 - Log in

- Search for Therapist
- View Therapist Profile
- Book Therapy Session
- Confirm Booking
- Attend Therapy Session
- **Therapist** should be connected to:
 - Update Profile
 - View Bookings
 - Decline Booking

6. Organize the Diagram

- Arrange the use cases and actors to make the diagram clear and easy to read.
- Group related use cases together (e.g., place all user-related use cases on one side and therapist-related use cases on the other).



Practice task scenario

Create a **Use Case Diagram**. This scenario is based on a **Library Management System**, which is a common and practical example. You can use this scenario to practice creating a Use Case Diagram in Visual Paradigm or any other diagramming tool.

Scenario: Library Management System

A library management system allows librarians to manage books and members, and members to search for books, borrow books, and return books. The system also handles notifications for overdue books.

Actors

1. **Librarian**: Manages books and members in the library.
2. **Member**: Borrows and returns books.
3. **System**: Sends notifications for overdue books.

Use Cases

Here are the use cases for the system:

Librarian Use Cases

1. **Add Book**: Add a new book to the library catalog.
2. **Update Book**: Update details of an existing book.
3. **Remove Book**: Remove a book from the library catalog.
4. **Add Member**: Register a new library member.
5. **Update Member**: Update member details.
6. **Remove Member**: Remove a member from the system.

Member Use Cases

1. **Search Book**: Search for a book by title, author, or category.
2. **Borrow Book**: Borrow a book from the library.
3. **Return Book**: Return a borrowed book.
4. **View Borrowing History**: View a list of books borrowed in the past.

System Use Cases

1. **Send Overdue Notification**: Send a notification to a member when a book is overdue.

Capital University of Science & Technology
Islamabad
Department of Software Engineering,
Faculty of Computing

Lab 15 A :
Requirement Modeling (Domain Model)

Week 15: Requirement Modeling Cont.

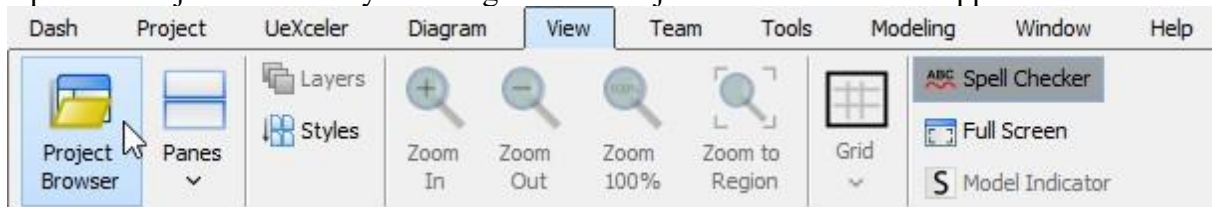
Activity Time Boxing

Task No.	Activity Name	Activity time	Total Time
1	Quiz	30 min	30 min
2	Walkthrough Tasks	60 min	90min
3	Practice Tasks	50 As per time specified for each task	140 min
4	Evaluation Task	40 min for each assigned task	180 min

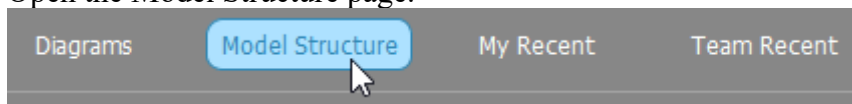
Walk through task for domain model

Creating domain class model

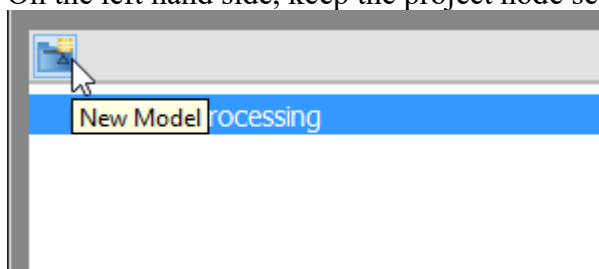
1. Create a new project by selecting Project > New from the application toolbar. In the New Project window, enter *Order Processing* as project name and click Create Blank Project.
2. Open the Project Browser by selecting View > Project Browser from the application toolbar.



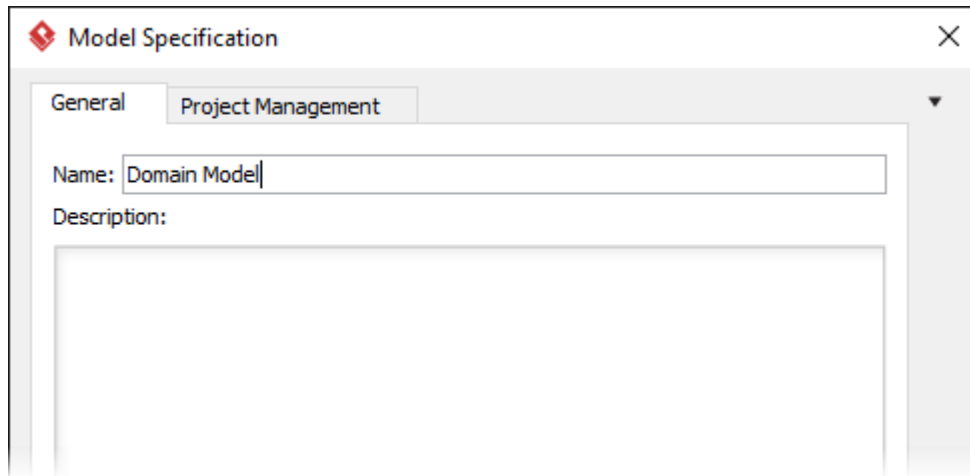
3. Open the Model Structure page.



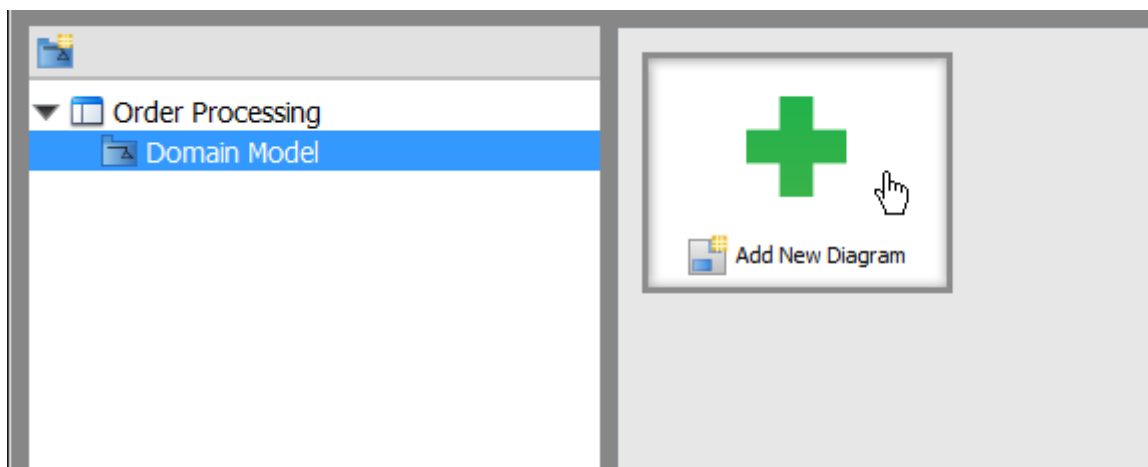
4. On the left hand side, keep the project node selected in the list and then click New Model.



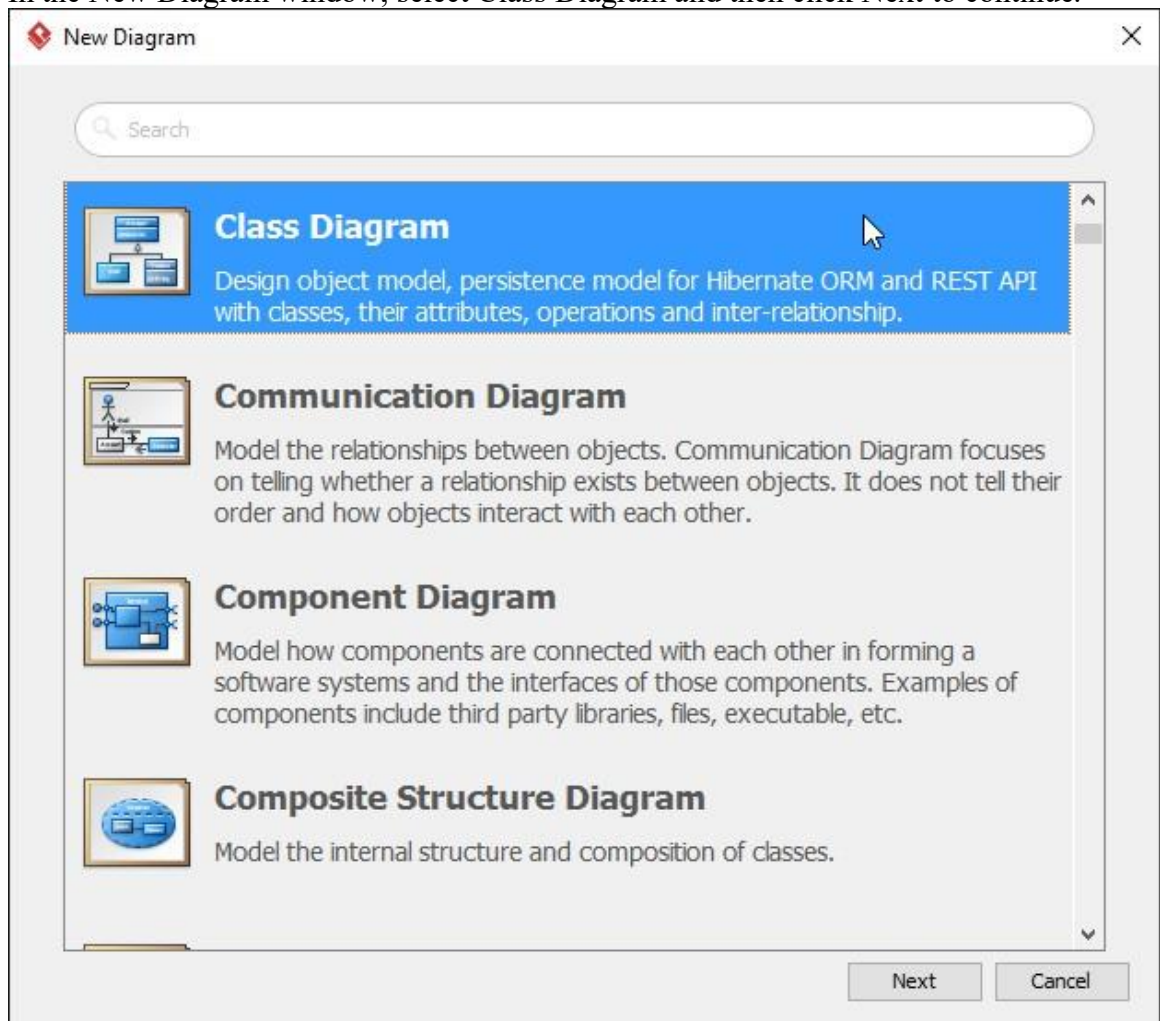
5. In the Model Specification window, enter *Domain Model* as model name. Click OK to confirm.



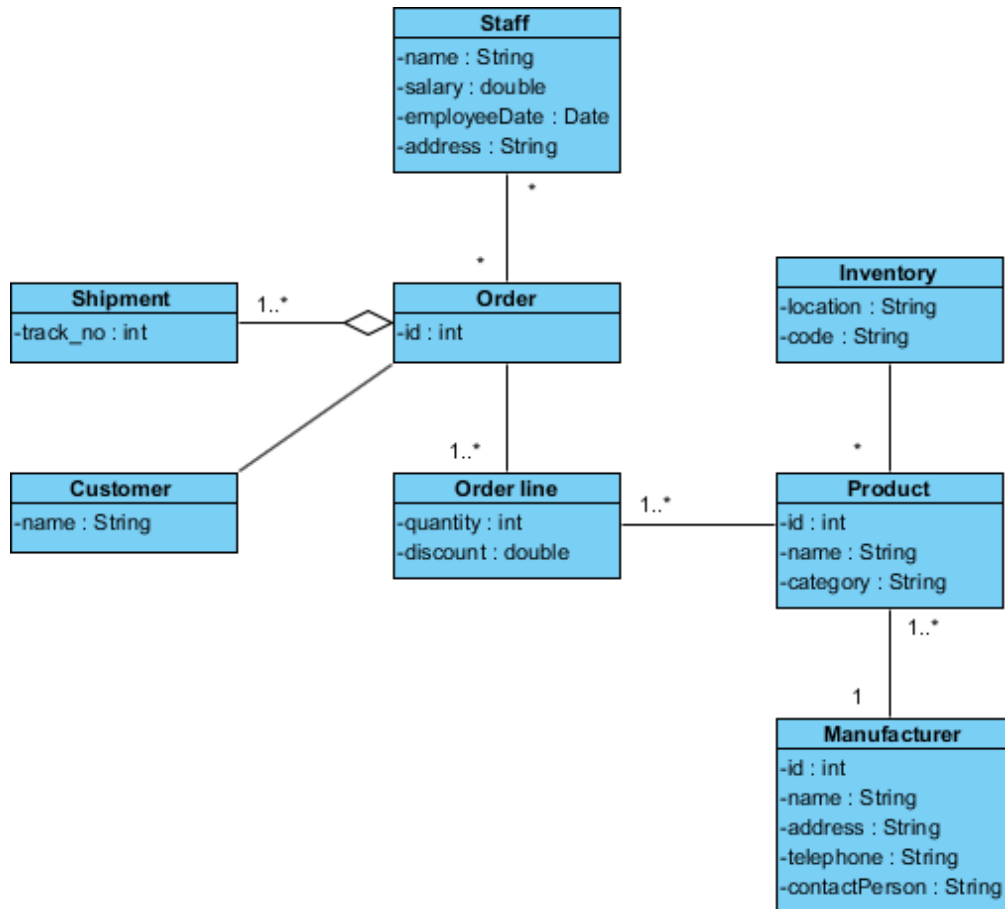
6. Keep Domain Model selected in the list. Then, click Add New Diagram.



7. In the New Diagram window, select Class Diagram and then click Next to continue.



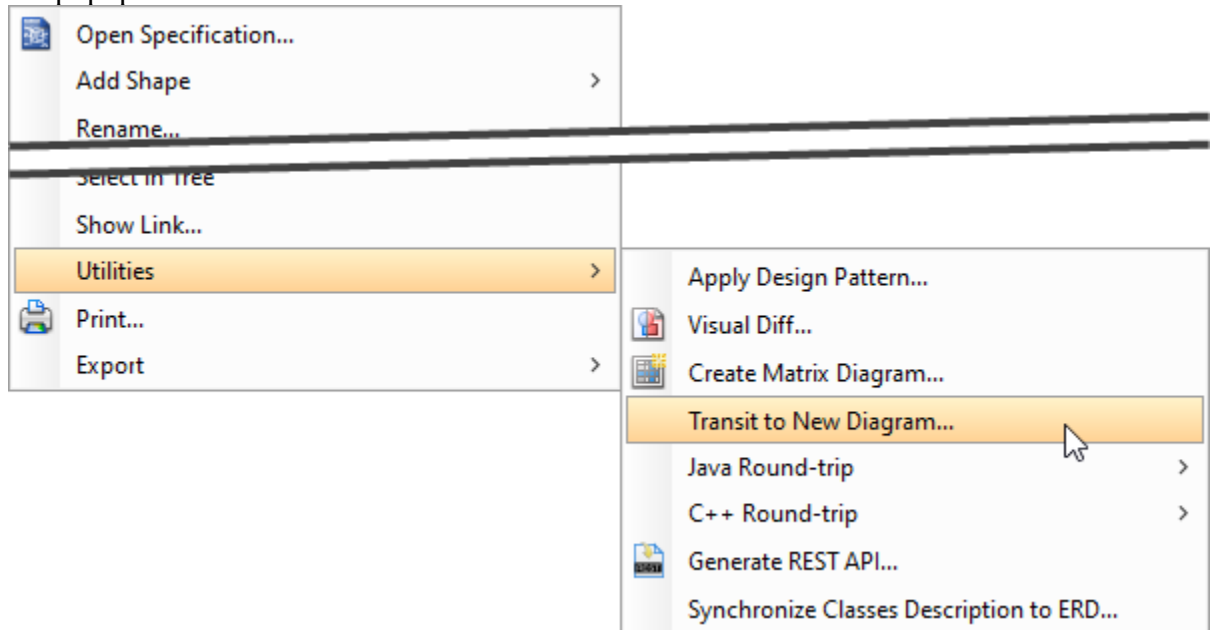
8. Keep the diagram name unchanged and then click OK to create the diagram.
9. Draw a domain model class diagram like this:



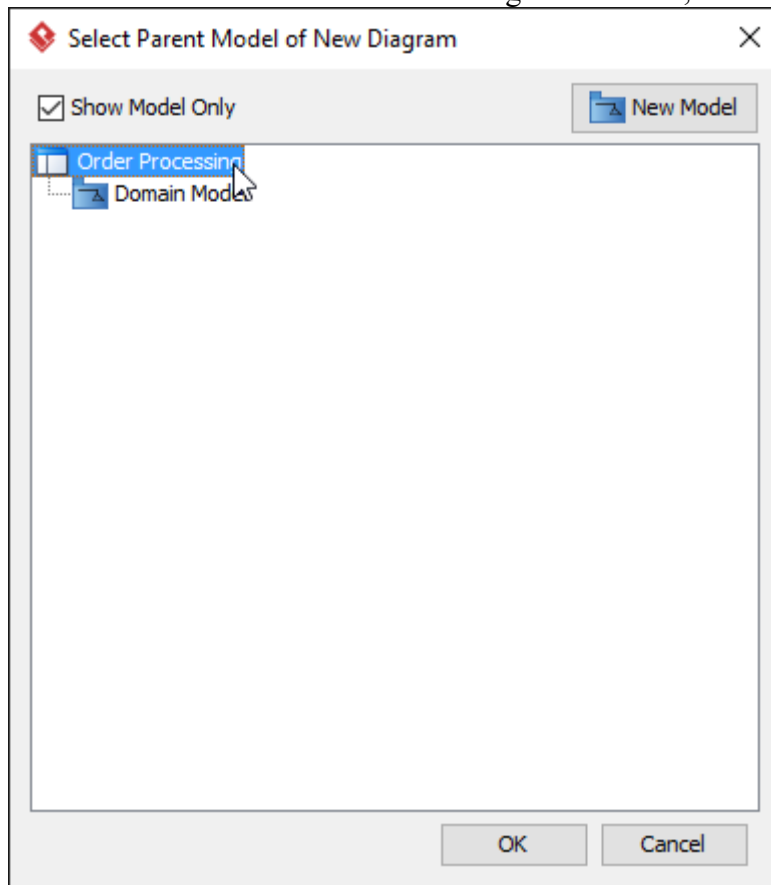
Creating implementation class model

Having completed the domain model, we are going to transcribe it into implementation model.

1. Right-click on the diagram background and select Utilities > Transit to New Diagram... from the popup menu.

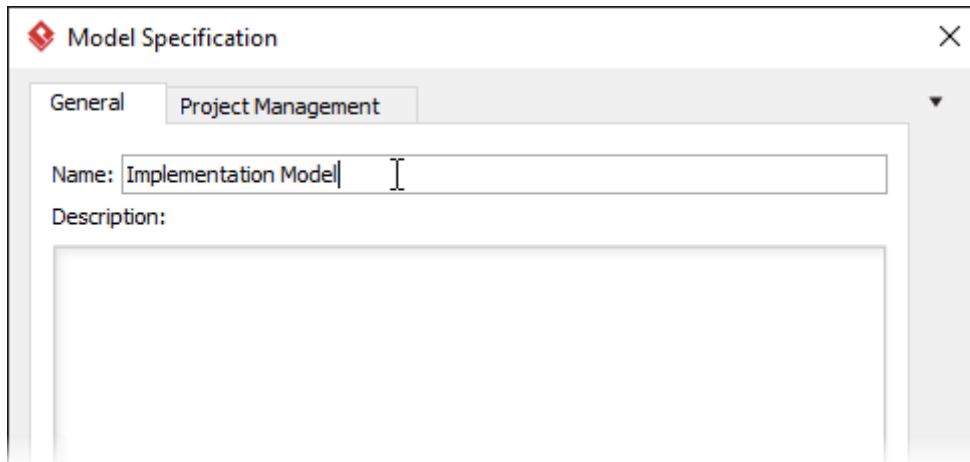


2. In the Select Parent Model of New Diagram window, select the project root node.

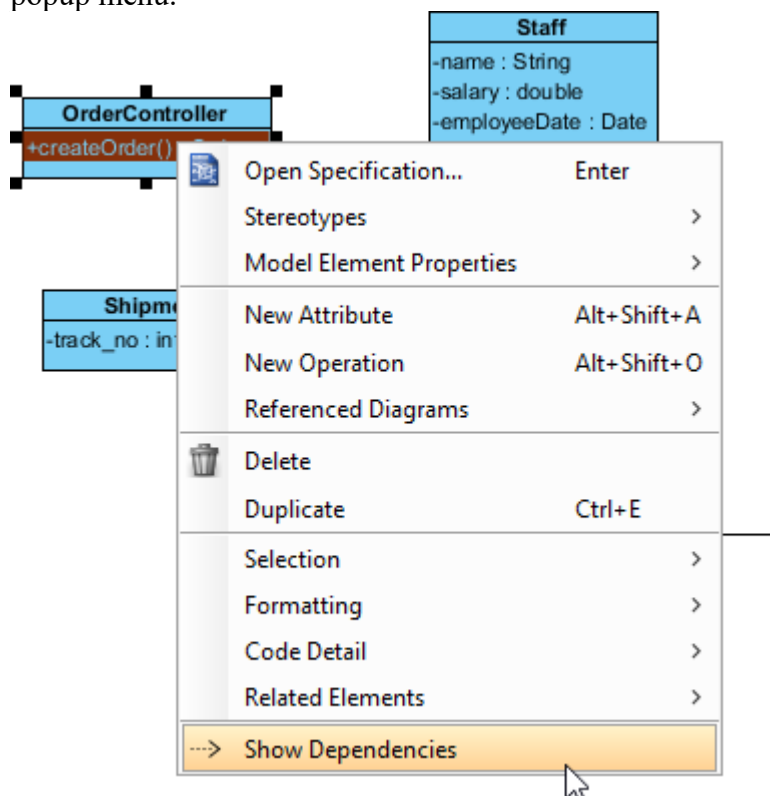


3. Click New Model at the top right of window.

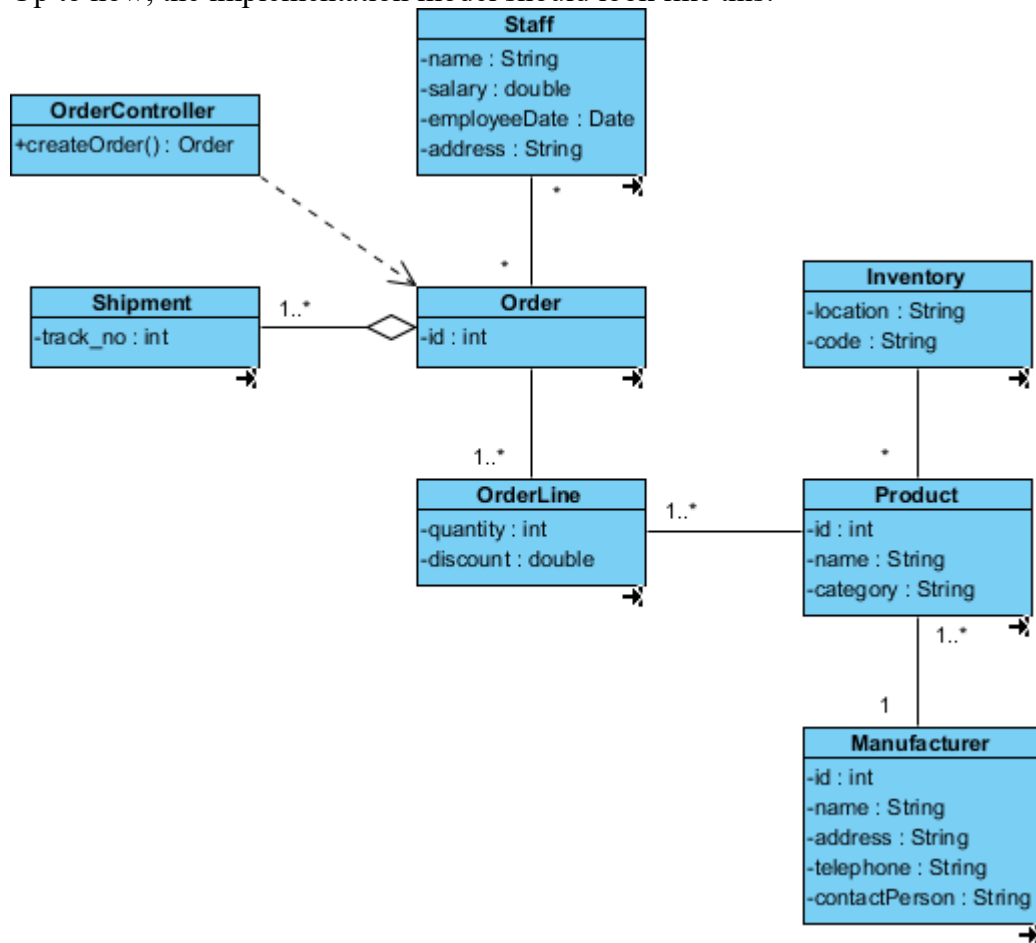
4. In the Model Specification window, enter *Implementation Model* as model name. Click OK to confirm.



5. Keep *Implementation Model* selected, click OK to confirm. This forms a new diagram with name *Implementation Model*, which looks the same as *Domain Model*. What we are going to do is to refine the design to make it become an implementation model.
6. Delete the class *Customer* by selecting it, and hit the delete key.
7. Rename class *Order line* to *OrderLine*.
8. Create a class *OrderController*.
9. Add an operation *createOrder() : Order* into *OrderController*.
10. Right-click on the operation *createOrder() : Order*, select Show Dependencies from the popup menu.



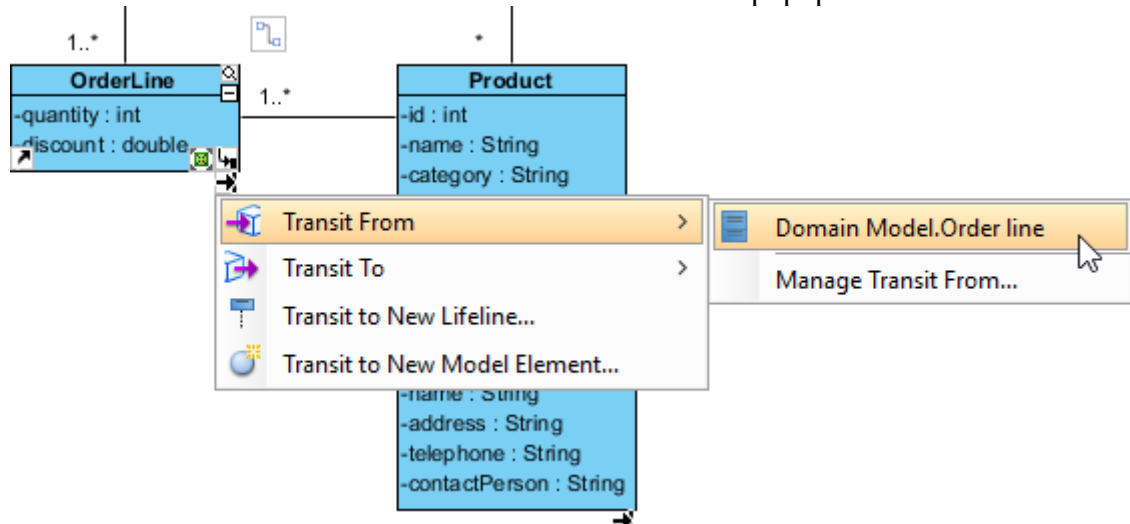
Up to now, the implementation model should look like this:



Using Model Transitor

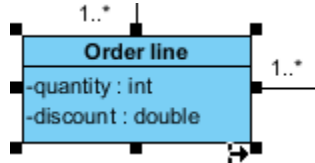
Let's refresh our memory on what we have changed for the class *OrderLine*.

1. Move your mouse pointer over the class *OrderLine*.
2. Click on the Model Transitor resource icon at the bottom right of the class shape and then select Transit From > Domain Model.Order line from the popup menu.



This populates the domain model, with former version of class *OrderLine* selected, which was

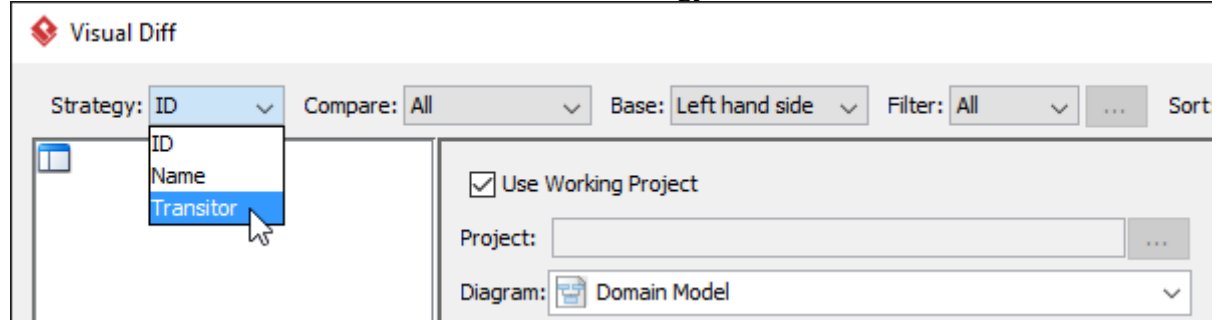
named *Order line*.



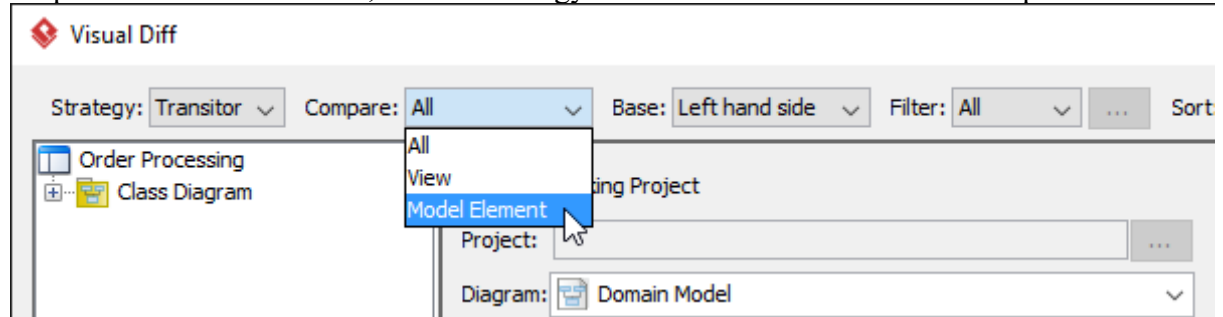
Comparing models with Visual Diff

In order to identify all the refinements made in implementation model, we need to make use of the Visual Diff tool.

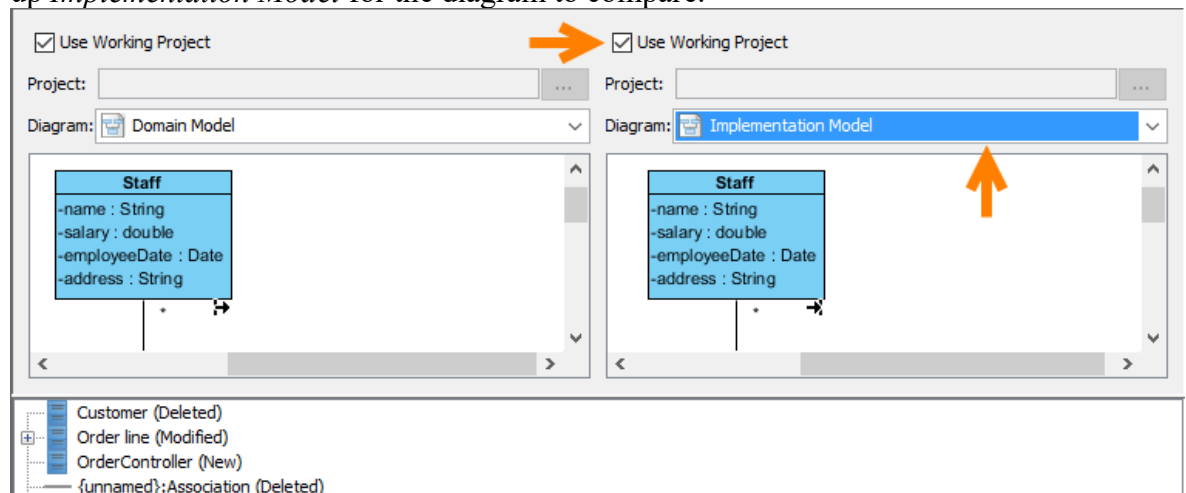
1. Open Visual Diff by selecting Modeling > Visual Diff from the application toolbar.
2. In the Visual Diff window, select Transitor as Strategy.



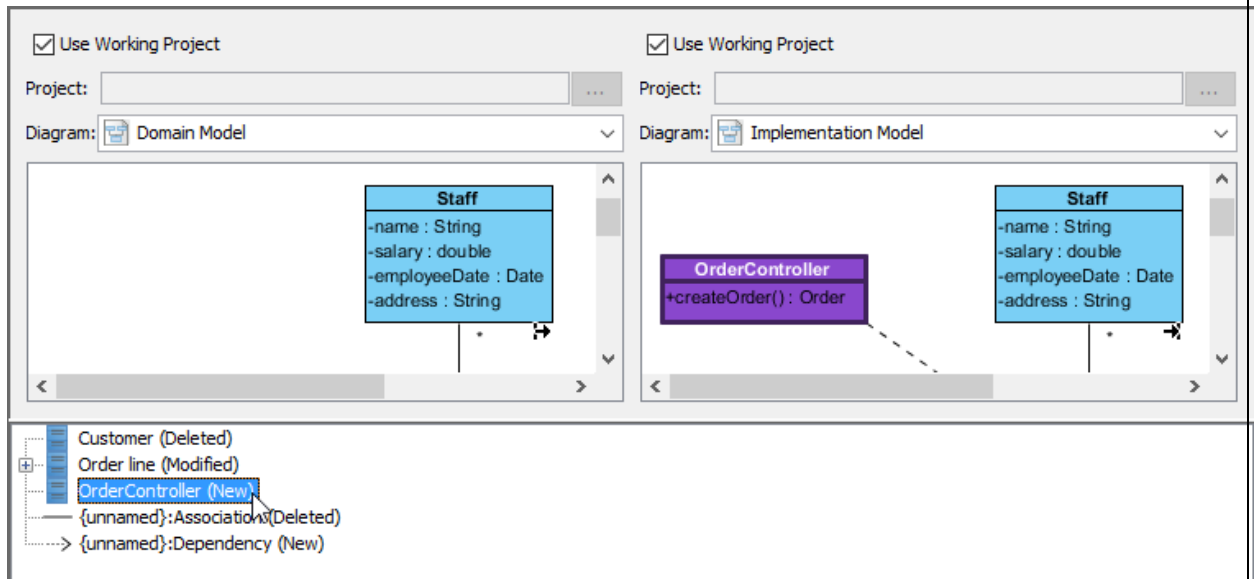
3. We wanted to know the changes of model element details rather than view details such as shape coordinate. Therefore, next to Strategy we select Model Element to Compare.



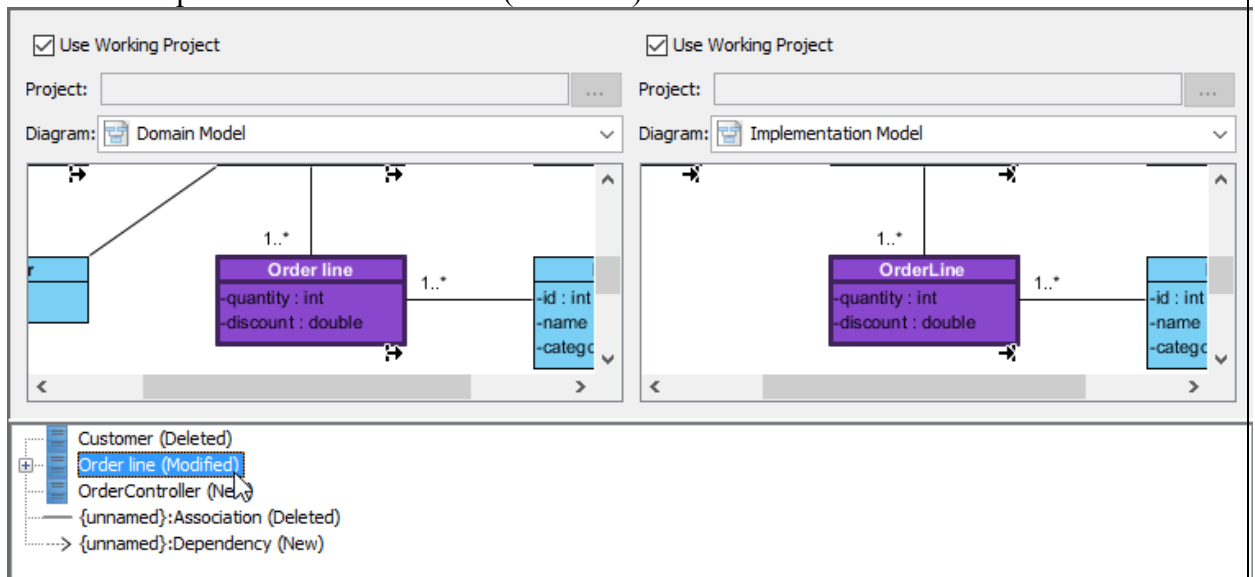
4. For the diagram pane on the right, keep Use Working Project selected, and pick up *Implementation Model* for the diagram to compare.



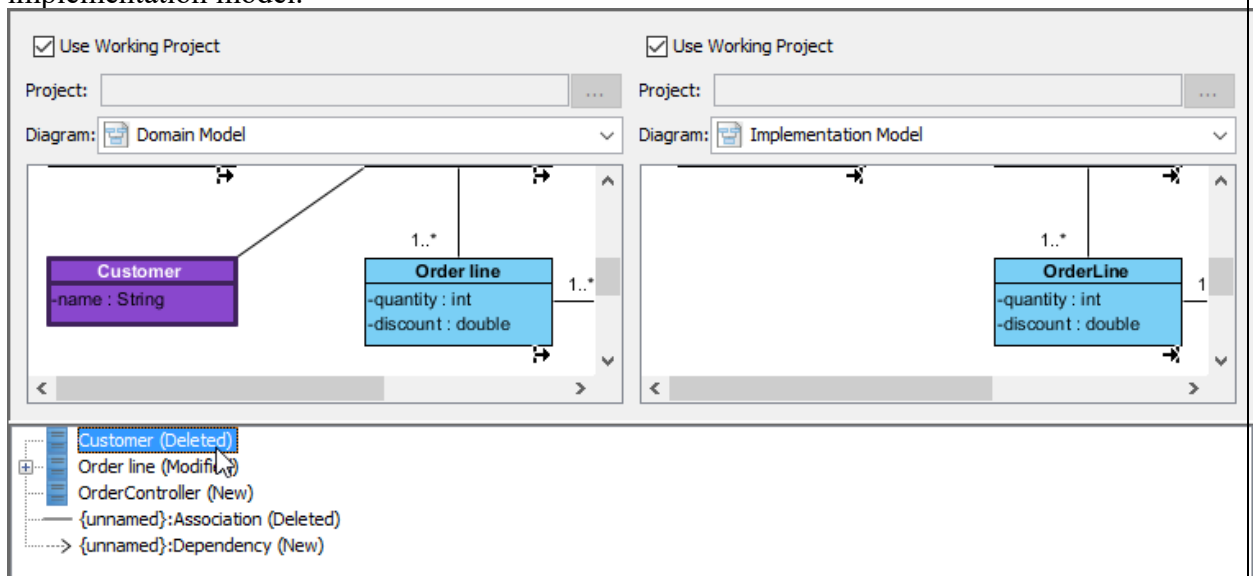
5. Now, we have obtained the differences between domain (left hand side) and implementation (right hand side) model, at the bottom pane. Click on the node OrderController (New). From the tag (New), and from the diagram, we know that it is newly added in the implementation model.



6. Select and expand the node Order line (Modified). We know that it is renamed.



7. Select the node Customer (Deleted). We can see that it does not exist anymore in the implementation model.



Practice Task

1. E-commerce System

Step-by-Step Guide

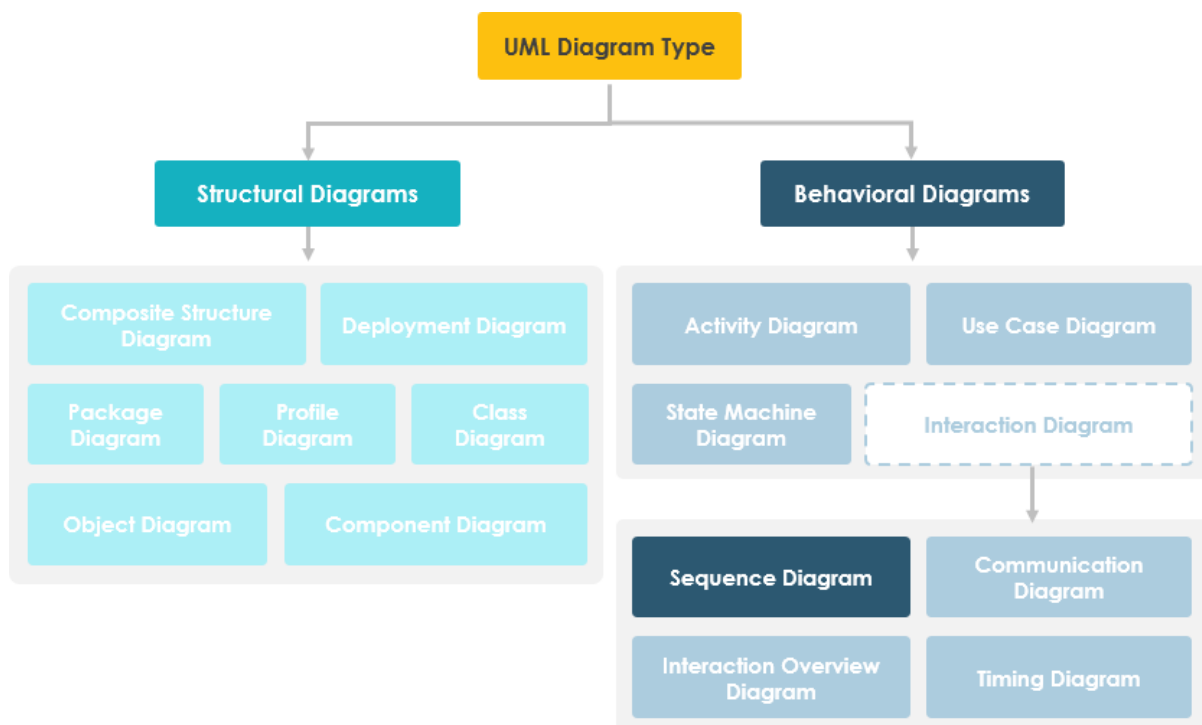
1. Identify Entities:
 - Customer: Represents users who purchase products.
 - Product: Items available for sale.
 - Order: A collection of products purchased by a customer.
 - Cart: Temporary storage for products before checkout.
2. Define Attributes:
 - Customer: Name, Email, Address.
 - Product: Name, Price, Description.
 - Order: Order Date, Total Cost.
 - Cart: Products added, Subtotal.
3. Establish Relationships:
 - A Customer can place multiple Orders.
 - An Order is associated with one Customer.
 - A Cart belongs to one Customer.
 - An Order can contain multiple Products.
4. Incorporate Behaviors:
 - Customer can "Add to Cart," "Place Order," "View Order History."
 - Product can be "Added to Cart," "Removed from Cart," "Purchased."
5. Constraints and Rules:
 - A product can only be added to the cart if it is available.
 - An order can only be placed if the cart is not empty.

**Lab 15B : Requirement Modeling (System
Sequence Diagram)**

Requirement Modeling Cont.

1. Activity Time Boxing

UML Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time focus and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent and when.



Purpose of Sequence Diagram

- Model high-level interaction between active objects in a system
- Model the interaction between object instances within a collaboration that realizes a use case
- Model the interaction between objects within a collaboration that realizes an operation
- Either model generic interactions (showing all possible paths through the interaction) or specific instances of a interaction (showing just one path through the interaction)

Sequence Diagrams at a Glance

- Sequence Diagrams show elements as they interact over time and they are organized according to object (horizontally) and time (vertically):

Object Dimension

- The horizontal axis shows the elements that are involved in the interaction
- Conventionally, the objects involved in the operation are listed from left to right according to when they take part in the message sequence. However, the elements on the horizontal axis

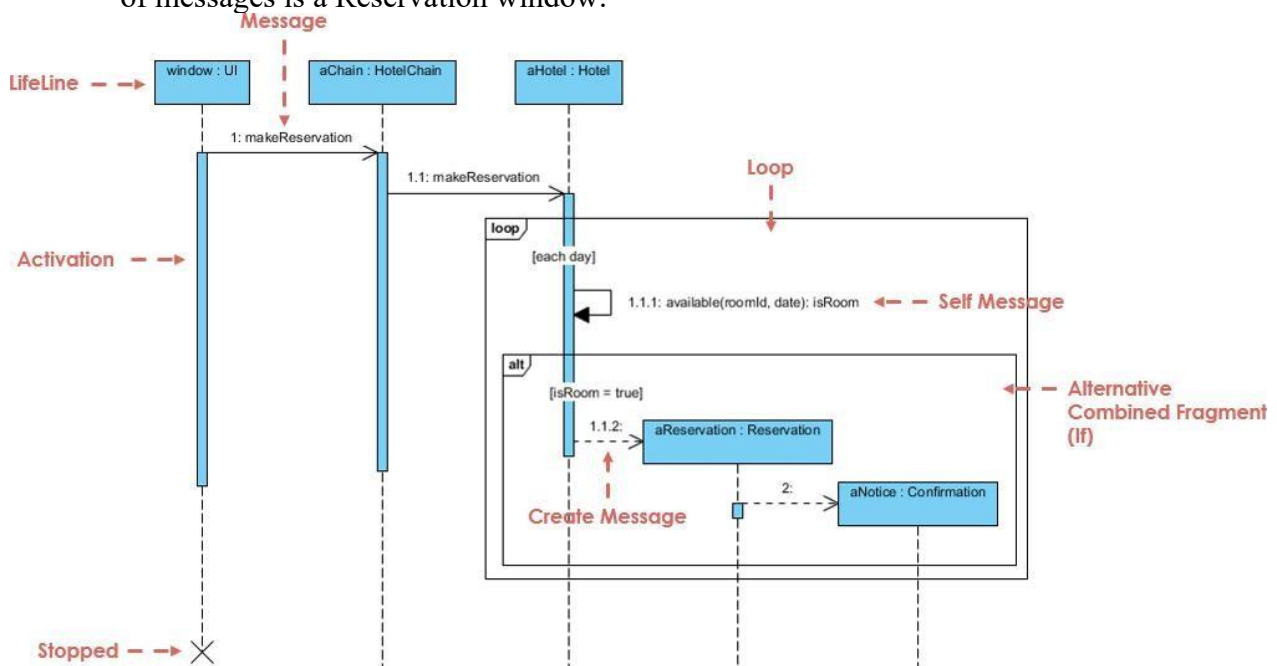
may appear in any order

Time Dimension

- The vertical axis represents time proceedings (or progressing) down the page.

Hotel System

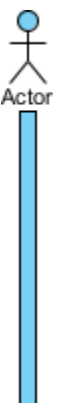
- Sequence Diagram is an interaction diagram that details how operations are carried out -- what messages are sent and when. Sequence diagrams are organized according to time. The time progresses as you go down the page. The objects involved in the operation are listed from left to right according to when they take part in the message sequence.
- Below is a sequence diagram for making a hotel reservation. The object initiating the sequence of messages is a Reservation window.



Sequence Diagram Notation

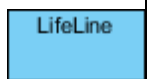
Actor

- A type of role played by an entity that interacts with the subject (e.g., by exchanging signals and data)
- External to the subject (i.e., in the sense that an instance of an actor is not a part of the instance of its corresponding subject).
- Represent roles played by human users, external hardware, or other subjects.
- Note that:
 - An actor does not necessarily represent a specific physical entity but merely a particular role of some entity
 - A person may play the role of several different actors and, conversely, a given actor may be played by multiple different person.



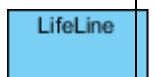
Lifeline

- A lifeline represents an individual participant in the Interaction.



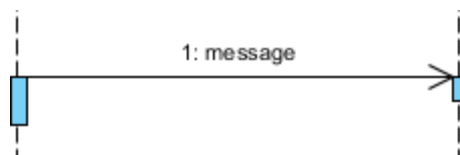
Activations

- A thin rectangle on a lifeline) represents the period during which an element is performing an operation.
- The top and the bottom of the of the rectangle are aligned with the initiation and the completion time respectively



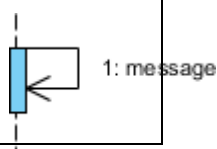
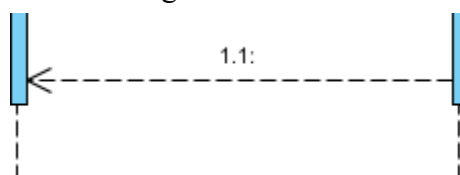
Call Message

- A message defines a particular communication between Lifelines of an Interaction.
- Call message is a kind of message that represents an invocation of operation of target lifeline.



Return Message

- A message defines a particular communication between Lifelines of an Interaction.
- Return message is a kind of message that represents the pass of information back to the caller of a corresponded former message.

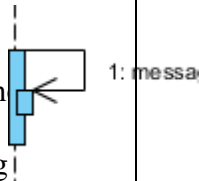


Self Message

- A message defines a particular communication between Lifelines of an Interaction.
- Self message is a kind of message that represents the invocation of message of the same lifeline.

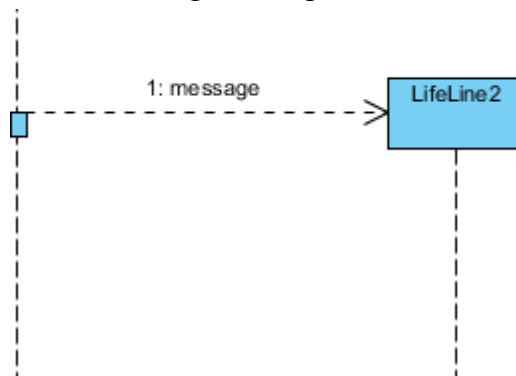
Recursive Message

- A message defines a particular communication between Lifelines of an Interaction.
- Recursive message is a kind of message that represents the invocation of message of the same lifeline. It's target points to an activation on top of the activation where the message was invoked from.
-



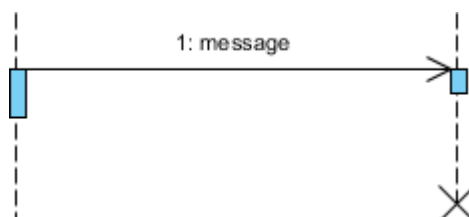
Create Message

- A message defines a particular communication between Lifelines of an Interaction.
- Create message is a kind of message that represents the instantiation of (target) lifeline.



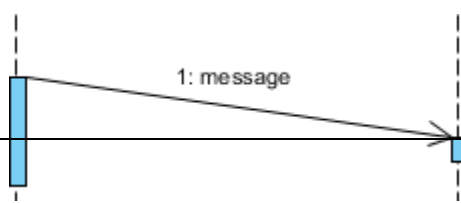
Destroy Message

- A message defines a particular communication between Lifelines of an Interaction.
- Destroy message is a kind of message that represents the request of destroying the lifecycle of target lifeline.



Duration Message

- A message defines a particular communication between Lifelines of an Interaction.
- Duration message shows the distance between two time instants for a message invocation.



Note

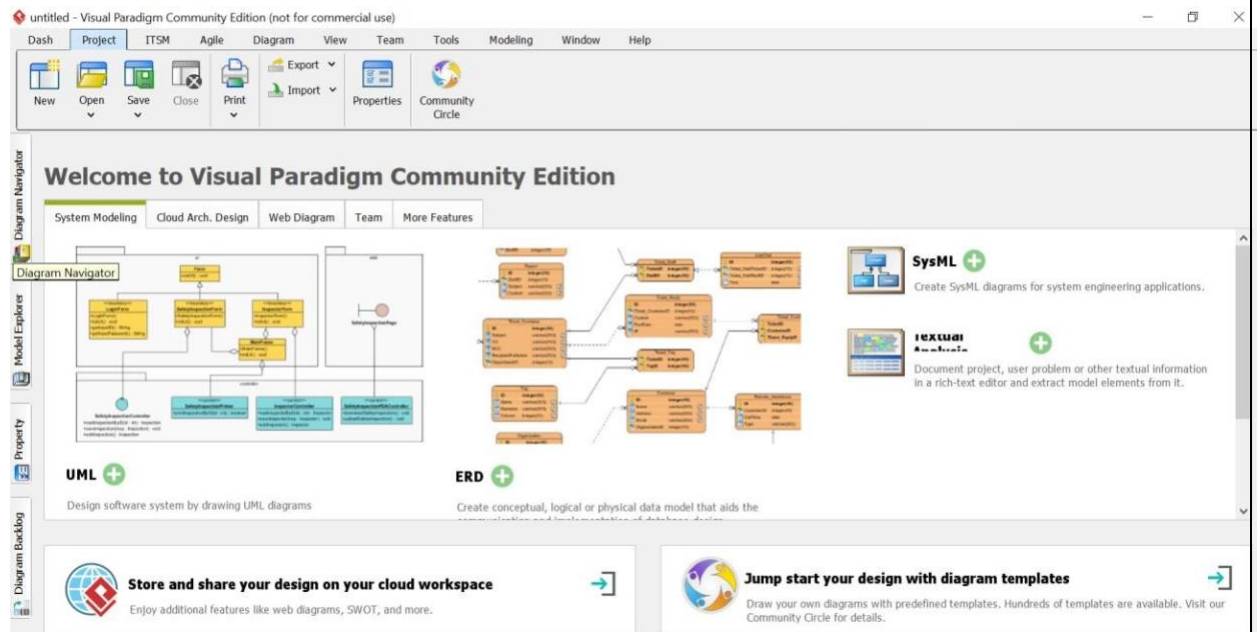
- A note (comment) gives the ability to attach various remarks to elements. A comment carries no semantic force, but may contain information that is useful to a modeler.



Walk Through Task

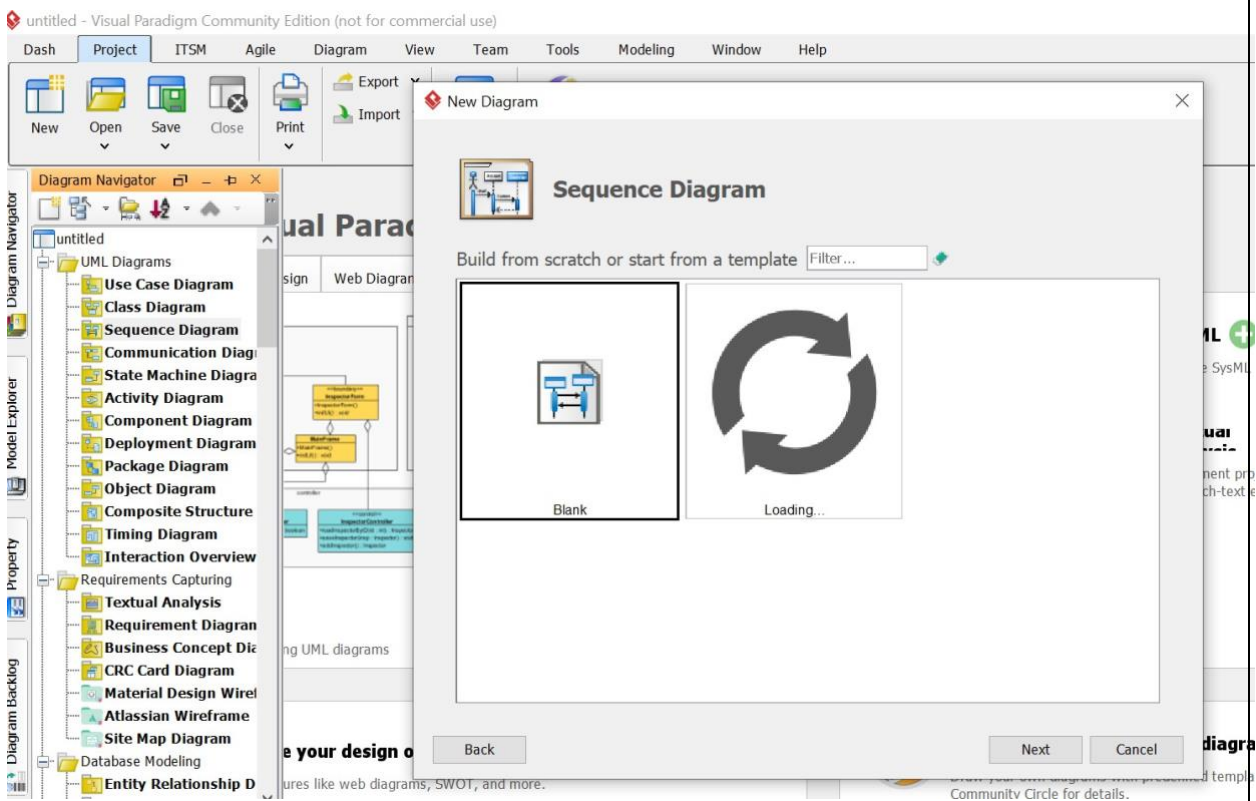
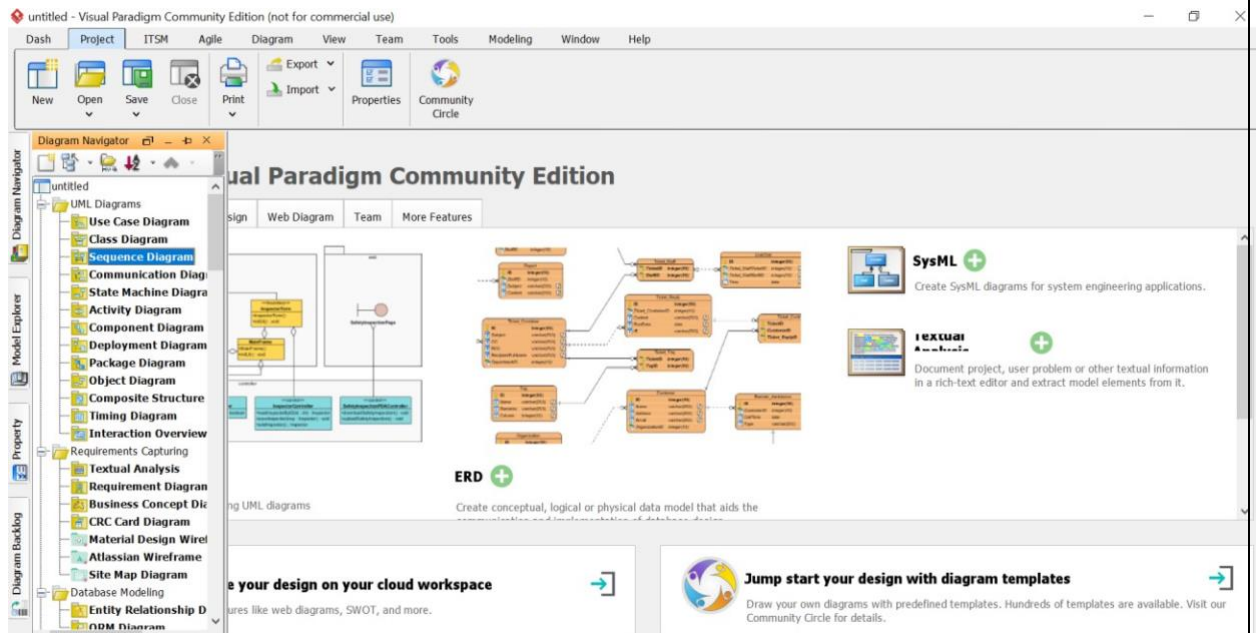
1. Open Visual Paradigm

- Launch Visual Paradigm on your computer.
- Create a new project or open an existing one.
-



2. Create a New Diagram

- Go to the Diagram Navigator (usually on the left side of the screen).
- Click on New Diagram or Create Diagram.
- In the New Diagram window, search for "Sequence Diagram" or "System Sequence Diagram".
- Select the appropriate template and click Next.



1. Open Visual Paradigm

- Launch Visual Paradigm and open your project or create a new one.

2. Create a New Sequence Diagram

- Go to the **Diagram Navigator**.
- Click on **New Diagram** or **Create Diagram**.
- Search for "Sequence Diagram" and select it.
- Click **Next** and name your diagram (e.g., "Login and Management System").

3. Define Actors and System

- Identify the actors and the system:

- **Actor:** Admin (the user interacting with the system).
- **System:** The management system handling hospital, patient, doctor, and appointment details.
- Drag and drop an **Actor** element from the toolbar onto the diagram and label it as "Admin."
- Drag and drop a **Lifeline** element to represent the "Management System."

4. Add Lifelines

- Add lifelines for the system components:
 - "Hospital Management"
 - "Patient Management"
 - "Doctor Management"
 - "Appointment Management"
- Drag and drop **Lifeline** elements for each component and label them accordingly.

5. Create Messages for Login

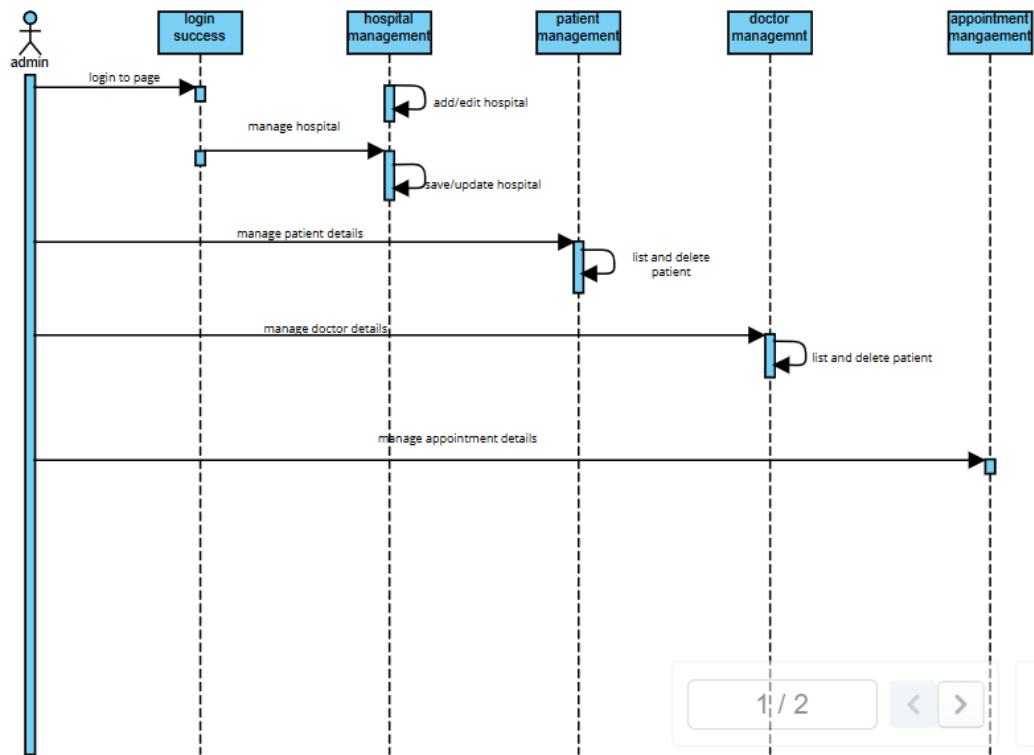
- Start with the login process:
 - Use the **Message** tool to draw an arrow from the "Admin" actor to the "Management System" lifeline.
 - Label the message as "Login Request."
 - Add a return message from the system to the actor labeled "Login Success."

6. Add Messages for Management Functions

- Add interactions for managing hospital, patient, doctor, and appointment details:
 - **Hospital Management:**
 - Draw a message from "Admin" to "Hospital Management" labeled "Add/Edit Hospital."
 - Add a return message labeled "Save/Update Hospital."
 - **Patient Management:**
 - Draw a message from "Admin" to "Patient Management" labeled "List and Delete Patient."
 - **Doctor Management:**
 - Draw a message from "Admin" to "Doctor Management" labeled "Manage Doctor Details."
 - **Appointment Management:**
 - Draw a message from "Admin" to "Appointment Management" labeled "Manage Appointment Details."

7. Add Activation Bars

- Add activation bars to show when each component is active:
 - Click on the **Activation** tool and place activation bars on the lifelines for "Hospital Management," "Patient Management," "Doctor Management," and "Appointment Management" during their respective interactions.



Practice Task

Scenario: Online Shopping System

An online shopping system allows users to browse products, add items to their cart, and place orders. The system also handles payment processing and order confirmation.

Actors

1. **Customer:** The user interacting with the system.
2. **Payment Gateway:** An external system for processing payments.

System Components

1. **Online Shopping System:** The main system handling product browsing, cart management, and order placement.
2. **Payment System:** Handles payment processing.

Steps for Creating the SSD

1. Open Visual Paradigm

- Launch Visual Paradigm and create a new project or open an existing one.

2. Create a New Sequence Diagram

- Go to the **Diagram Navigator**.
- Click on **New Diagram** and select **Sequence Diagram**.
- Name the diagram (e.g., "Online Shopping System SSD").

3. Add Actors and Lifelines

- Add the **Customer** actor and the **Payment Gateway** actor.

- Add lifelines for:
 - **Online Shopping System**
 - **Payment System**

4. Define Interactions

Here are the interactions to include in the SSD:

1. **Browse Products:**
 - The customer sends a "Browse Products" request to the Online Shopping System.
 - The system returns a "Display Product List" response.
2. **Add to Cart:**
 - The customer sends an "Add Item to Cart" request to the Online Shopping System.
 - The system updates the cart and returns a "Cart Updated" confirmation.
3. **Checkout:**
 - The customer sends a "Checkout" request to the Online Shopping System.
 - The system sends a "Process Payment" request to the Payment System.
 - The Payment System returns a "Payment Successful" response to the Online Shopping System.
 - The Online Shopping System sends an "Order Confirmed" response to the customer.

5. Draw Messages

- Use the **Message** tool to draw arrows between the lifelines to represent the interactions.
- Label each message with the appropriate action (e.g., "Browse Products," "Add Item to Cart," "Process Payment").

6. Add Activation Bars

- Add activation bars to show when each component is active during the interactions.

7. Refine the Diagram

- Adjust the layout for clarity.
- Add notes or comments if needed to explain specific interactions.

8. Save and Export

- Save the diagram within the project.
- Export the diagram as an image or PDF.

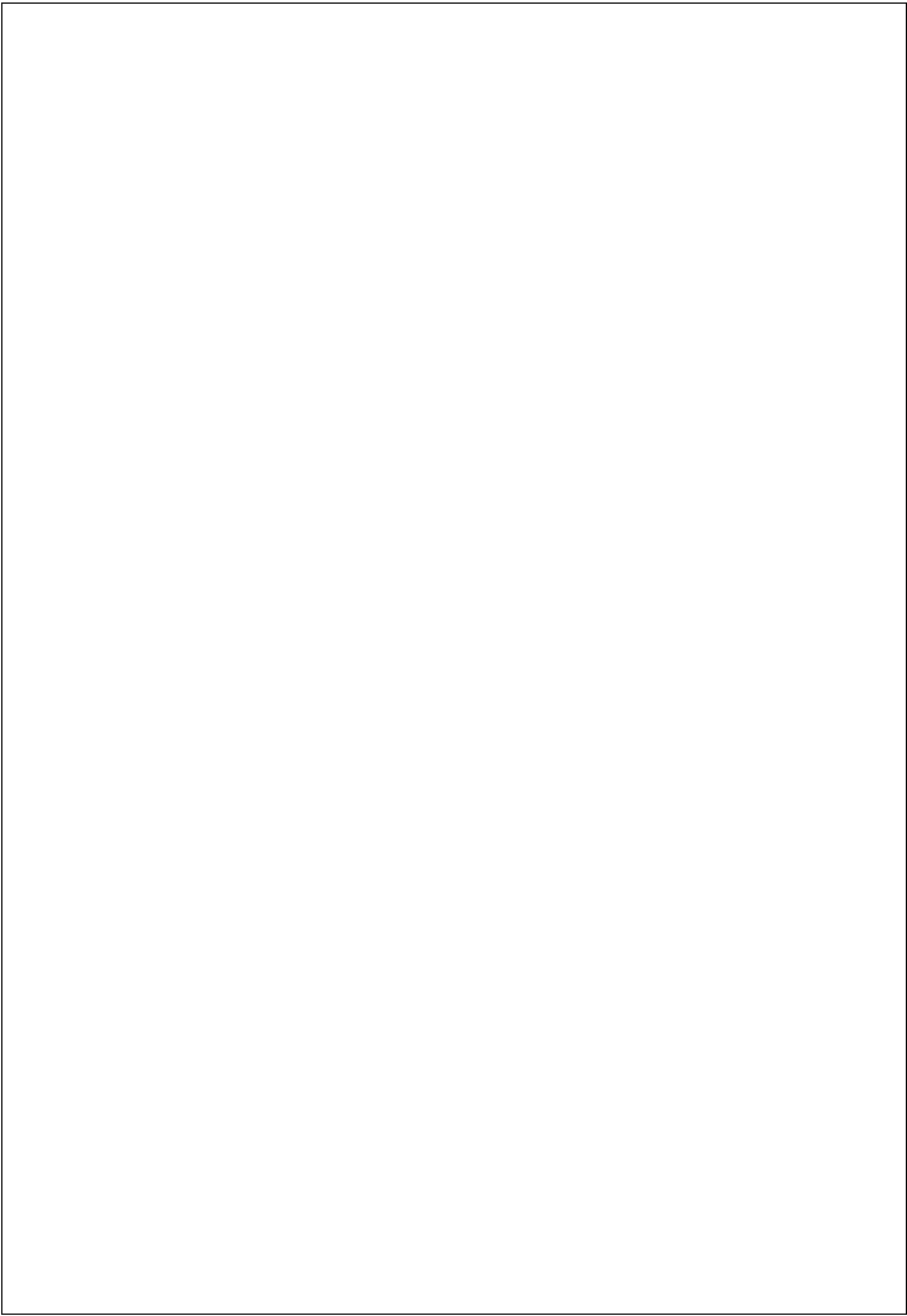
Example SSD Interactions

Here's how the interactions might look in the diagram:

1. **Customer → Online Shopping System:** Browse Products
Online Shopping System → Customer: Display Product List
2. **Customer → Online Shopping System:** Add Item to Cart
Online Shopping System → Customer: Cart Updated
3. **Customer → Online Shopping System:** Checkout
Online Shopping System → Payment System: Process Payment
Payment System → Online Shopping System: Payment Successful
Online Shopping System → Customer: Order Confirmed

Note:

- It includes multiple actors (Customer, Payment Gateway).
- It involves a sequence of interactions (browsing, adding to cart, checkout, payment).
- It demonstrates both system-to-user and system-to-system interactions.



Lab Manual for Software Requirement Engineering

Lab-16

Requirements Management

Lab 16: Lab Title: Managing Requirements

OBJECTIVE

1. Understand the importance of requirement management and traceability in software projects.
2. Learn how to use Jira for managing and tracing requirements.
3. Set up a project in Jira and create a traceability matrix.
4. Practice managing changes to requirements and maintaining traceability.

Relevant Lecture Material

a) Textbooks:

Managing Software Requirements: A Use Case Approach, Second Edition by Dean Leffingwell, Don Widrig, Addison Wesley

b) Reference Material:

Requirements Engineering: From System Goals to UML Models to Software Specifications, Axel V. Lamsweerde, (2009) ISBN- 10: 0470012706

5. **ACTIVITY TIME BOXING:**

ACTIVITY NAME	ACTIVITY TIME	TOTAL
Setting Up and Setting	30 min	30 min
Walkthrough Task	15 min	60 min
Practice Task	12 to 15 min for each task	30 min
Evaluation Task	10 min for all assign task	20 min

6. **CONCEPT MAP:**

Software Requirements Engineering Management is a critical aspect of software development that involves the systematic process of gathering, analyzing, documenting, validating, and managing the requirements of a software system. Effective management of software requirements ensures that the final product meets the needs of stakeholders and aligns with business objectives. Below is an overview of the key components and processes involved in software requirements engineering management.

2. Phases of Requirements Engineering

The requirements engineering process typically involves the following phases:

a. Requirements Elicitation

- Gathering requirements from stakeholders through interviews, workshops, surveys, and observation.
- Techniques: Brainstorming, use cases, user stories, prototyping.
- Challenges: Miscommunication, conflicting stakeholder needs, incomplete information.

b. Requirements Analysis

- Analyzing and refining the gathered requirements to ensure they are clear, complete, and consistent.
- Identifying dependencies, conflicts, and gaps in requirements.
- Techniques: SWOT analysis, gap analysis, feasibility studies.

c. Requirements Specification

- Documenting the requirements in a structured and detailed manner.
- Types of documentation:
 - **Software Requirements Specification (SRS):** A formal document detailing functional and non-functional requirements.
 - **User Stories:** Short descriptions of features from the user's perspective.
 - **Use Case Diagrams:** Visual representations of system interactions.

d. Requirements Validation

- Ensuring the requirements accurately reflect stakeholder needs and are feasible to implement.
- Techniques: Reviews, walkthroughs, prototyping, and testing.
- Common validation methods: Stakeholder sign-off, acceptance criteria.

e. Requirements Management

- Managing changes to requirements throughout the software development lifecycle.
- Ensuring traceability of requirements to design, development, and testing activities.

Tools and Resources

- **Tool:** Jira (Cloud or Desktop version)
- **Templates:** Traceability matrix template (can be created in Excel or Jira).
- **Sample Project:** Use a case study like an **Online Library Management System**.

Pre-Lab Preparation

1. **Install Jira:** Ensure Jira is installed and accessible.
2. **Familiarize Yourself with Jira:**

- Understand basic Jira concepts: Projects, Issues, Epics, User Stories, and Tasks.
- Learn about Jira workflows and boards.

Task 1: Set Up a Project in Jira

1. Create a New Project:

- Log in to Jira.
- Click on **Create Project** and select **Software Development**.
- Choose a project template (e.g., Scrum or Kanban).
- Name the project (e.g., "Online Bookstore System").

Pre-Lab Preparation

3. **Install Jira:** Ensure Jira is installed and accessible. If using the cloud version, create an account on [Atlassian](#).
4. **Familiarize Yourself with Jira:**
 - Understand basic Jira concepts: Projects, Issues, Epics, User Stories, and Tasks.
 - Learn about Jira workflows and boards.
5. **Case Study:** Prepare a list of sample requirements for a project (e.g., Online Bookstore System).

Lab Tasks

Task 1: Set Up a Project in Jira

2. Create a New Project:

- Log in to Jira.
- Click on **Create Project** and select **Software Development**.
- Choose a project template (e.g., Scrum or Kanban).
- Name the project (e.g., "Online Bookstore System").

2. Define Project Structure:

1. Create **Epics** to represent high-level requirements (e.g., "User Authentication," "Book Search," "Order Management").
2. Add **User Stories** under each Epic to break down requirements (e.g., "As a user, I want to log in using my email and password").
3. Add **Tasks** under User Stories for detailed implementation steps.

Task 2: Create and Manage Requirements

1. Add Requirements as Issues:

- Create issues for each requirement (e.g., "The system shall allow users to search for books by title, author, or category").
- Assign issues to team members and set priorities.

2. Link Requirements:

- Use Jira's **Issue Linking** feature to link related requirements (e.g., link a User Story to its corresponding Epic).

- Create dependencies between requirements (e.g., "User Authentication" must be completed before "Order Management").

Task 3: Set Up Traceability

1. Create a Traceability Matrix:

- Export the list of requirements (issues) from Jira to a spreadsheet.
- Create columns for Requirement ID, Description, Source (e.g., stakeholder), Priority, Status, and Linked Test Cases.
- Populate the matrix with the exported data.

2. Use Jira for Traceability:

- Use Jira's **Advanced Search (JQL)** to query and track requirements.
- Create a dashboard to visualize requirement status and traceability.

Task 4: Manage Changes to Requirements

1. Simulate a Change Request:

- Modify an existing requirement (e.g., change "Search by title" to "Search by title and ISBN").
- Update the requirement in Jira and document the change.

2. Track Impact of Changes:

- Use Jira's **Dependency Graph** to identify impacted requirements.
- Update the traceability matrix to reflect the changes.

Task 5: Validate and Verify Requirements

1. Link Requirements to Test Cases:

- Create test cases in Jira (or a linked test management tool like Zephyr).
- Link each test case to the corresponding requirement.

2. Verify Traceability:

- Ensure all requirements are linked to test cases and implementation tasks.
- Use Jira's reporting features to generate a traceability report.

Post-Lab Activities

1. Submit the Following:

- Screenshots of the Jira project setup (Epics, User Stories, and Tasks).
- Exported traceability matrix (spreadsheet or Jira report).
- A brief report on how changes to requirements were managed and traced.

2. Reflection:

- Discuss the challenges faced during the lab.
- Suggest improvements to the requirement management process.

Evaluation Criteria

- **Project Setup:** Proper creation of Epics, User Stories, and Tasks in Jira.

- **Traceability:** Accuracy and completeness of the traceability matrix.
- **Change Management:** Effective handling of requirement changes.
- **Documentation:** Clear and concise lab report.

Walkthrough task

Requirements Management Using JIRA

Background:

A mid-sized software development company, **TechSolutions Inc.**, is working on a new project to develop a **Customer Relationship Management (CRM)** system. The project involves multiple stakeholders, including business analysts, developers, testers, and product owners. The team is facing challenges in managing requirements effectively, leading to miscommunication, missed deadlines, and scope creep.

Problem Statement:

The team lacks a centralized system to manage requirements, track changes, and ensure alignment between stakeholders. Requirements are scattered across emails, documents, and spreadsheets, making it difficult to maintain traceability and prioritize tasks.

Solution:

The company decides to implement **JIRA** as a requirements management tool to streamline the process. Here's how they use JIRA to solve their requirements management challenges:

Step-by-Step Implementation in JIRA:

1. Setting Up the Project

- The team creates a new project in JIRA called "**CRM System Development**."
- They choose a **Scrum** template to manage the project in an Agile framework.
- The project is configured with the following issue types:
 - **Epic:** High-level requirements or features (e.g., "User Management").
 - **Story:** Detailed requirements (e.g., "As a user, I want to log in to the system").
 - **Task:** Subtasks or activities to complete a story.
 - **Bug:** Issues identified during testing.

2. Capturing Requirements

- Business analysts create **Epics** to represent high-level features of the CRM system.
 - Example Epics:
 - "User Authentication"
 - "Customer Data Management"
 - "Reporting and Analytics"
- Each Epic is broken down into **Stories** that describe specific functionalities.
 - Example Stories:

- "As an admin, I want to reset user passwords."
- "As a sales rep, I want to view customer contact details."

3. Prioritizing Requirements

- The product owner prioritizes the Epics and Stories using the **Backlog** feature in JIRA.
- Stories are assigned a priority level (e.g., High, Medium, Low) and ranked in the backlog.
- The team uses **Sprints** to work on high-priority requirements in time-boxed iterations.

4. Traceability and Linking

- The team establishes traceability by linking Stories to Epics using JIRA's **Issue Linking** feature.
- Test cases are created in JIRA and linked to Stories to ensure that each requirement is validated.
- Dependencies between Stories are identified and linked using the **Blocks/Is Blocked By** relationship.

5. Collaboration and Feedback

- Stakeholders are added to JIRA as users, and their roles are defined (e.g., Reporter, Developer, Tester).
- Comments and attachments are used to provide feedback on requirements directly in JIRA.
- The team holds regular **Backlog Refinement** meetings to review and update requirements.

6. Tracking Progress

- The team uses JIRA's **Dashboards** and **Reports** to track progress:
 - **Burndown Chart**: Tracks the completion of Stories within a Sprint.
 - **Velocity Chart**: Measures the team's capacity to deliver Stories over time.
 - **Requirement Traceability Matrix**: Ensures all requirements are covered by test cases.
- The product owner monitors the **Release Burndown** to ensure the project stays on track.

7. Handling Changes

- When new requirements or changes are requested, the product owner creates a new Story or updates an existing one.
- The team uses JIRA's **Versioning** feature to manage releases and ensure that changes are included in the correct release cycle.
- Changes are documented in the **Comments** section, and stakeholders are notified via JIRA notifications.

8. Testing and Validation

- Testers create **Test Cases** in JIRA and link them to the corresponding Stories.
- Bugs identified during testing are logged as issues and linked to the relevant Story.
- The team uses JIRA's **Automation** feature to move Stories to the "Done" status only when

all linked test cases pass.

Results:

By implementing JIRA for requirements management, **TechSolutions Inc.** achieves the following outcomes:

1. **Centralized Requirements:** All requirements are stored in one place, eliminating the need for scattered documents.
2. **Improved Traceability:** Links between Epics, Stories, and Test Cases ensure full traceability.
3. **Better Collaboration:** Stakeholders can provide feedback and track progress in real-time.
4. **Increased Transparency:** Dashboards and reports provide visibility into project status.
5. **Efficient Change Management:** Changes are documented and tracked, reducing scope creep.

Key Takeaways:

- JIRA is a powerful tool for managing requirements in Agile projects.
- Proper configuration of Epics, Stories, and Test Cases ensures traceability and alignment.
- Regular backlog refinement and sprint planning are essential for prioritizing requirements.
- Dashboards and reports provide valuable insights into project progress and team performance.

Practice task:

Company: ShopEasyInc. A mid-sized e-commerce company)

Project: Development of a **next-generation e-commerce platform** with enhanced features such as AI-powered recommendations, multi-vendor support, and seamless checkout.

Challenges in Requirements Management

- **Scattered requirements** across emails, spreadsheets, and documents.
- **Lack of traceability** between business needs and development tasks.
- **Frequent requirement changes** leading to scope creep.
- **Poor collaboration** between stakeholders (business analysts, developers, testers).

Solution: You have to implement **JIRA** as a requirements management tool to streamline all these challenges.